



INSI UNIVERSITY

**INSTITUT NATIONAL SUPERIEUR
D'INFORMATIQUE**

**MEMOIRE DE PROJET EN VUE DE LA VALIDATION DE LA
MASTER 2**

Domaine : Science et technologie

Mention : Informatique

Parcours : MSI

Présenté Par : Monsieur RATOLOJANAHARY Tina Hubert

**RESTRUCTURATION DU BACKEND D'UNE PLATEFORME
E-COMMERCE : TRANSITION D'UN MONOLITHE VERS UNE
ARCHITECTURE MICROSERVICES**

Soutenue le septembre devant la Commission d'Examen composée de :

Président du Jury : -

Examineur : -

Encadreur : -

Année universitaire : 2024–2025

REMERCIEMENTS

Avant d’entamer ce mémoire, nous souhaitons exprimer nos sincères remerciements à toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce travail et à l’aboutissement de ce projet.

- Au Seigneur Tout-Puissant, pour la force, la santé et l’inspiration qu’Il nous a accordées durant ce parcours universitaire et pour avoir rendu possible l’achèvement de ce mémoire.
- À Docteur RAMASONDRANO Andriamanjaka, directeur de l’Institut National Supérieur d’Informatique (INSI), en reconnaissance de l’environnement d’apprentissage offert et du soutien institutionnel apporté tout au long de nos études.
- À Docteur ...notre encadreur pédagogique, pour son accompagnement, ses conseils constructifs et son implication précieuse dans la conception et la finalisation de ce mémoire.
- Aux enseignants de la filière genie logiciel (GL), pour les connaissances théoriques et pratiques transmises durant cette deuxième année de licence, qui ont jeté les bases nécessaires à ce projet.
- À nos famille et à nos proches, pour leur soutien moral et leurs encouragements constants, qui ont été une source de motivation essentielle durant tout ce parcours.
- À nos camarades de promotion, pour l’entraide et la solidarité partagées tout au long de nos études.
- Enfin, à toutes les personnes qui, de près ou de loin, ont apporté leur aide dans la réalisation de ce mémoire et la conception du projet Jereo.

SOMMAIRE

Table des matières

REMERCIEMENTS

LISTE DES ABRÉVIATIONS, SIGLES ET ACRONYMES

GLOSSAIRE

LISTE DES TABLEAUX

LISTE DES FIGURES

INTRODUCTION

CHAPITRE I - CONTEXTE GÉNÉRAL DU PROJET

- I.1. Présentation de l'INSI
- I.2. Contexte professionnel et entreprise d'accueil
- I.3 Problématique et justification du projet
- I.4 Objectifs et périmètre du projet
- I.5 Méthodologie et plan de travail

CHAPITRE II - ANALYSE ET CONCEPTION

- II.1 Analyse de l'existant et État de l'Art
- II.2 Fondements Théoriques et Contraintes Distribuées
- II.3 Découpage Métier : Le Domain-Driven Design (DDD)
- II.4 Modélisation des Microservices et Communications
- II.5 Architecture de Persistance et Base de Données
- II.6 Modélisation de la Sécurité : Zero Trust Architecture
- II.7 Modélisation des Flux et Cas d'Usage

CHAPITRE III - RÉALISATION

- III.1 Organisation du Projet et Arborescence
- III.2 Environnement Technologique et Outils
- III.3 Architecture Interne et Patrons de Conception
 - III.3.3 Tableau Récapitulatif des Microservices
- III.4 Développement de l'API Gateway
- III.5 Implémentation des Microservices Core
- III.6 Sécurisation et Gestion des Accès
- III.7 Orchestration et DevOps
- III.8 Observabilité et Traçabilité Distribuée

III.9 Stratégie de Test et Validation

III.10 Visualisation de l'Interface Utilisateur (Frontend)

CHAPITRE IV - RÉSULTATS ET ANALYSE

IV.1 Évaluation des performances et de la scalabilité

IV.2 Analyse de la qualité logicielle et maintenabilité

IV.3 Bilan critique et limites de la solution

IV.4 Validation fonctionnelle via Postman

IV.5 Perspectives d'évolutions et améliorations futures

CONCLUSION GÉNÉRALE

RÉFÉRENCES BIBLIOGRAPHIQUES

WÉBOGRAPHIE

ANNEXES

Annexe A — Documentation de l'API REST

RÉSUMÉ

ABSTRACT

LISTE DES ABRÉVIATIONS, SIGLES ET ACRONYMES

ACID	Atomicité, Consistance, Isolation, Durabilité
API	Application Programming Interface
ARSB	Administration Réseaux, Systèmes et Bases de données
AWS	Amazon Web Services
BASE	Basically Available, Soft state, Eventually consistent
BFF	Backend-for-Frontend
CAP	Consistency, Availability, Partition tolerance
CRUD	Create, Read, Update, Delete
DDD	Domain-Driven Design
DTO	Data Transfer Object
DX	Developer Experience
E2E	End-to-End
GL	Génie Logiciel
HTTP	Hypertext Transfer Protocol
I2AD	Intelligence Artificielle, Analyse de Données et Automatisation
IAD	Intelligence Artificielle et Data Science
IC	Infrastructures et Cybersécurités
INSI	Institut National Supérieur d'Informatique
JSON	JavaScript Object Notation
JWT	JSON Web Token
LMD	Licence Master Doctorat
MQ	Message Queue
MSI	Management des Systèmes d'Information
NoSQL	Not only SQL
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
REST	Representational State Transfer
RMQ	RabbitMQ
RPC	Remote Procedure Call
RS256	RSA Signature avec SHA-256
S3	Simple Storage Service (AWS)
SI	Systèmes d'Information
SPOF	Single Point of Failure
SQL	Structured Query Language
TALN	Traitement Automatique du Langage Naturel
TLS	Transport Layer Security

UML	Unified Modeling Language
URL	Uniform Resource Locator

GLOSSAIRE

Terme	Définition
Microservices	Architecture logicielle où une application est décomposée en petits services indépendants, chacun responsable d'une fonctionnalité spécifique.
Monolithe	Architecture logicielle où toutes les fonctionnalités d'une application sont regroupées dans une seule et même base de code.
API Gateway	Composant qui agit comme un point d'entrée unique pour les clients, gérant les requêtes vers les différents microservices.
Containerisation	Technique de virtualisation légère qui permet d'exécuter des applications dans des environnements isolés appelés conteneurs.
Orchestration	Gestion automatisée du déploiement, de la mise à l'échelle et de la gestion des conteneurs.
Scalabilité	Capacité d'un système à gérer une augmentation de la charge en ajoutant des ressources.

LISTE DES TABLEAUX

1	Les parcours en Licence et Master	6
2	Comparaison entre l'architecture monolithique et microservices	16
3	Comparaison des systèmes de persistance retenus	23
4	Flux d'événements dans la Saga de commande	26
5	Récapitulatif des microservices de la plateforme Techcare	36
6	Référence synthétique des endpoints de l'API Gateway Techcare	39
7	Comparaison de la complexité cyclomatique : Monolithe vs. Microservices (Lizard) . . .	60
8	Bilan quantitatif des performances et de la structure du code	62
9	Endpoints du module Authentification	xii
10	Endpoints du module Produits	xiii
11	Endpoints du module Catégories	xiii
12	Endpoints du module Inventaire	xiii
13	Endpoints du module Panier	xiv
14	Endpoints du module Commandes	xiv
15	Endpoints du module Paiement	xv
16	Endpoints du module Fichiers	xv
17	Codes d'erreur HTTP retournés par l'API Gateway	xvi

LISTE DES FIGURES

1	Organigramme de l'INSI	3
2	Diagramme de cas d'utilisation du système Techcare	13
3	Architecture du module NestJS	17
4	Diagramme d'interaction et de communication des microservices (Chorégraphie)	21
5	Diagramme de classes global de l'architecture microservices	22
6	Diagramme de séquence de l'authentification utilisateur	28
7	Diagramme d'activité du processus de commande	30
8	Diagramme de séquence du pattern Saga avec compensation	31
9	Structure globale (Monorepo)	33
10	Structure d'un service	33
11	Visualisation de l'architecture des conteneurs Docker en cours d'exécution	48
12	Interface de connexion et d'accès sécurisé (Client)	50
13	Interface de catalogue de médicaments (Client)	51
14	Tableau de bord administrateur : Statistiques et pilotage en temps réel	52
15	Interface de gestion des produits et mise en ligne du catalogue (Admin)	53
16	Rapports de performance Artillery : Comparaison du temps de réponse médian	57
17	Rapports de performance Artillery : Latence de queue sous charge extrême	57
18	Logs d'exécution : Temps de démarrage avec cache Docker pour le Monolithe (1,280 s). Montre la rapidité de lancement du conteneur unique.	58
19	Logs d'exécution : Temps de démarrage avec cache Docker pour les Microservices (19,671 s). Illustre le surcoût temporel lié à l'orchestration distribuée.	59
20	Résultats cloc : Charge cognitive du Monolithe avec une base de code unique (8 087 lignes).	60
21	Résultats cloc : Charge cognitive réduite pour un service isolé comme le Microservice Produit (716 lignes).	60
22	Traçabilité d'une défaillance MongoDB via Jaeger : localisation immédiate du composant défaillant	61
23	Capture d'écran Postman : Test de réussite de l'authentification	64
24	Capture d'écran Postman : Test de refus d'accès (401 Unauthorized)	64

25	Capture d'écran Postman : Récupération de la liste des produits	65
26	Capture d'écran Postman : Création réussie d'un produit (201 Created)	66
27	Interface RabbitMQ : Pic d'activité sur la file product_manager_queue	67
28	Capture d'écran Postman : Rejet d'une requête mal formée (400 Bad Request)	68
29	Interface S3 : Validation de l'upload de l'image du produit	69

INTRODUCTION

Depuis la création de l'internet, le commerce en ligne a connu une forte croissance et participe désormais de manière significative à l'économie mondiale. Les plateformes e-commerce traditionnelles reposent souvent sur des architectures monolithiques, où toutes les fonctionnalités sont regroupées dans une seule application.

Bien que cette approche ait permis un développement rapide, elle présente plusieurs limitations, notamment en termes de scalabilité, de maintenabilité et d'agilité de développement. C'est pourquoi de nombreuses entreprises se tournent vers des architectures basées sur des microservices, qui offrent une plus grande flexibilité et une meilleure capacité à s'adapter aux évolutions du marché. Cette transition vers les microservices nécessite une réflexion approfondie sur l'architecture logicielle, les choix technologiques et les processus de développement.

De cette analyse, nous avons dégagé une problématique centrale : comment concevoir une plateforme e-commerce basée sur des microservices tout en garantissant une expérience utilisateur fluide et cohérente ?

Pour répondre à cette problématique, ce mémoire s'intitule : « RESTRUCTURATION DU BACKEND D'UNE PLATEFORME E-COMMERCE : TRANSITION D'UN MONOLITHE VERS UNE ARCHITECTURE MICROSERVICES ». Son objectif est de migrer une architecture monolithique existante vers une architecture basée sur des microservices en utilisant des méthodologies et des techniques modernes de développement logiciel. Nous allons explorer les différentes étapes de cette migration, en mettant l'accent sur les aspects techniques, organisationnels et stratégiques. Nous aborderons également les défis rencontrés lors de cette transition et les solutions mises en place pour les surmonter.

Ce mémoire est structuré en plusieurs chapitres. Le premier chapitre présente le contexte tel que la présentation des parties prenantes et le cadre du projet. Le second chapitre analyse l'architecture existante et la conception de l'architecture cible. Le troisième chapitre détaille le processus de migration, les outils utilisés et les bonnes pratiques adoptées. Enfin, le dernier chapitre évalue les résultats obtenus et propose des pistes d'amélioration pour l'avenir.

CHAPITRE I

CONTEXTE GÉNÉRAL DU PROJET

I.1. Présentation de l'INSI

I.1.1. Information d'ordre général

INSI (Institut National Supérieur d'Informatique) est un institut universitaire situé à Antananarivo, spécialisé dans le domaine de l'informatique. Il accueille principalement des bacheliers ayant choisi cette filière, mais reste également ouvert à toute personne manifestant un intérêt particulier pour les métiers du numérique, à travers son centre de formation professionnelle. L'établissement a été fondé le 17 août 2021 et son siège se trouve à Ankazotokana Ambanidia, Lot VF 32 Ter.

Pour tout renseignement, vous pouvez nous contacter par email à contact@insi.mg ou visiter notre site web : www.insi.mg

INSI University bénéficie du soutien d'entreprises partenaires, dont **Ilo Madagascar SARLU** et **Data Pulse Center**, qui sont également à l'origine de sa création.

I.1.2. Missions et historiques

INSI University a été fondée le 17 août 2021 à Antananarivo, dans un contexte marqué par une forte demande de compétences dans le domaine de l'informatique et des technologies numériques.

Créé à l'initiative de **Ilo Madagascar SARLU** et **Data Pulse Center**, l'Institut a vu le jour avec pour ambition de former une nouvelle génération de professionnels qualifiés, capables de répondre aux besoins du marché local et international.

Depuis sa création, INSI ne cesse d'évoluer en adaptant ses formations aux enjeux technologiques actuels, tout en favorisant un apprentissage pratique, orienté vers les réalités du monde professionnel.

La mission principale de l'INSI est de **former des experts compétents, éthiques et innovants dans le domaine de l'informatique**, capables de contribuer activement au développement économique et numérique de Madagascar.

L'Institut s'engage à :

- Offrir des formations de qualité, à jour des évolutions technologiques,
- Favoriser l'insertion professionnelle de ses apprenants,
- Créer un lien fort entre la formation académique et les besoins réels des entreprises,
- Encourager l'entrepreneuriat et l'innovation technologique,
- Contribuer à la démocratisation de l'accès au savoir numérique.

I.1.3. Organigramme de l'INSI

L'organigramme ci-dessous présente la structure organisationnelle de l'INSI, mettant en évidence les différentes fonctions et responsabilités au sein de l'institut, ainsi que les liens hiérarchiques entre les divers services.



ORGANIGRAMME INSI

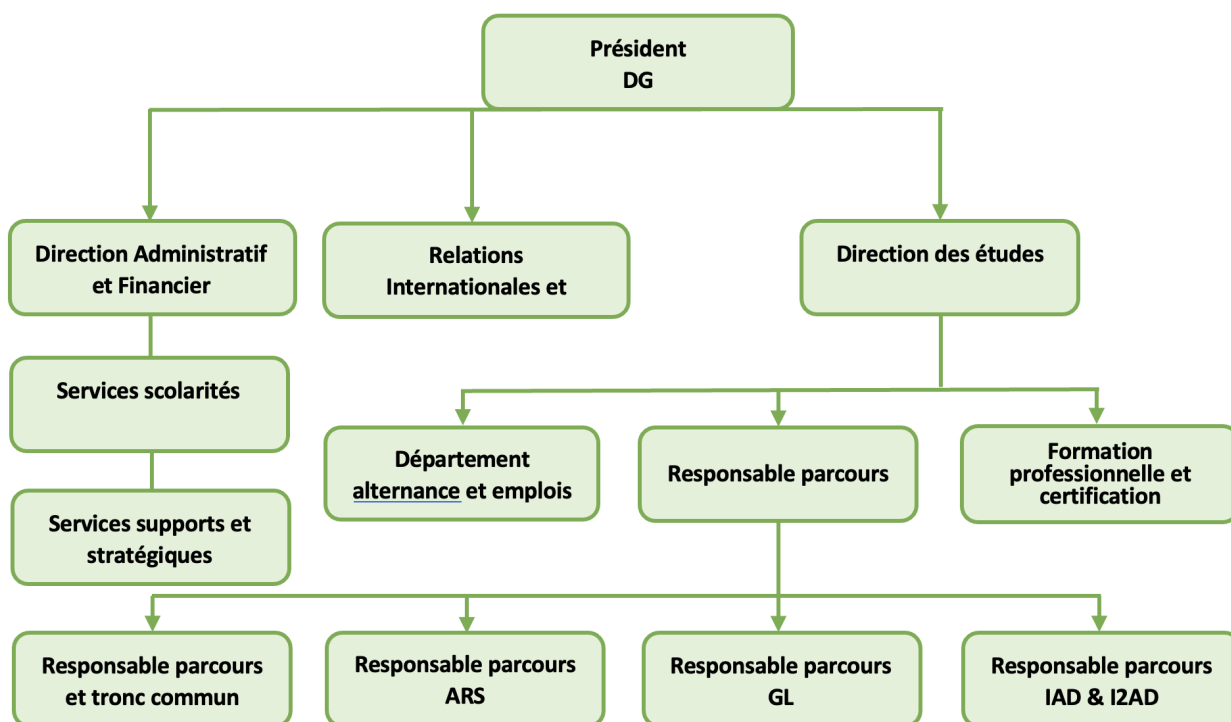


FIGURE 1 – Organigramme de l'INSI

(Source : INSI, 2025)

L'organigramme de l'INSI, en partenariat avec le Collège de Paris, est structuré autour d'une direction générale incarnée par le Président Directeur Général. Ce dernier assure la coordination de trois pôles principaux : la **Direction Administrative et Financière**, la **Direction des Études**, et le service des **Relations Internationales et Mobilités**.

Le **Président Directeur Général** assure la direction stratégique de l'institut. Il supervise l'ensemble des activités pédagogiques, administratives et de développement.

Il veille à la qualité de l'enseignement, à la bonne gestion des ressources humaines et matérielles, et représente l'établissement auprès des partenaires et autorités.

La **Direction Administrative et Financière** est chargée de la gestion des affaires internes, notamment

à travers deux services essentiels : le **service des scolarités**, qui s'occupe des dossiers étudiants, inscriptions et suivi académique, et les **services supports et stratégiques**, qui assurent un appui logistique, administratif et technologique à l'ensemble de l'institut.

Le pôle des **Relations Internationales et Mobilités** gère les échanges académiques avec l'étranger ainsi que les projets de mobilité étudiante ou enseignante.

La **Direction des Études** supervise quant à elle l'ensemble des programmes pédagogiques et le corps enseignant. Elle travaille en étroite collaboration avec un **Responsable Parcours**, qui coordonne les différentes spécialisations proposées. Cette direction intègre également une cellule dédiée à la **formation professionnelle et à la certification**, en charge des dispositifs de validation des compétences, des certifications professionnelles et des parcours de formation continue. Il comprend également le **Département alternance et emplois**, responsable du suivi des étudiants en contrat d'alternance, des stages professionnels, et de l'insertion professionnelle.

Sous la partie **Responsable Parcours**, plusieurs responsables pédagogiques gèrent les différentes filières de formation : le **parcours tronc commun**, qui regroupe les matières fondamentales partagées par tous les étudiants en début de cycle ; le **parcours ARSB & IC** ; le **parcours GL & MSI** ; et enfin le **parcours IAD & IAD**.

D'autres structures, bien que non représentés sur cet organigramme, contribuent également de manière significative à la mission de l'INSI.

Le **Conseil de l'École**, il regroupe des membres de l'équipe pédagogique et administrative. Il joue un rôle consultatif ou décisionnel sur les questions académiques : validation des programmes, suivi des résultats, discipline, méthodologie, etc.

Le **responsable de la formation en ligne**, il conçoit et supervise les dispositifs d'apprentissage à distance. Il s'assure que les contenus en ligne sont accessibles, interactifs et conformes aux standards pédagogiques, tout en accompagnant enseignants et étudiants dans leur utilisation.

L'**Association et Clubs étudiants**, encadré et soutient les initiatives étudiantes, les clubs thématiques (informatique, innovation, sport, culture, etc.), et les événements organisés dans le cadre de la vie étudiante. Cela favorise l'engagement, le développement personnel et les compétences transversales des étudiants.

Cette structure témoigne d'une organisation à la fois pédagogique, administrative et tournée vers l'international et l'employabilité, assurant une formation complète et professionnalisante aux étudiants.

I.1.4. Domaine de spécialisation

INSI University propose des **parcours académiques professionnalisants** dans le domaine de l'informatique et des technologies numériques, articulés autour de licences professionnelles, de masters spécialisés, formations professionnelles continues et d'opportunités de mobilité internationale.

a) Licences professionnelles

Les programmes de licence offrent une formation de base solide, axée sur la pratique, et préparant à une insertion rapide dans le monde professionnel :

- **GL (Génie Logiciel)** : Conception, développement et maintenance d'applications logicielles.
- **ARSB (Administration Réseaux et Systèmes et Base de données)** : Gestion d'infrastructures informatiques, réseaux, serveurs et systèmes.
- **IAD (Intelligence Artificielle et Data Science)** : Introduction aux algorithmes d'IA, au traitement de données et à l'apprentissage automatique.

b) Masters professionnels

Les masters permettent d'approfondir les compétences techniques et managériales, en lien étroit avec les évolutions du secteur :

- **MSI (Management des Systèmes d'Information)** : Pilotage de projets numériques, gouvernance des SI.
- **IC (Infrastructures et Cybersécurité)** : Sécurisation des systèmes, audit, réseaux avancés.
- **I2AD (Intelligence Artificielle, Analyse de Données et Automatisation)** : Applications avancées de l'IA, automatisation des processus et science des données.

c) Formations professionnelles

INSI University propose également des formations professionnelles continues, spécifiquement conçues pour les professionnels déjà en poste souhaitant approfondir leurs compétences ou se spécialiser dans l'un des domaines proposés. Ces formations sont adaptées aux contraintes des professionnels (horaires flexibles, modules pratiques) et visent à renforcer leur employabilité ou à accompagner leur évolution de carrière.

d) Mobilité internationale & double diplôme

Dans une optique d'ouverture et de reconnaissance internationale, INSI propose à ses étudiants des opportunités de mobilité académique et de double diplôme, à partir de la deuxième année (L2).

- **Format hybride** : 6 mois de formation à Madagascar + 6 mois à l'étranger selon le parcours choisi.
- **Double diplôme en L3 ou M2** : En partenariat avec des institutions internationales telles que :
 - Collège de Paris (France)
 - Keyce Informatique (France)
 - Eskills (Québec, Canada)

Conditions d'éligibilité

- Niveau DALF C1 en français pour la mobilité vers des pays francophones.
- Niveau B2 ou C1 en anglais pour les destinations anglophones.

I.1.5 Architectures des formations pédagogiques

Au sein de l'INSI, une seule mention est proposée : Informatique, déclinée en trois parcours distincts aux niveaux Licence et Master :

Licence	Master
Génie Logiciel (GL)	Management des Systèmes d'Information (MSI)
Administration Réseaux, Systèmes et Bases de données (ARSB)	Infrastructures et Cybersécurité (IC)
Intelligence Artificielle et Data Science (IAD)	Intelligence Artificielle, Analyse de Données et Automatisation (IADA)

TABLE 1 – Les parcours en Licence et Master
(Source : INSI,2025)

L'architecture des études à trois niveaux conformément au système LMD (Licence Master et Doctorat) permet les comparaisons et les équivalences académiques des diplômes au niveau international.

L = Licence (Bac + 3) = L1, L2, L3 = 6 semestres S1 à S6

M = Master (Bac + 5) = M1, M2 = 4 semestres S7 à S10

Le diplôme de licence est obtenu en 3 années des études après Baccalauréat. Et le diplôme de Master est obtenu en 2 ans après obtenu du diplôme de LICENCE.

Le MASTER PROFESSIONNEL est un diplôme destiné à la recherche emploi au terme des études.

Spécificités pédagogiques

- École d'alternance dès la L2 S4 : alternance cours/entreprise pour une meilleure insertion
- Écoles connectées & e-learning : plateforme numérique de formation, cloud intégré, Google Workspace. Cours accessibles en ligne selon les parcours
- Pédagogie active & par projets : hackathons, soutenances exposées, projets réels en entreprise

I.1.6. Relations de l'INSI avec les entreprises et les organismes

L'Institut National Supérieur en Informatique (INSI) entretient des liens étroits avec le monde professionnel. L'INSI collabore activement avec de nombreux organismes publics, semi-publics et privés, tant au niveau national qu'international. Cette collaboration se traduit concrètement par l'organisation régulière de stages obligatoires pour les étudiants, favorisant leur immersion professionnelle et leur employabilité.

Parmi les entreprises et institutions partenaires accueillant nos étudiants en stage chaque année, on peut citer : Data Pulse Center, RELIA Consulting, Code Talent, Bocasay, Minimad, Smartone AI, NOVITY, Eclosia Madagascar, VISEO Groupe, Océane Trade, ACCES Banque,

I.1.7. Partenariat au niveau international

L'INSI s'est inscrit dans une dynamique d'ouverture à l'international grâce à son partenaire fondateur, **ILO Madagascar**, représentant de l'Organisation Internationale du Travail. Cette orientation se renforce aujourd'hui à travers des **partenariats stratégiques avec des institutions académiques et des**

entreprises internationales, permettant à l’Institut de proposer une formation alignée sur les standards mondiaux.

L’INSI bénéficie actuellement de **programmes de co-diplomation et de double diplôme** en collaboration avec plusieurs établissements internationaux de renom, tels que :

- **Collège de Paris International**
- **Keyce Informatique** (France)
- **Eskills Academy** (Québec, Canada)
- **École Supérieure Polytechnique d’Amistranana (ESPA)** – pour une synergie régionale renforcée.

Ces partenariats permettent non seulement d’enrichir l’offre pédagogique, mais aussi d’ouvrir des perspectives de **mobilité académique et professionnelle** aux étudiants, en leur offrant une reconnaissance de leurs compétences à l’échelle internationale.

Par ailleurs, à travers les **stages annuels et projets pédagogiques**, l’INSI collabore également avec des entreprises à dimension régionale ou mondiale, telles que :

- **WISEO Groupe**
- **Edoisia Madagascar** (Groupe mauricien)
- **Smartone A1**
- **NOVITY**, entre autres.

Ces relations actives témoignent de l’ancrage de l’INSI dans un **écosystème global**, plaçant l’Institut comme un acteur majeur de la formation en informatique à Madagascar, tourné vers l’innovation et les exigences du marché international.

I.1.8. Parcours professionnels des diplômés

Les diplômés de la Licence en Informatique de l’INSI disposent d’un socle solide de compétences en programmation, systèmes, réseaux, bases de données et développement logiciel. Cette polyvalence leur permet d’intégrer directement le marché du travail ou de poursuivre en Master. À ce niveau, ils peuvent exercer en tant que :

- **Technicien système et réseau**
- **Développeur web / mobile (front-end ou back-end)**
- **Administrateur système junior**
- **Technicien support IT**
- **Intégrateur / testeur logiciel**
- **Assistant en sécurité informatique**

— **Assistant en projets d’IA ou de data (selon profil)**

Les stages annuels en entreprise leur offrent une première expérience professionnelle valorisante dans des structures variées : entreprises technologiques, banques, administrations, startups, ONG, etc.

La formation de Master proposée à l’INSI, grâce à ses parcours spécialisés, forme des professionnels capables d’intervenir sur des projets complexes, innovants et à dimension stratégique. Les débouchés varient selon la spécialisation choisie :

Parcours Ingénierie logicielle, réseaux et systèmes :

- **Ingénieur en développement logiciel / full-stack**
- **Architecte logiciel**
- **Ingénieur système et réseaux**
- **Administrateur cloud / DevOps**
- **Chef de projet informatique**

Parcours Cybersécurité :

- **Expert en sécurité des systèmes d’information**
- **Analyste SOC / CERT**
- **Auditeur en cybersécurité**
- **Responsable de la sécurité IT (RSSI)**

Parcours Intelligence Artificielle et Data :

- **Data Scientist**
- **Data Analyst**
- **Ingénieur en Intelligence Artificielle**
- **Machine Learning Engineer**
- **Développeur d’applications intelligentes**
- **Consultant en IA**
- **Spécialiste en traitement automatique du langage naturel (TALN)**
- **Spécialiste en vision par ordinateur (Computer Vision)**

Grâce aux partenariats internationaux et aux projets en entreprise, les diplômés du Master peuvent également prétendre à des postes dans des structures multinationales, ou poursuivre une carrière académique et de recherche (doctorat, enseignement supérieur).

I.1.9. Ressources humaines

L'INSI s'appuie sur une équipe pédagogique et administrative compétente, dynamique et engagée dans la réussite de ses étudiants. Ses ressources humaines se répartissent comme suit :

- Président Directeur de l'INSI : Dr RAMASONDRANO Andriamanjaka
- Responsable parcours ARSB & IC : Mr RATIANARISON Hery Mampionona
- Responsable parcours tronc commun : Dr RAKOTONDRAMANANA R, Sitraka
- Responsable parcours GL & MSI : Mr ANDRIAMIADANA TANTELY ANTOINE
- Responsable parcours IAD & I2AD : par intérim PDG
- Nombre des enseignants : 40
- Nombre des personnels administratives : 04
- Personnels d'appui : 6

I.2. Contexte professionnel et entreprise d'accueil

I.2.1. Présentation de l'entreprise d'accueil

L'entreprise d'accueil de ce projet est **Techcare**, une startup technologique spécialisée dans la vente de produits pharmaceutiques en ligne à Madagascar.

Fondée en 2024, Techcare vise à révolutionner l'accès aux médicaments et aux produits de santé en offrant une plateforme e-commerce conviviale et fiable.

Techcare collabore avec un réseau de pharmacies partenaires à travers le pays pour garantir la disponibilité des produits et assurer une livraison rapide aux clients.

L'entreprise met l'accent sur la qualité du service client, la sécurité des transactions en ligne, et la conformité aux réglementations locales en matière de santé et de commerce électronique.

Grâce à son approche innovante, Techcare aspire à devenir le leader du marché de la pharmacie en ligne à Madagascar, en facilitant l'accès aux soins de santé pour tous les citoyens, même dans les régions éloignées.

I.2.2. Contexte du projet

Le projet de migration d'une architecture monolithique vers une architecture basée sur des microservices s'inscrit dans le cadre de l'évolution technologique de Techcare.

Avec la croissance rapide du commerce en ligne et l'augmentation du nombre d'utilisateurs, l'architecture monolithique actuelle de la plateforme e-commerce de Techcare présente des limitations en termes de scalabilité, de maintenabilité et de flexibilité.

La décision de migrer vers une architecture basée sur des microservices vise à répondre à ces défis en permettant une meilleure gestion des ressources, une adaptation plus rapide aux besoins changeants du

marché, et une amélioration de l'expérience utilisateur.

Ce projet est également motivé par la volonté de Techcare d'adopter des pratiques de développement logiciel modernes, favorisant l'innovation et la compétitivité dans un secteur en constante évolution.

I.3 Problématique et justification du projet

La problématique centrale de ce projet est de déterminer comment migrer efficacement une architecture monolithique vers une architecture basée sur des microservices, tout en garantissant une expérience utilisateur fluide et cohérente. Cette migration soulève plusieurs défis techniques, organisationnels et stratégiques, notamment en ce qui concerne la décomposition de l'application monolithique en services indépendants, la gestion des communications entre ces services, et la mise en place de processus de développement adaptés.

La justification de ce projet réside sur ces avantages clés offerts par les microservices :

- Amélioration de la scalabilité : chaque microservice peut être mis à l'échelle indépendamment en fonction de la demande.
- Meilleure maintenabilité : les équipes peuvent travailler sur des services spécifiques sans affecter l'ensemble de l'application.
- Flexibilité accrue : possibilité d'adopter différentes technologies et langages pour différents services.
- Résilience améliorée : une défaillance dans un microservice n'affecte pas nécessairement l'ensemble du système.

I.4 Objectifs et périmètre du projet

L'objectif principal de ce projet est de migrer l'architecture monolithique existante de la plateforme e-commerce de Techcare vers une architecture basée sur des microservices, en utilisant des méthodologies et des techniques modernes de développement logiciel.

Les objectifs spécifiques incluent :

- Analyser l'architecture monolithique actuelle et identifier les composants clés à migrer.
- Concevoir une architecture cible basée sur des microservices, en définissant les services, leurs interactions et les technologies à utiliser.
- Mettre en œuvre la migration en développant et déployant les microservices.
- Tester et valider la nouvelle architecture pour assurer sa performance, sa scalabilité et sa fiabilité.
- Documenter le processus de migration et les leçons apprises pour faciliter les futures évolutions.

la gestion des utilisateurs, l'inventaire et les notifications.

I.5 Méthodologie et plan de travail

Pour mener à bien ce projet de migration complexe, nous avons adopté une démarche structurée combinant des principes agiles et une planification rigoureuse par phases.

I.5.1 Méthodologie de gestion de projet : Scrum

Le choix s'est porté sur la méthodologie **Scrum**, une approche agile permettant de gérer le développement de manière itérative et incrémentale. Cette méthode est particulièrement adaptée à une architecture microservices où chaque service peut être considéré comme une unité fonctionnelle indépendante.

- **Sprints** : Le travail a été découpé en cycles de deux semaines (Sprints). Chaque sprint se termine par la livraison d'une fonctionnalité ou d'un service testable.
- **Cérémonies** : Des réunions de planification (Sprint Planning) et des revues de sprint ont permis d'ajuster les priorités en fonction des défis techniques rencontrés (notamment sur la gestion des communications asynchrones).
- **Backlog** : Les besoins ont été traduits en "User Stories" regroupées dans un backlog produit, priorisées selon leur importance critique pour le système (commençant par l'authentification et la passerelle API).

I.5.2 Plan de travail et étapes de réalisation

Le projet s'articule autour de cinq phases majeures réparties sur la durée du stage :

1. **Phase d'Analyse (Semaine 1-2)** : Étude approfondie du monolithe existant, identification des domaines fonctionnels (Bounded Contexts) et définition de la stratégie de découpage.
2. **Phase de Conception (Semaine 3-4)** : Modélisation des microservices, choix des protocoles de communication (RabbitMQ pour l'asynchronisme, REST pour le synchrone) et conception des schémas de données distribués.
3. **Phase d'Implémentation (Semaine 5-12)** : Développement itératif des microservices (User, Product, Inventory, Notification, File Manager) et de l'API Gateway.
4. **Phase de Tests et Intégration (Semaine 13-14)** : Tests unitaires, tests d'intégration entre services et validation de la sécurité via JWT.
5. **Phase de Déploiement et Documentation (Semaine 15-16)** : Conteneurisation des services, mise en place de l'infrastructure de production et finalisation du rapport technique.

I.5.3 Outils de collaboration et environnement technique

La traçabilité et la qualité du code ont été assurées par l'utilisation d'outils standards de l'industrie :

- **Git** : Pour la gestion de version et le travail collaboratif.

- **Concurrently** : Pour l'orchestration du démarrage des services en environnement de développement.
- **Docker** : Pour la gestion des dépendances d'infrastructure (PostgreSQL, MongoDB, RabbitMQ).
- **Trello/Jira** : Pour le suivi du tableau Kanban et l'avancement des tâches Scrum.

Conclusion partielle

Ce premier chapitre a permis de poser les bases du projet en présentant l'organisme d'accueil, l'institut de formation et surtout les enjeux majeurs liés à la modernisation de la plateforme Techcare. L'analyse des besoins a mis en lumière les limites critiques de l'architecture monolithique actuelle face aux ambitions de croissance de l'entreprise. Cette compréhension du contexte justifie pleinement le passage vers une architecture distribuée, dont la conception détaillée fera l'objet du chapitre suivant.

CHAPITRE II

ANALYSE ET CONCEPTION : VERS UNE ARCHITECTURE DISTRIBUÉE

Ce chapitre expose la réflexion conceptuelle menée pour répondre aux exigences de scalabilité et de haute disponibilité de Techcare. Nous y abordons les fondements théoriques des systèmes distribués, le découpage métier via le Domain-Driven Design (DDD) et la modélisation des flux de données.

II.1 Analyse de l'existant et État de l'Art

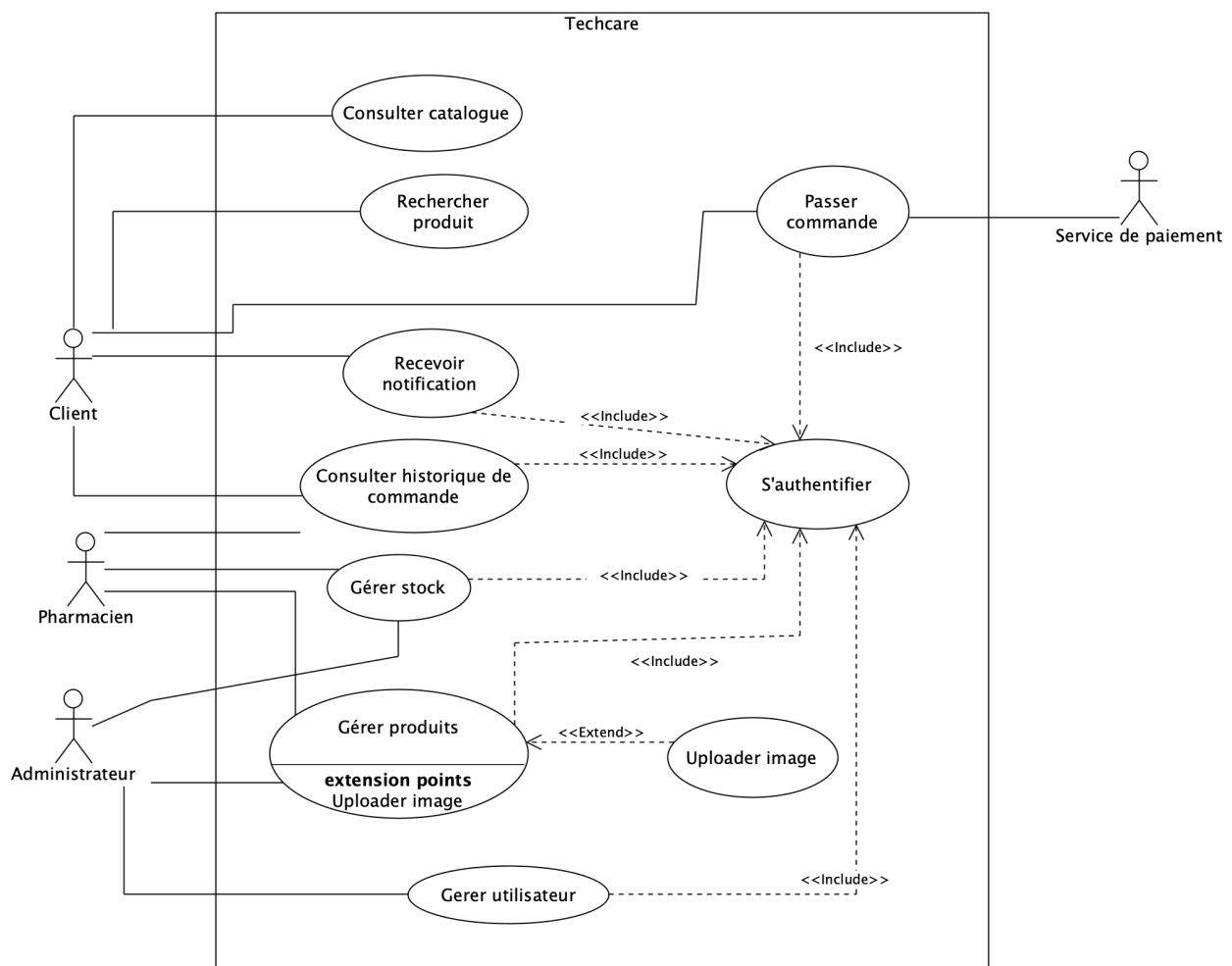


FIGURE 2 – Diagramme de cas d'utilisation du système Techcare

Source : Auteur

L'analyse de ce diagramme de cas d'utilisation permet de dégager les points suivants :

- **Interactions et Acteurs :** Ce diagramme illustre les interactions fondamentales entre les trois acteurs principaux (Client, Pharmacien et Administrateur) et les fonctionnalités clés, couvrant l'intégralité du cycle de vie médical, de la recherche à la délivrance.

- **Périmètre et Dépendances** : Il met en évidence les relations de dépendance, notamment l'authentification systématique requise pour les actions sensibles, et sert de base pour définir le périmètre fonctionnel du système Techcare.

II.1.1 Évolution des architectures logicielles : Un parcours historique

L'histoire de l'ingénierie logicielle est marquée par une oscillation constante entre centralisation et décentralisation, motivée par la recherche d'un équilibre entre contrôle, performance et agilité.

1. **L'ère des Mainframes (années 1960-1980)** : Le calcul était totalement centralisé sur des machines massives. Les utilisateurs interagissaient via des "terminaux passifs" sans puissance de calcul propre. Bien que robuste, ce modèle offrait une flexibilité nulle et un coût d'entrée prohibitif.
2. **L'ère Client-Serveur (années 1990)** : Avec l'avènement du PC, l'intelligence a été répartie. Le modèle *2-tier* (une application lourde communiquant avec une base de données) a permis l'essor des interfaces graphiques complexes, mais posait des problèmes majeurs de maintenance (chaque mise à jour nécessitait une réinstallation sur chaque poste client).
3. **L'Architecture N-tier et le Monolithe Web (années 2000)** : Pour résoudre les problèmes du client-serveur, le calcul est revenu vers le centre (serveur d'application), mais de manière structurée en couches (Présentation, Métier, Données). C'est l'âge d'or des architectures monolithiques bien structurées, où l'intégralité de la logique réside dans un unique bloc déployable.
4. **SOA (Service-Oriented Architecture)** : Avant les microservices, la SOA a tenté de modulariser l'entreprise. En utilisant des protocoles lourds (SOAP, XML) et un bus de communication central (ESB), elle visait la réutilisabilité. Cependant, la complexité de l'ESB créait souvent un nouveau goulot d'étranglement centralisé.
5. **La révolution des Microservices et du Cloud (2010-Aujourd'hui)** : En réponse aux besoins de "Scale" des géants du web (Netflix, Amazon), le concept de microservices a émergé. Il prône un découplage total, où chaque service est une unité autonome.

Le passage du monolithe aux microservices chez Techcare n'est donc pas une simple tendance, mais l'aboutissement de cette évolution vers la décentralisation contrôlée, répondant à la ****loi de Conway**** : pour que Techcare soit agile, son architecture doit refléter une structure de communication fluide et distribuée.

II.1.2 L'Architecture Monolithique : Analyse du Modèle Traditionnel

L'architecture monolithique constitue le modèle canonique de conception logicielle. Bien qu'aujourd'hui souvent critiquée face à l'essor des microservices, elle a dominé l'industrie pendant des décennies et reste pertinente pour nombre de projets. Pour Techcare, comprendre ses mécanismes est crucial pour justifier la transition.

II.1.2.1 Caractéristiques et Modèle de Déploiement

Dans un système monolithique, l'intégralité de la logique métier, de l'accès aux données et de l'interface est compilée dans un unique artefact exécutable. Cette structure implique plusieurs contraintes techniques :

- **Couplage Spatial et Temporel** : Tous les composants s'exécutent dans le même espace mémoire et partagent le même cycle de vie. Un redémarrage du serveur pour mettre à jour le module de facturation rend momentanément indisponible le catalogue de médicaments.
- **Gestion des Ressources Partagées** : Le moteur d'exécution (JVM, Node.js, etc.) gère un pool de threads unique. Une boucle infinie ou une fuite de mémoire dans un composant "non critique" peut saturer les ressources et provoquer l'arrêt total du système.
- **Base de données Centralisée** : Le monolithe communique généralement avec un unique schéma relationnel (Shared Database), facilitant les jointures complexes mais créant un point de contention majeur lors de la montée en charge.

II.1.2.2 Atouts Techniques et Opérationnels

Avant d'atteindre ses limites, le monolithe offre des avantages indéniables :

- **Intégrité Transactionnelle (ACID)** : Les opérations inter-modules bénéficient de transactions locales garanties par le moteur de base de données, évitant la complexité des transactions distribuées (Saga).
- **Latence Zéro Inter-Composants** : Les échanges d'informations se font par passage de références en mémoire. Contrairement aux microservices, il n'y a ni sérialisation (JSON/Protobuf), ni latence réseau, ni risque de timeout entre les fonctions.
- **Facilité de Refactoring** : Grâce au typage fort et à la vision globale du code, les outils de l'IDE permettent de renommer ou de restructurer des composants à travers toute l'application en une seule opération sécurisée.

II.1.2.3 Limites Structurelles et le phénomène "Big Ball of Mud"

Pour Techcare, le monolithe a fini par dériver vers ce que Brian Foote et Joseph Yoder appellent une **"Big Ball of Mud"** (grosse boule de boue) : une architecture sans structure apparente où le couplage devient si dense que toute modification devient imprévisible.

- **Lourdeur du Continuous Deployment (CD) et complexité du débogage** : Le temps de build n'est pas vraiment le problème majeur, le débogage oui. Avec la complexité croissante du code monolithique, identifier la source d'une régression lors d'un déploiement nécessitait de parcourir une base de code dense et fortement couplée, allongeant considérablement le temps de résolution des incidents.

- **Scalabilité Inefficace** : La scalabilité est obligatoirement "homogène". Pour absorber un pic de trafic sur la recherche de médicaments, nous devons dupliquer l'intégralité du socle (incluant les modules lourds et inutilisés en lecture seule), entraînant un surcoût d'infrastructure inutile.
- **Obsolescence Technologique Subie** : Le système est "prisonnier" de ses choix initiaux. Passer d'une version majeure de framework ou changer de langage est une mission quasi-impossible car elle impacterait des millions de lignes de code interdépendantes.

Bilan comparatif : Pour synthétiser cette transition architecturale, le tableau 2 met en évidence les différences fondamentales justifiant le passage aux microservices pour la plateforme Techcare.

Critère	Ancien Monolithe	Nouvelle Architecture (Microservices)
Déploiement	Lourd et risqué. Un seul artefact englobant tout.	Agile et indépendant par service (conteneurs isolés).
Scalabilité	Homogène. Duplication totale de l'application (lourdeur matérielle).	Ciblée. Seuls les services sollicités sont dupliqués.
Tolérance aux pannes	Point de défaillance unique (SPOF). Tout le système tombe en cas de crash.	Haute résilience. L'échec d'un service n'impacte pas le reste.
Persistance	Base de données unique et centralisée (couplage fort).	Base de données par service (Persistance Polyglotte).
Stack technologique	Unique et figé (Risque d'obsolescence).	Hétérogène (Le bon outil pour la bonne tâche).

TABLE 2 – Comparaison entre l'architecture monolithique et microservices

Source : Auteur

II.1.3 Le framework NestJS : Historique et Structure

Face aux limites de l'architecture monolithique, la mise en œuvre de microservices modernes requiert des outils robustes. C'est dans ce contexte que s'inscrit NestJS.

Historique et genèse

Créé en 2017 par Kamil Myśliwiec, NestJS (souvent appelé simplement Nest) est un framework pour la création d'applications Node.js côté serveur. Historiquement, l'écosystème Node.js était dominé par des bibliothèques minimalistes comme Express.js ou Koa. Bien que ces outils offrent une grande flexibilité, ils n'imposent aucune structure architecturale. Cela conduit fréquemment à du code non structuré (spaghetti code) dans les projets d'envergure. NestJS a été conçu pour résoudre ce problème architectural en s'inspirant fortement des concepts d'Angular.

Concepts fondamentaux et architecture de NestJS

NestJS utilise nativement TypeScript (bien qu'il supporte JavaScript pur) et combine des éléments de Programmation Orientée Objet (POO), de Programmation Fonctionnelle (FP) et de Programmation Réactive Fonctionnelle (FRP).

L'architecture de NestJS s'articule autour de plusieurs notions clés qu'il est indispensable de maîtriser :

- **Le CLI (Command Line Interface)** : L'outil NestCLI est essentiel pour l'initialisation et le développement. Il permet de générer rapidement des projets, de scaffolder des éléments (modules, contrôleurs, services) tout en respectant scrupuleusement les conventions de nommage et de structure du framework.
- **Modules** : Un module est une classe annotée avec le décorateur `@Module()`. Il permet d'encapsuler logiquement un domaine métier (par exemple, un `UsersModule` ou `ProductsModule`) en regroupant ses contrôleurs et ses providers. Chaque application possède un module racine. Cette approche encourage fortement la séparation des préoccupations, idéale pour concevoir des microservices indépendants.

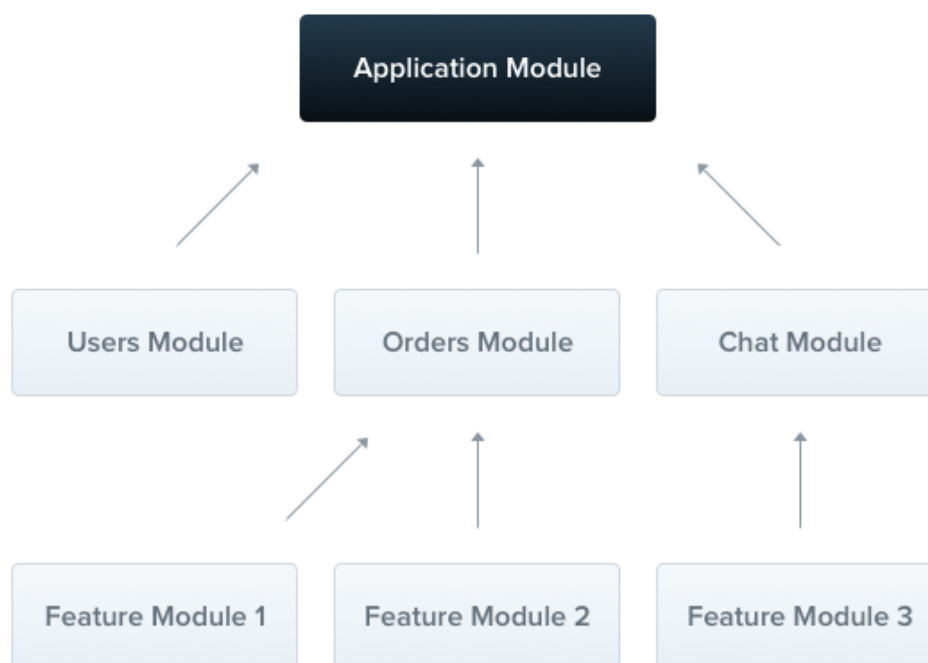


FIGURE 3 – Architecture du module NestJS

Source : nestjs.com

- **Contrôleurs (Controllers)** : Annotés par `@Controller()`, ils constituent la couche de présentation. Leur unique responsabilité est de réceptionner les requêtes entrantes, d'extraire les paramètres (via des décorateurs comme `@Body()`, `@Param()`) et de déléguer la logique métier

aux services. Dans un contexte microservices, ils gèrent aussi le trafic asynchrone (RabbitMQ, TCP) via `@MessagePattern`.

- **Services et Providers** : Les services sont les porteurs de la logique métier (Business Logic) et de l'accès aux bases de données. Dans NestJS, presque toute classe utilitaire (services, repositories, factories) est considérée comme un "Provider" (annotée via `@Injectable()`).
- **L'Injection de Dépendances (Dependency Injection - DI)** : C'est le cœur du système NestJS. Au lieu d'instancier manuellement une classe (via le mot-clé `new`), NestJS gère un conteneur IoC (Inversion of Control) qui injecte automatiquement les instances nécessaires (les Providers) dans les classes qui en ont besoin (comme les Contrôleurs). Cela garantit un code lâchement couplé, hautement modulaire et facilement testable.

Grâce à cette structure rigoureuse et standardisée, NestJS s'impose comme une référence incontournable pour la construction d'architectures distribuées complexes.

Avantages, inconvénients et performances de NestJS

L'adoption de NestJS pour Techcare repose sur un compromis technique équilibré. Il convient d'en évaluer les bénéfices et les limites.

Les Avantages majeurs :

- **Maintenabilité et scalabilité d'équipe** : L'architecture fortement opinionnée oblige tous les développeurs à suivre la même structure, ce qui simplifie l'intégration de nouveaux membres et la revue de code.
- **Testabilité** : L'injection de dépendances (DI) permet de créer facilement des *Mocks* et des *Stubs* pour isoler les composants lors des tests unitaires.
- **Agnosticisme du protocole** : NestJS s'adapte naturellement aux microservices en masquant la complexité des protocoles (HTTP, WebSockets, RabbitMQ, gRPC) derrière une API uniforme.

Les Inconvénients :

- **Courbe d'apprentissage abrupte** : Contrairement à Express.js qui s'apprend en quelques heures, NestJS nécessite une maîtrise de TypeScript, des décorateurs, de l'Inversion de Contrôle (IoC) et de RxJS.
- **Surcharge (Overhead) initiale** : Le framework embarque beaucoup de bibliothèques par défaut, générant une empreinte mémoire légèrement supérieure et un temps de démarrage plus long (Cold Start), ce qui peut être un frein dans des architectures Serverless (FaaS).

Analyse des performances (Benchmark) : En termes de performances brutes, NestJS ajoute une légère couche d'abstraction (overhead) s'il s'exécute sur le moteur par défaut (Express.js). Un benchmark classique de requêtes simples ("Hello World") montre que :

- **Express.js (natif)** : ~ 15 000 requêtes par seconde.

- **NestJS (sous Express)** : ~ 12 000 requêtes par seconde (en raison de l’interception et de l’injection de dépendances).

Toutefois, NestJS pallie cet écart en permettant de basculer son moteur sous-jacent sur **Fastify**. Lorsqu’il est configuré avec Fastify, NestJS atteint jusqu’à **30 000 requêtes par seconde**, dépassant largement Express.js tout en conservant son architecture robuste. Pour Techcare, cette modularité garantit que l’architecture logicielle ne deviendra pas un goulot d’étranglement lors de la montée en charge.

II.2 Fondements Théoriques et Contraintes Distribuées

II.2.1 Le Théorème CAP : Le dilemme de la distribution

Formulé par Eric Brewer, le théorème CAP stipule que pour un système informatique distribué, il est impossible de garantir simultanément plus de deux des trois propriétés suivantes :

- **Consistance (Consistency)** : Tous les nœuds voient les mêmes données au même moment.
- **Disponibilité (Availability)** : Chaque requête reçoit une réponse (succès ou échec), sans garantie qu’elle contienne la version la plus récente.
- **Partition Tolerance** : Le système continue de fonctionner malgré la perte de messages ou la défaillance d’une partie du réseau.

Pour Techcare, notre choix s’est porté sur une architecture ****AP (Availability and Partition Tolerance)****. Dans un contexte pharmaceutique, il est préférable qu’un client puisse consulter le catalogue de médicaments (quitte à voir un stock légèrement désynchronisé de quelques secondes) plutôt que de se confronter à une erreur 500 bloquante. Nous adoptons donc le modèle de **Consistance Éventuelle** (*Eventual Consistency*), où les données se propagent de manière asynchrone via RabbitMQ.

II.2.2 Patterns de Résilience : Prévenir l’effet domino

L’architecture distribuée multiplie les points de défaillance réseau. Afin de prévenir la propagation des pannes (effet domino), notre phase de conception théorique intègre plusieurs patrons d’architecture (patterns) majeurs de l’état de l’art, dont l’implémentation effective en production constitue un axe d’amélioration clé pour la plateforme :

- **Circuit Breaker (Disjoncteur)** : Ce pattern surveille les échecs d’un service distant. S’ils dépassent un seuil, le disjoncteur "s’ouvre" et redirige immédiatement les appels vers un mode dégradé (ex : cache ou message d’erreur rapide), évitant ainsi d’épuiser les ressources du client en attendant un timeout.
- **Retry Pattern** : Pour les erreurs transitoires (micro-coupures réseau), le système réessaye automatiquement la requête avec une stratégie d’*Exponential Backoff* pour ne pas saturer le service cible.

- **Bulkhead** : Inspiré des compartiments étanches des navires, ce pattern isole les pools de threads par service. Une saturation du *Payment Service* n'impactera pas les performances du *Product Service*.

II.3 Découpage Métier : Le Domain-Driven Design (DDD)

Le Domain-Driven Design est au cœur de notre stratégie de décomposition. Il permet de s'assurer que le code reflète fidèlement la complexité métier.

- **Bounded Contexts (Contextes Délimités)** : C'est la pierre angulaire de notre architecture. Au lieu d'avoir un modèle "Produit" géant, nous avons fragmenté ce concept. Dans le *Product Manager*, le produit est défini par son marketing (images, description). Dans l'*Inventory Manager*, il est défini par sa logistique (code-barres, emplacement, seuil d'alerte).
- **Ubiquitous Language (Langage Commun)** : Nous avons structuré le code pour refléter fidèlement les notions de gestion d'officine (ex : l'événement `stock_unavailable` pour la rupture de stock, et les types précis de mouvements de stock `RESERVATION` et `RELEASE` dans l'inventaire) plutôt que de simples opérations génériques d'incrémentation.
- **Aggregate Roots** : Chaque microservice gère une entité racine (ex : l'Ordre dans le service Commande) qui garantit l'intégrité de ses composants internes.

II.4 Modélisation des Microservices et Communications

II.4.1 API Gateway et Backend-for-Frontend (BFF)

L'API Gateway constitue le point d'entrée unique pour tous les clients du système Techcare. Elle agit comme une couche d'abstraction entre les clients (Web, Mobile) et la complexité interne des microservices.

- **Façade et Routage intelligent** : La Gateway redirige les requêtes entrantes vers les microservices appropriés en fonction de l'URL, masquant ainsi la topologie réelle du réseau interne et les adresses spécifiques de chaque service.
- **Agrégation de requêtes** : Pour optimiser les performances, elle peut fusionner les réponses de plusieurs services en un seul appel (ex : réunir les informations marketing de *ProductService* et les niveaux de stock de *InventoryService*), réduisant ainsi le "bavardage" réseau (*chatter*) et la latence côté client.
- **Gouvernance centralisée** : Elle assure des fonctions transverses (*Cross-Cutting Concerns*) telles que la terminaison SSL, la gestion des CORS, le *Rate Limiting* pour prévenir les abus, et la validation initiale des jetons JWT.
- **Pattern BFF (Backend-for-Frontend)** : Cette approche nous permet de fournir des réponses adaptées aux besoins spécifiques de chaque interface utilisateur. Un client mobile recevra une

version allégée des données pour économiser la bande passante, tandis qu'un portail administrateur pourra obtenir des informations plus denses.

II.4.2 Chorégraphie : La force de l'asynchronisme

Pour la communication entre services, nous avons exclu l'orchestration centralisée (qui crée un couplage fort et un SPOF) au profit de la **Chorégraphie** via le broker de messages RabbitMQ :

- **Découplage temporel** : Un service peut émettre un événement même si le destinataire est temporairement hors ligne. Le broker garantit la persistance des messages.
- **Scalabilité horizontale** : Plusieurs instances d'un service peuvent consommer la même file d'attente pour traiter les tâches en parallèle.
- **Réduction de la latence** : L'utilisateur reçoit une réponse immédiate ("Commande acceptée"), tandis que le traitement lourd (paiement, notification, mise à jour des stocks) s'effectue en arrière-plan.

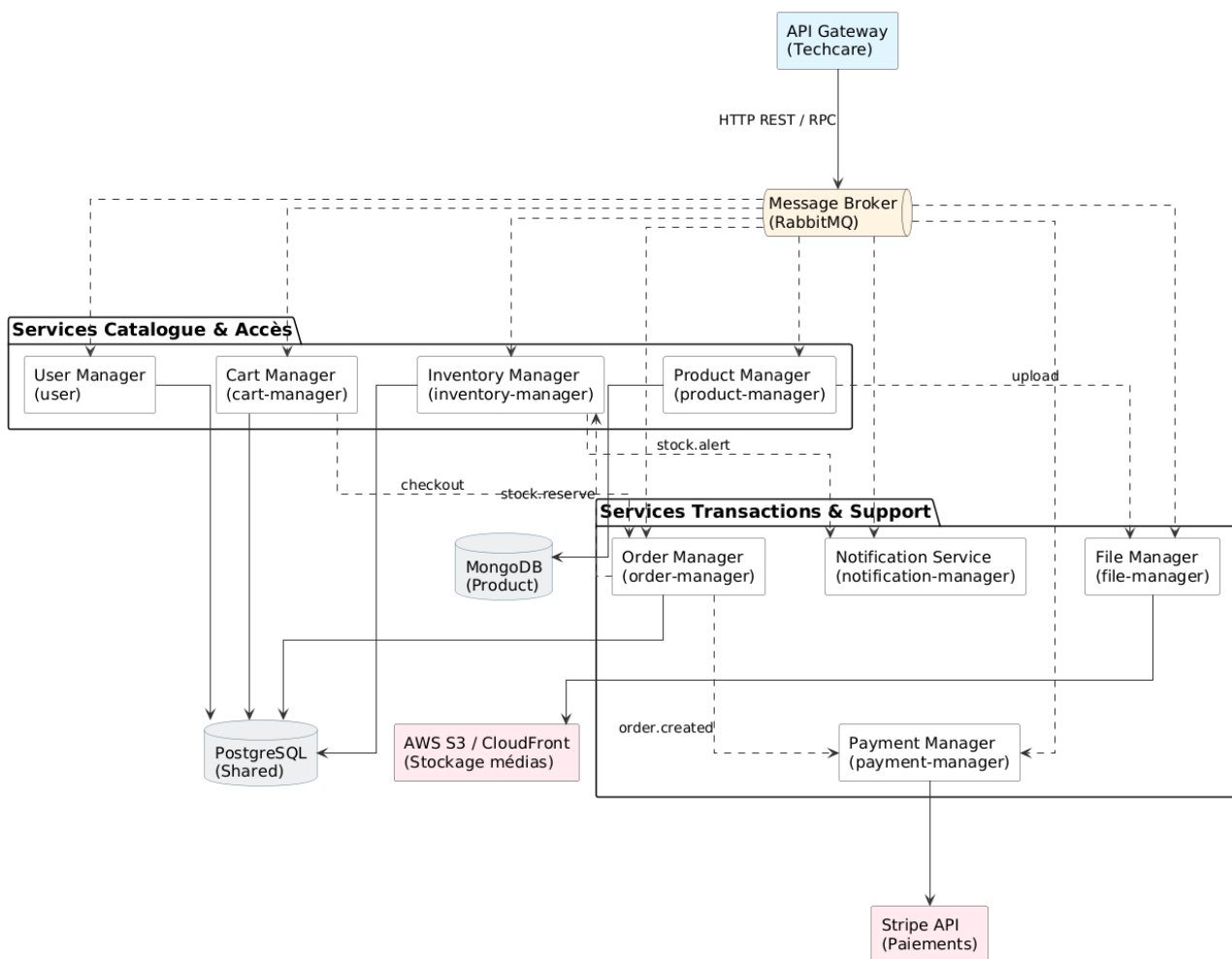


FIGURE 4 – Diagramme d'interaction et de communication des microservices (Chorégraphie)

Source : Auteur

L'analyse de ces flux d'interaction met en évidence le rôle central du Message Broker (RabbitMQ) dans la synchronisation des états et l'indépendance de la persistance (PostgreSQL et MongoDB).

II.4.3 Modélisation des classes logiques des microservices

Chaque microservice, bien qu'isolé dans son propre conteneur, adopte une architecture logicielle interne structurée et uniforme (Controller, Service, Entity). Le diagramme de classes global illustré par la figure 5 présente cette organisation structurée. Pour des raisons de lisibilité et de clarté, les attributs et méthodes détaillés de chaque classe ne sont pas représentés afin de se concentrer sur l'architecture générale, l'organisation en packages et les relations entre composants logiques.

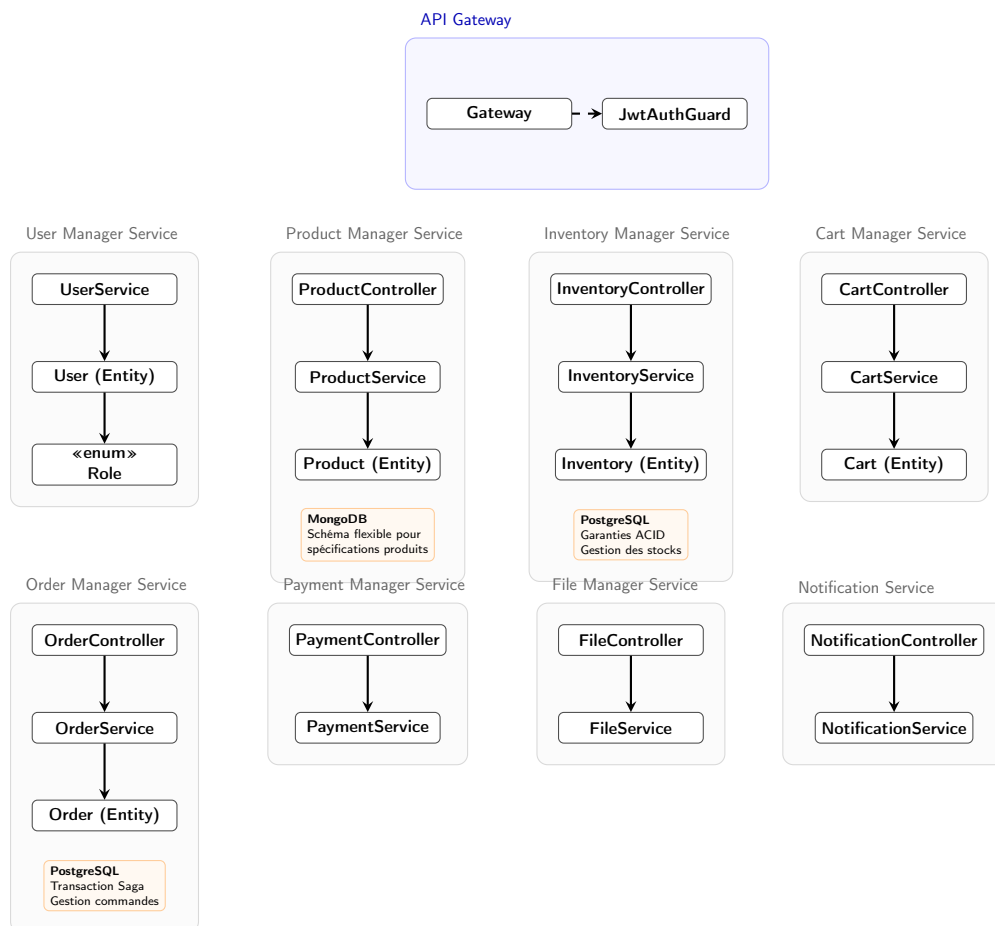


FIGURE 5 – Diagramme de classes global de l'architecture microservices

Source : Auteur

L'analyse de cette architecture de classes révèle deux aspects fondamentaux de la conception :

- **Organisation Domain-Driven (DDD)** : Contrairement au monolithe, les entités sont regroupées par contextes délimités (Catalog, Inventory, Order), garantissant un faible couplage et permettant à chaque domaine de posséder sa propre logique métier indépendante.
- **Flexibilité et Évolutivité** : La structure met en évidence le rôle de l'API Gateway et la séparation des services, ce qui permet au système d'évoluer organiquement et de faciliter l'adoption de

nouvelles technologies par service sans impacter l'ensemble du système.

II.5 Architecture de Persistance et Base de Données

Le choix de la **Persistance Polyglotte** répond au besoin de performance :

- **PostgreSQL (ACID)** : Indispensable pour l'Inventory Manager afin d'éviter le "Double Spending" de médicaments.
- **MongoDB (BASE)** : Idéal pour le Product Manager afin de stocker des métadonnées hétérogènes (composition, posologie, photos).

II.5.1 Justification des choix technologiques

Le tableau suivant synthétise les critères de sélection des bases de données :

Critère	PostgreSQL	MongoDB
Modèle de données	Relationnel (schéma fixe)	Documentaire (schéma flexible)
Consistance	ACID stricte	Consistance éventuelle
Cas d'usage Techcare	Stocks, Utilisateurs	Catalogue produits
Performance	Excellent pour transactions	Excellent pour lectures massives

TABLE 3 – Comparaison des systèmes de persistance retenus

Source : Auteur

II.5.1.1 Justification du choix de PostgreSQL

PostgreSQL a été retenu pour les services nécessitant une **intégrité transactionnelle stricte**. Voici les raisons détaillées de ce choix :

- Garanties ACID indispensables** : Pour le service **Inventory Manager**, la gestion des stocks de médicaments ne tolère aucune incohérence. Les propriétés ACID de PostgreSQL garantissent :
 - **Atomicité** : Une opération de décrémentation de stock est soit totalement effectuée, soit totalement annulée.
 - **Consistance** : Les contraintes de vérification (ex : `quantity >= 0`) sont appliquées.
 - **Isolation** : Les verrous empêchent les réservations doubles simultanées.
 - **Durabilité** : Les modifications sont persistées même en cas de panne.
- Support des contraintes relationnelles** : Le service **User Manager** exploite les relations entre tables. PostgreSQL impose l'intégrité référentielle, évitant les incohérences de données orphelines.

- c) **Performance des jointures complexes** : Les requêtes agrégées (ex : historique de commandes détaillé) bénéficient de l'optimiseur de PostgreSQL, très performant pour les jointures massives.
- d) **Typage strict et sécurité** : L'utilisation de Drizzle ORM avec PostgreSQL offre une sécurité type-safe dès la compilation, réduisant drastiquement les bugs.

II.5.1.2 Justification du choix de MongoDB

MongoDB a été sélectionné pour le service **Product Manager** en raison de la nature hétérogène des produits pharmaceutiques.

- a) **Flexibilité du schéma documentaire** : Les produits médicaux ont des caractéristiques variables (sirops, comprimés, dispositifs médicaux). MongoDB permet de stocker ces variations dans une même collection sans migration de schéma complexe.
- b) **Performance des lectures à grande échelle** : Le catalogue étant majoritairement consulté en lecture, MongoDB excelle grâce à son indexation avancée et son cache mémoire intégré.
- c) **Support natif des données imbriquées** : Un produit contient des tableaux d'images ou de contre-indications. MongoDB stocke ces données nativement, évitant les jointures SQL coûteuses.
- d) **Évolutivité sans migration** : L'ajout de nouveaux types de produits ne nécessite aucune modification de la structure de la base de données existante.

Conclusion sur la persistance (Trade-offs) : Bien que la persistance polyglotte offre une flexibilité maximale, elle impose une complexité opérationnelle accrue. L'équipe de développement doit maîtriser deux langages de requêtage différents et la gestion des sauvegardes (*backups*) devient plus hétérogène, nécessitant des stratégies distinctes pour le relationnel et le documentaire.

II.5.1.3 Justification des technologies applicatives et services tiers

Le choix des outils de développement et des services externes a été guidé par des impératifs d'architecture distribuée et de sécurité de l'information.

- a) **Le framework NestJS** : Comme détaillé précédemment dans l'état de l'art (Section II.1.3), NestJS s'est imposé naturellement pour structurer l'ensemble de nos microservices grâce à son architecture fortement modulaire, son typage strict (TypeScript) et son intégration native des flux asynchrones.
- b) **Stockage Objet (AWS S3) pour l'approche Stateless** : Dans une architecture distribuée, les conteneurs doivent être éphémères et sans état (*stateless*). Si un microservice sauvegarde une image de produit localement sur son disque, cette donnée sera perdue au redémarrage du conteneur ou sera inaccessible pour les autres instances en cas de montée en charge (*Scale-out*). L'utilisation d'un stockage objet déporté comme Amazon S3 permet de centraliser les médias de manière hautement disponible, tout en déchargeant nos serveurs applicatifs.

- c) **Délégation de paiement (Stripe) et conformité** : Le traitement des paiements en ligne soulève des contraintes légales et sécuritaires majeures, notamment la norme **PCI-DSS** (Payment Card Industry Data Security Standard). Traiter et stocker soi-même des données bancaires expose la plateforme à des risques de cyberattaques sévères. La délégation complète de cette logique à une passerelle spécialisée comme Stripe garantit une sécurité transactionnelle de niveau bancaire sans alourdir notre infrastructure.

Limites et contraintes identifiées : L'adoption de ces services infonuagiques (*Cloud Computing*) n'est pas sans contreparties. La forte dépendance à des services tiers comme AWS S3 ou Stripe introduit un risque de **Vendor Lock-in** (dépendance vis-à-vis d'un fournisseur unique). De plus, elle soumet le système à la disponibilité réseau de ces prestataires externes : si l'API de Stripe tombe en panne, le processus de commande sur Techcare est bloqué malgré la résilience de nos propres microservices.

II.5.2 Gestion des transactions distribuées : Pattern Saga

Dans une architecture microservices, l'absence de transaction globale (ACID) inter-services pose un défi majeur. Le pattern **Saga**, introduit par Hector Garcia-Molina et Kenneth Salem en 1987, apporte une solution élégante à ce problème.

Définition formelle

Une Saga est une séquence de transactions locales $T_1, T_2, \dots, T_n$ où chaque transaction T_i possède une transaction compensatoire C_i . Si une transaction T_k échoue, le système exécute les compensations $C_{k-1}, C_{k-2}, \dots, C_1$ dans l'ordre inverse pour annuler les effets des transactions déjà validées.

Les deux approches de Saga :

II.5.2.1 Orchestration (Centralisée)

Un coordinateur central (Orchestrator) pilote l'exécution des transactions.

- **Avantages** : Visibilité complète du flux, facilité de debugging
- **Inconvénients** : Point de défaillance unique (SPOF)

II.5.2.2 Chorégraphie (Décentralisée)

Choix retenu pour Techcare. Chaque service écoute des événements et déclenche ses propres actions.

- **Avantages** : Pas de SPOF, meilleure scalabilité
- **Inconvénients** : Complexité accrue du monitoring

Exemple concret : Commande de médicament

Considérons le scénario où un client commande un médicament sur Techcare :

Étapes de la Saga (Happy Path) :

1. **ProductService** : Vérifie la disponibilité du produit → Émet ProductValidated
2. **InventoryService** : Réserve le stock → Émet StockReserved
3. **PaymentService** : Traite le paiement → Émet PaymentCompleted
4. **NotificationService** : Envoie confirmation → Émet OrderConfirmed

Scénario de compensation (Failure Path) : Si le PaymentService échoue à l'étape 3, les compensations suivantes sont déclenchées :

- C_2 : InventoryService libère la réservation de stock
- C_1 : ProductService marque le produit comme disponible
- Notification d'échec envoyée au client

Implémentation via RabbitMQ : Chaque service publie des événements dans des échanges dédiés. Les autres services écoutent les événements qui les concernent via des queues. Cette architecture événementielle garantit la résilience : si un service est temporairement indisponible, les messages restent dans la queue jusqu'à son retour en ligne.

Événement	Émetteur	Écouteur
ProductValidated	ProductService	InventoryService
StockReserved	InventoryService	PaymentService
PaymentFailed	PaymentService	InventoryService
StockReleased	InventoryService	NotificationService

TABLE 4 – Flux d'événements dans la Saga de commande

Source : Auteur

Ce mécanisme de chorégraphie distribué est illustré techniquement par le diagramme de séquence présenté en section II.7.1 (page 31).

II.5.3 Traçabilité Distribuée : L'enjeu de l'observabilité

Dans une architecture distribuée où une simple requête utilisateur peut traverser la Gateway et plusieurs microservices via RabbitMQ, la localisation de l'origine d'une panne devient extrêmement complexe. Si un "Timeout" survient, il est impossible de déterminer instantanément quel maillon de la chaîne a cédé en consultant les journaux standards.

Pour pallier cette perte de visibilité, notre conception intègre dès l'origine le concept de **Traçabilité Distribuée (Distributed Tracing)**. Ce pattern consiste à attacher un identifiant unique (Trace ID) à chaque requête entrante. Cet identifiant est ensuite propagé de service en service, permettant de reconstruire le cycle de vie complet de la transaction. L'implémentation de cette solution s'appuiera sur des standards ouverts afin d'offrir une vue globale (topologie réseau, temps de traitement de chaque nœud) et d'isoler rapidement les composants défaillants sans modifier la logique métier.

II.6 Modélisation de la Sécurité : Zero Trust Architecture

Dans notre conception, nous n'accordons aucune confiance implicite au réseau interne.

II.6.1 Architecture de sécurité en couches

1. **Authentication Stateless** : Utilisation de JWT signés par algorithme asymétrique (RS256).
2. **Propagation d'identité** : Le jeton est transmis de service en service, permettant un audit complet de chaque action.
3. **Chiffrement End-to-End et en transit** :
 - **Transactions financières** : Cryptage de bout en bout entre le client et Stripe (les données bancaires ne transitent jamais sur nos serveurs).
 - **Données sensibles** : Utilisation de TLS 1.3 pour toutes les communications avec AWS S3 et entre l'API Gateway et les clients.
 - **Sécurisation au repos** : Hachage des mots de passe via l'algorithme Bcrypt dans le User Service.

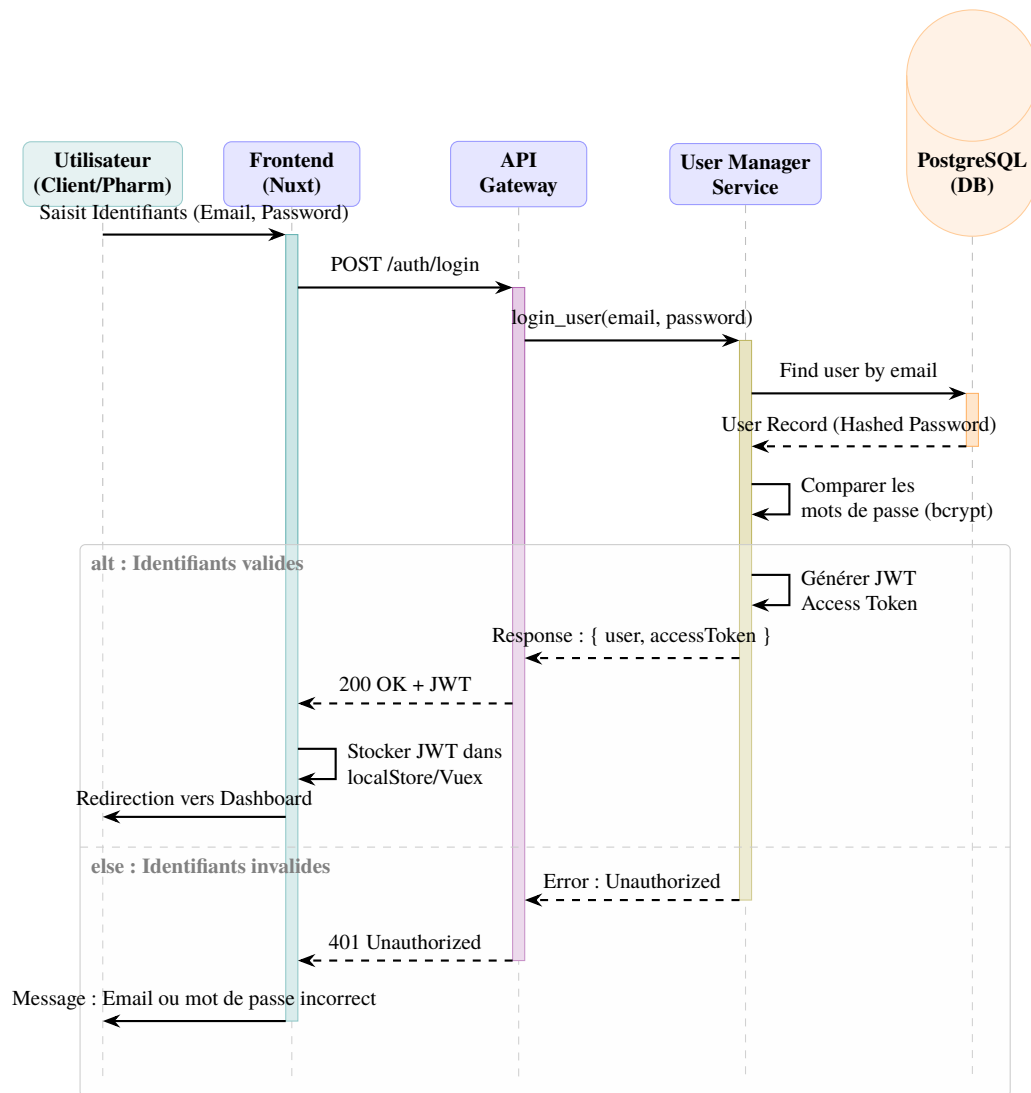


FIGURE 6 – Diagramme de séquence de l’authentification utilisateur

Source : Auteur

Ce flux d’authentification met en exergue deux piliers de notre stratégie de sécurité :

- **Authentification Stateless avec JWT** : Le diagramme détaille l’utilisation de jetons signés cryptographiquement (RS256) transmis au client, ce qui élimine le besoin de sessions persistantes sur le serveur et facilite la scalabilité horizontale.
- **Validation Décentralisée** : Cette approche délègue la vérification de l’identité à chaque micro-service de manière autonome, supprimant les goulots d’étranglement centraux et renforçant la résilience globale lors de la navigation sur la plateforme.

II.6.2 Gestion des rôles et permissions

Le système implémente un contrôle d’accès basé sur les rôles (RBAC) :

- **Admin** : Accès complet (CRUD produits, gestion stocks, utilisateurs)
- **Pharmacien** : Lecture catalogue, gestion commandes, mise à jour stocks

— **Client** : Consultation catalogue, passage de commandes

II.7 Modélisation des Flux et Cas d'Usage

Pour illustrer l'architecture, prenons le cas d'usage critique : "Passer une commande".

II.7.1 Processus métier de commande

Le flux se déroule ainsi :

1. Le client envoie une requête HTTP POST à l'API Gateway
2. La Gateway vérifie le JWT et route vers le Product Manager via RabbitMQ
3. Le Product Manager valide la disponibilité
4. Un événement "ProductReserved" est publié
5. L'Inventory Manager écoute cet événement et décrémente le stock
6. Une notification est envoyée au client via Socket.io

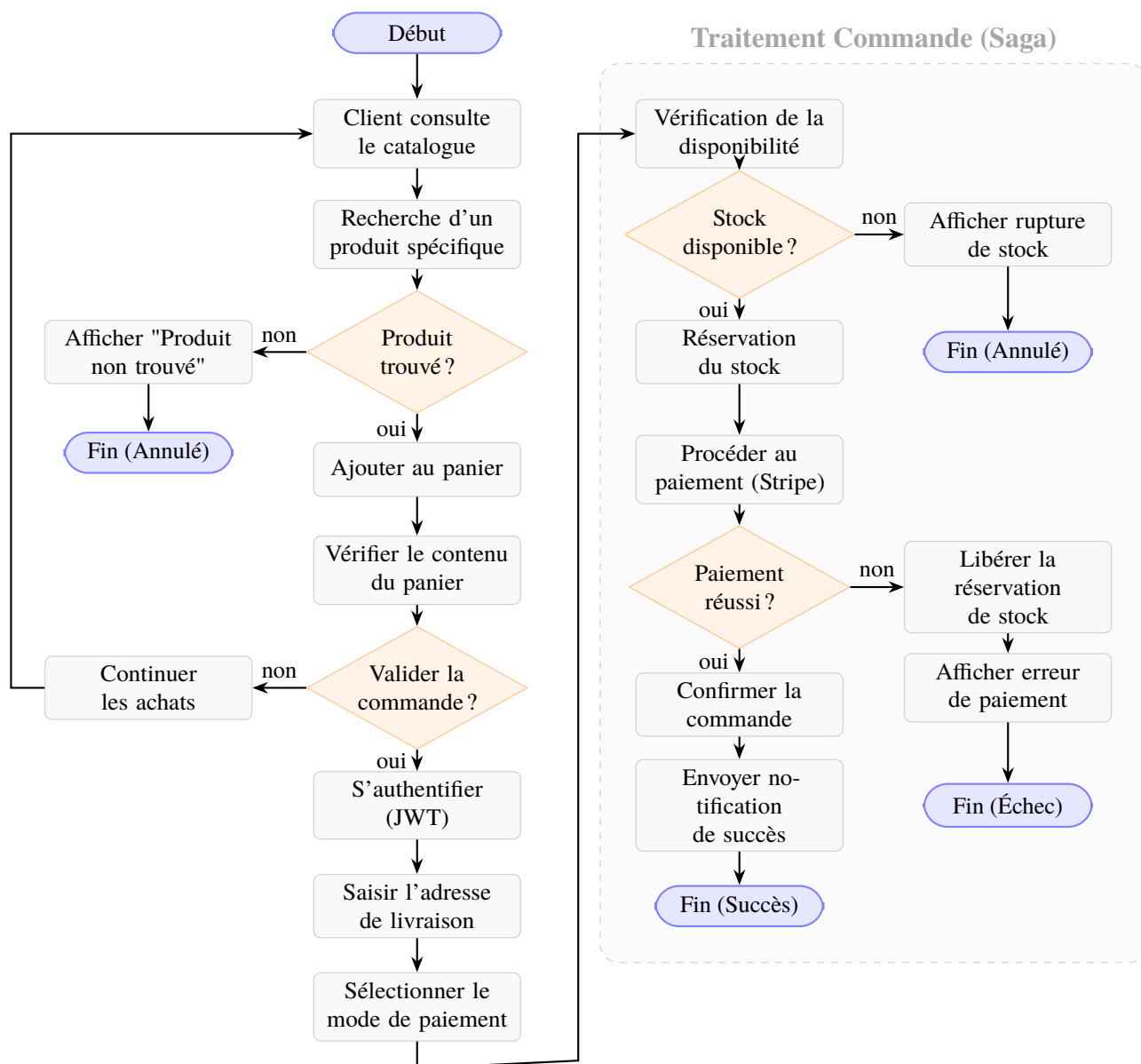


FIGURE 7 – Diagramme d'activité du processus de commande

Source : Auteur

Ce processus métier peut être analysé selon deux axes principaux :

- **Logique Métier et Décisions** : Ce flux retrace le cheminement critique d'une commande, soulignant les points de décision comme la validation des stocks ou le succès du paiement, et les bifurcations en cas d'erreur.
- **Optimisation de l'Expérience** : La visualisation de ces étapes a permis d'identifier les zones de contact utilisateur, facilitant l'intégration de notifications en temps réel pour informer le client de l'état d'avancement de son traitement.

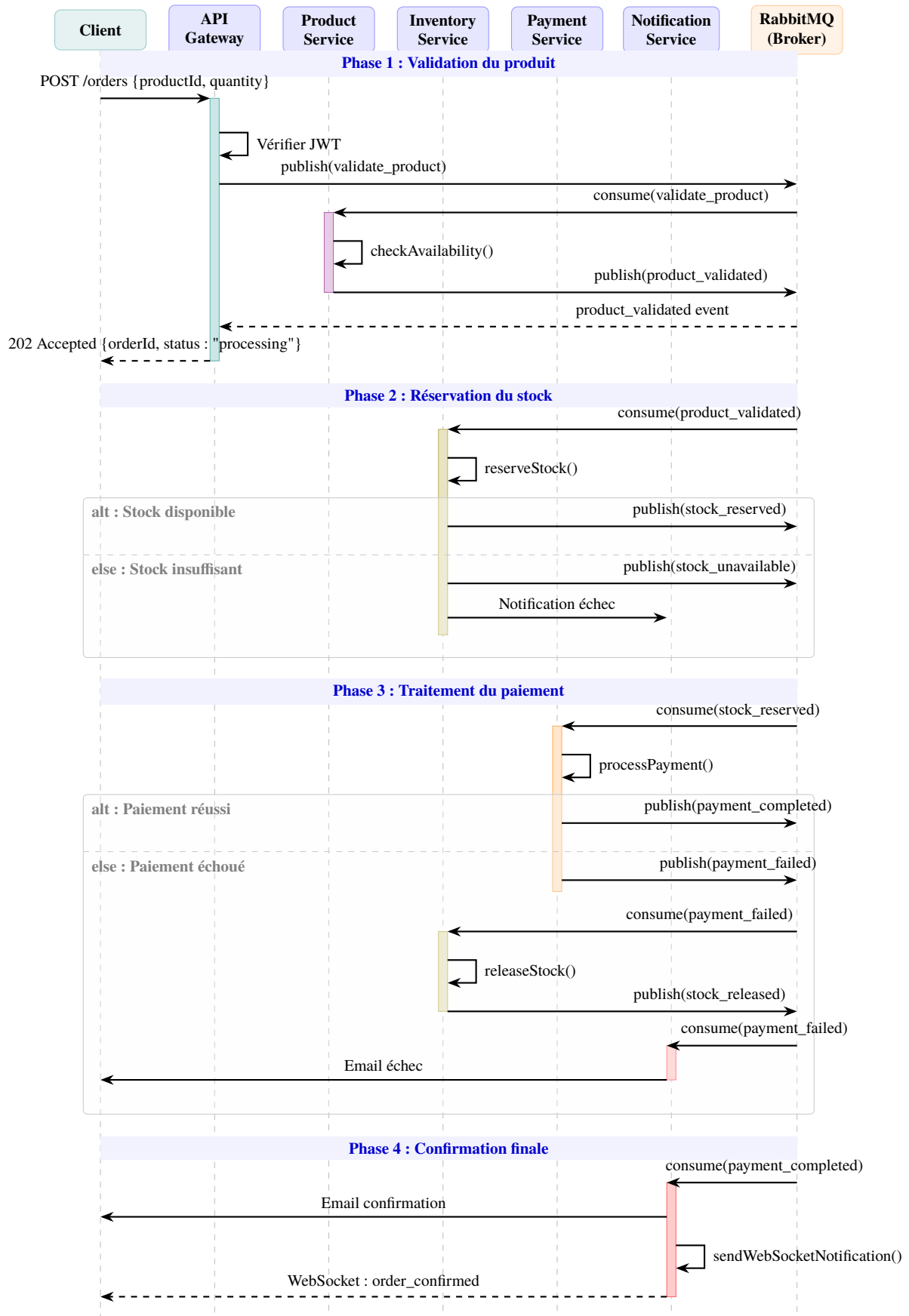


FIGURE 8 – Diagramme de séquence du pattern Saga avec compensation

Source : Auteur

Le pattern Saga ainsi modélisé apporte des garanties essentielles à l'architecture distribuée :

- **Chorégraphie Asynchrone** : L'implémentation repose sur le broker de messages RabbitMQ pour coordonner les services sans point de défaillance unique (SPOF), chaque microservice réagissant intelligemment aux événements métiers.
- **Transactions Compensatoires** : Le diagramme montre comment le système assure l'intégrité des données à travers des services séparés en exécutant des actions de "rollback" (ex : libération de stock) en cas d'échec d'une étape transactionnelle ultérieure.

Conclusion partielle

En somme, ce deuxième chapitre a permis d'étalhir une conception robuste et cohérente de l'architecture cible. En nous appuyant sur les principes du Domain-Driven Design et du pattern Saga, nous avons modélisé un système capable de garantir la résilience et la scalabilité nécessaires à Techcare. Cette réflexion théorique et cette modélisation UML constituent le plan directeur indispensable pour la phase de réalisation technique, détaillée dans le chapitre suivant.

CHAPITRE III

RÉALISATION ET MISE EN ŒUVRE TECHNIQUE

Ce chapitre décrit l'étape de concrétisation technique de l'architecture conçue précédemment. Nous détaillons ici le stack technologique retenu, les spécificités de l'implémentation de chaque microservice et les défis rencontrés lors de l'intégration.

III.1 Organisation du Projet et Arborescence

Afin de gérer efficacement la multiplicité des microservices, le projet est structuré sous forme de **Monorepo**. Cette organisation permet de centraliser la configuration globale tout en isolant le code de chaque service.

```
techcare-root/  
|-- apps/  
|   |-- techcare/ (Gateway)  
|   |-- user/  
|   |-- product-manager/  
|   '-- ... (8 services)  
|-- libs/  
|   |-- database/  
|   '-- shared/ (DTOs, Interfaces, Enums)  
|-- Dockerfile  
'-- docker-compose.yml  
  
src/ (Microservice)  
|-- domain/  
|   |-- entities/  
|   '-- interfaces/  
|-- application/  
|   '-- services/  
|-- infrastructure/  
|   |-- controllers/  
|   '-- persistence/  
'-- main.ts
```

FIGURE 9 – Structure globale (Monorepo)

Source : Auteur

FIGURE 10 – Structure d'un service

Source : Auteur

L'ajout de la bibliothèque `libs/shared` est un élément crucial de cette organisation. Elle permet de centraliser les définitions de types, les interfaces et les objets de transfert de données (DTOs) utilisés par plusieurs services. Cette approche évite la duplication de code et garantit une synchronisation parfaite des contrats d'interface entre la Gateway et les microservices Backend.

III.2 Environnement Technologique et Outils

Le choix du stack a été dicté par le besoin de performance, de modularité et de rapidité de développement.

III.1.1 Framework NestJS

NestJS est utilisé comme socle de développement pour tous les microservices. Son architecture modulaire permet de structurer le code de manière prévisible et facilite l'application de l'architecture hexagonale détaillée ci-après.

III.1.2 Communication : RabbitMQ

RabbitMQ sert de colonne vertébrale à notre système. Il assure le transport des messages entre l'API Gateway et les services Backend de manière asynchrone, garantissant la livraison même en cas de charge élevée.

Configuration de base :

```
ClientsModule.register([
  {
    name: 'PRODUCT_SERVICE',
    transport: Transport.RMQ,
    options: {
      urls: ['amqp://admin:admin@localhost:5672'],
      queue: 'product_manager_queue',
      queueOptions: { durable: false }
    }
  }
])
```

III.1.3 Bases de données et ORM

- **PostgreSQL + Drizzle ORM** : Pour le typage strict et les transactions SQL. Drizzle offre une meilleure expérience développeur que TypeORM avec une syntaxe fluide.
- **MongoDB + Mongoose** : Pour la flexibilité du stockage documentaire. Le plugin de schéma validation assure l'intégrité malgré le NoSQL.

III.2.1 Gestion de la Configuration et des Secrets

Dans une architecture microservices, chaque service possède ses propres paramètres (identifiants de base de données, clés API, URLs). Centraliser ces valeurs en dur dans le code constitue une faille de sécurité critique. Techcare adopte une gestion rigoureuse via le **ConfigModule** de NestJS et des fichiers `.env` isolés par service.

```
// Fichier .env (non versionné dans Git)
RABBITMQ_URL=amqp://admin:secret@rabbitmq:5672
DATABASE_URL=postgresql://user:pass@postgres:5432/db
JWT_SECRET=super_secret_key_rsa_256
AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
STRIPE_SECRET_KEY=sk_live_...
```

Ces variables sont chargées et validées au démarrage de chaque service grâce au **ConfigModule** :

```
// app.module.ts
@Module({
  imports: [
    ConfigModule.forRoot({
      isGlobal: true,      // Accessible dans tout le service
      envFilePath: '.env',
      validationSchema: Joi.object({
        RABBITMQ_URL: Joi.string().required(),
        DATABASE_URL: Joi.string().required(),
        JWT_SECRET: Joi.string().min(32).required(),
      }),
    }),
  ],
})
export class AppModule {}
```

En production (via Docker Compose), les secrets ne sont jamais codés en dur dans les images. Ils sont injectés comme variables d'environnement au moment du lancement des conteneurs, ce qui garantit qu'une image compromise ne révèle aucune donnée sensible :

```
# docker-compose.yml (extrait)
services:
  gateway:
    environment:
      - JWT_SECRET=${JWT_SECRET}    # Lu depuis le .env
      - RABBITMQ_URL=${RABBITMQ_URL} # de l'hôte Docker
```

III.3 Architecture Interne et Patrons de Conception

Au-delà de l'architecture globale distribuée, chaque microservice de Techcare suit une structure interne rigoureuse basée sur les principes de l' "Architecture Hexagonale" (ou *Ports and Adapters*). Cette approche garantit que la logique métier reste isolée des préoccupations techniques (base de données, protocoles de communication).

III.2.1 Organisation en couches

Pour assurer une maintenabilité optimale, nous avons divisé chaque service en trois couches distinctes :

1. **Couche Domaine (Cœur)** : Contient les entités métier et la logique pure du projet, sans dépendance externe.
2. **Couche Application** : Regroupe les services qui orchestrent les cas d'usage (ex : `OrderService`).
3. **Couche Infrastructure** : Gère les détails techniques comme les adaptateurs de base de données (Drizzle, Mongoose) et les clients de messagerie.

III.2.2 Flux de données et DTOs

La communication entre ces couches et avec le monde extérieur (API Gateway) est normalisée via des **DTOs (Data Transfer Objects)**. Initialement dupliqués, ces objets ont été centralisés dans la bibliothèque partagée (@app/shared). Ils assurent la validation des données entrantes grâce à `class-validator`, garantissant qu’aucune donnée corrompue ne pénètre dans le système. Cette centralisation permet également d’imposer un typage strict sur l’ensemble du Monorepo, éliminant les erreurs de désynchronisation entre services.

III.3.3 Tableau Récapitulatif des Microservices

Le tableau suivant synthétise l’ensemble des microservices composant la plateforme Techcare, leurs responsabilités et leurs caractéristiques techniques :

Service	Responsabilité	Protocole	Base de données	Événements clés
API Gateway	Routage, Auth, CORS	HTTP REST	–	–
User Manager	Authentification, Profils	RabbitMQ	PostgreSQL	<code>user.created</code>
Product Manager	Catalogue, Catégories	RabbitMQ	MongoDB	<code>product.validated</code>
Inventory Manager	Stocks, Réservations	RabbitMQ	PostgreSQL	<code>stock.reserved</code> , <code>stock.released</code>
Order Manager	Commandes, Saga	RabbitMQ	PostgreSQL	<code>order.created</code> , <code>order.cancelled</code>
Payment Manager	Paiement Stripe	RabbitMQ	–	<code>payment.completed</code> , <code>payment.failed</code>
File Manager	Upload S3	RabbitMQ	–	–
Notification Manager	Emails, Socket.io	RabbitMQ	–	<code>notification.sent</code>

TABLE 5 – Récapitulatif des microservices de la plateforme Techcare

Source : Auteur

III.4 Développement de l’API Gateway

La Gateway est le point d’entrée HTTP unique. Elle ne se contente pas de router les requêtes, elle joue un rôle actif dans l’orchestration et la sécurisation des flux.

III.4.1 Architecture et Responsabilités

L'organisation interne de la Gateway suit le modèle MVC de NestJS, mais avec une séparation stricte par domaine :

- **CORS et Sécurité** : Centralisation des politiques de partage de ressources et protection contre les attaques courantes.
- **Gestion des Sessions** : Validation des tokens JWT et extraction des claims.
- **Normalisation** : Transformation des exceptions microservices (RPC) en codes HTTP standard.

Le point d'entrée de la Gateway (`main.ts`) illustre la configuration centralisée de ces responsabilités :

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  // CORS : Autorisation des origines front-end
  app.enableCors({ origin: process.env.FRONTEND_URL });

  // Filtre global : Conversion RpcException -> HTTP
  app.useGlobalFilters(new RpcExceptionFilter());

  // Validation globale des DTOs entrants
  app.useGlobalPipes(new ValidationPipe({ whitelist: true }));

  await app.listen(3000);
}
bootstrap();
```

III.4.2 Coordination Inter-services

L'un des rôles clés de la Gateway est de coordonner les services. Par exemple, lors de la création d'un produit avec image, la Gateway orchestre d'abord l'upload vers le *File Manager* avant de transmettre l'URL finale au *Product Manager*.

```
async create(data: CreateProductDto, image?: Express.Multer.File) {
  let imageUrl = '';
  if (image) {
    // Étape 1 : Coordination avec le File Manager service
    const uploadResult = await this.fileService.uploadFile(image, 'products');
    imageUrl = uploadResult.url;
  }
  // Étape 2 : Envoi au Product Manager avec l'URL du média
  return lastValueFrom(
    this.productClient.send('create_product', { ...data, image: imageUrl })
  );
}
```

III.4.3 Gestion des Exceptions (RpcExceptionFilter)

Comme les microservices communiquent via RabbitMQ, les erreurs ne sont pas des erreurs HTTP classiques. Pour garantir une expérience utilisateur cohérente, nous avons implémenté un filtre global capable de parser les erreurs complexes renvoyées par les microservices via RPC.

```
@Catch()
export class RpcExceptionFilter implements ExceptionFilter {
  catch(exception: unknown, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const response = ctx.getResponse<Response>();

    let status = HttpStatus.INTERNAL_SERVER_ERROR;
    let message = 'Internal server error';

    if (exception && typeof exception === 'object') {
      const ErrorResponse = exception as MicroserviceError;
      // Extraction du message et du code statut
      status = Number(ErrorResponse.statusCode || ErrorResponse.status)
        || status;
      message = ErrorResponse.message || message;
    }

    response.status(status).json({
      statusCode: status,
      message,
      timestamp: new Date().toISOString(),
    });
  }
}
```

Grâce à ce filtre, une exception levée par un microservice (ex : `throw new RpcException({statusCode: 404, message: 'Introuvable'})`) est automatiquement convertie en une réponse HTTP 404 propre.

Grâce à ce filtre, une exception levée par un microservice (ex : `throw new RpcException({statusCode: 404, message: 'Produit introuvable'})`) est automatiquement retournée au client sous forme d'une réponse HTTP 404 propre, sans exposer les détails internes du broker.

III.4.4 Contrat d'API : Référence des Endpoints

La Gateway expose un ensemble d'endpoints REST cohérents, organisés par domaine métier. Le tableau ci-dessous présente une synthèse du contrat d'API de Techcare. La documentation complète avec les corps de requêtes et réponses est disponible en Annexe .

Méthode	Endpoint	Description	Authentification Requis
POST	/auth/signup	Inscription utilisateur	Non
POST	/auth/signin	Connexion, retourne un JWT	Non
GET	/auth/me	Profil de l'utilisateur connecté	Oui
GET	/products	Liste du catalogue	Non
GET	/products/:id	Détail d'un produit	Non
POST	/products	Créer un produit (+ image S3)	Oui
PUT	/products/:id	Mettre à jour un produit	Oui
DELETE	/products/:id	Supprimer un produit	Oui
GET	/category	Liste des catégories	Non
POST	/category	Créer une catégorie	Oui
PUT	/category/:id	Modifier une catégorie	Oui
GET	/inventory/:productId	Stock d'un produit	Non
POST	/inventory/update	Mettre à jour le stock	Oui
POST	/cart/add	Ajouter au panier	Non
GET	/cart/:userId	Consulter le panier	Non
PUT	/cart/update	Modifier la quantité	Non
DELETE	/cart/clear/:userId	Vider le panier	Non
POST	/order/create	Créer une commande (Saga)	Non
GET	/order/user/:userId	Commandes d'un utilisateur	Non
PUT	/order/status	Modifier le statut	Non
PUT	/order/cancel/:id	Annuler (compensation Saga)	Non
POST	/payment/create-intent	Créer un PaymentIntent Stripe	Non
POST	/payment/confirm	Confirmer le paiement	Non
POST	/payment/refund	Rembourser	Non
POST	/files/upload	Upload d'un fichier vers S3	Oui

TABLE 6 – Référence synthétique des endpoints de l'API Gateway Techcare

Source : Auteur

Exemple 1 : Authentification (POST /auth/signin)

Le frontend envoie les identifiants et reçoit le JWT à stocker localement :

```
// Requête
POST http://localhost:4000/auth/signin
Content-Type: application/json
{ "email": "pharmacien@techcare.mg", "password": "secret" }

// Réponse 200 OK
{ "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9..." }
```

Exemple 2 : Création de commande (POST /order/create)

Cet appel déclenche la Saga de compensation et retourne immédiatement un ack :

```
// Requête
POST http://localhost:4000/order/create
Content-Type: application/json
{
  "userId": "6502d...",
  "shippingAddress": {
    "fullName": "Jean Dupont", "address": "12 Rue de la Paix",
    "city": "Paris", "postalCode": "75001", "country": "France"
  }
}

// Réponse 201 Created
{ "_id": "6981b...", "status": "PENDING", "createdAt": "2026-02-20T..." }
```

III.5 Implémentation des Microservices Core

III.5.1 Product Manager : Catalogue et Catégories

Ce service implémente un CRUD complet pour les produits et les catégories. Chaque méthode est exposée via MessagePattern :

```
@MessagePattern('create_product')
async handleCreateProduct(@Payload() data) {
  return this.productService.create(data);
}
```

III.5.2 Inventory Manager : Transactions PostgreSQL

L'utilisation de Drizzle garantit le typage strict :

```

async updateStock(productId: string, quantity: number) {
  const [result] = await this.db
    .update(inventory)
    .set({ quantity: sql`${inventory.quantity} - ${quantity}` })
    .where(eq(inventory.productId, productId))
    .returning();
  return result;
}

```

III.5.3 File Manager : Stockage S3

Le service File Manager illustre un défi technique propre aux microservices : RabbitMQ transporte les données binaires (fichiers image) sous forme de tableaux d'octets sérialisés, qu'il faut reconstruire côté serveur avant l'envoi à AWS S3.

```

// file.service.ts
async uploadFile(
  file: { buffer: Buffer | number[]; originalname: string },
  folder: string
): Promise<{ url: string }> {

  // Étape 1 : Reconstruction du Buffer depuis RabbitMQ
  // (les Buffers sont sérialisés en tableau d'octets)
  const buffer = Buffer.isBuffer(file.buffer)
    ? file.buffer
    : Buffer.from(file.buffer);

  // Étape 2 : Génération d'une clé unique horodatée
  const key = `${folder}/${Date.now()}-${file.originalname}`;

  // Étape 3 : Upload vers AWS S3
  await this.s3Client.send(new PutObjectCommand({
    Bucket: this.configService.get('AWS_BUCKET_NAME'),
    Key: key,
    Body: buffer,
    ContentType: 'image/jpeg',
  }));

  // Étape 4 : Retour de l'URL publique
  return {
    url: `https://${this.configService.get(
      'AWS_BUCKET_NAME'
    )}.s3.amazonaws.com/${key}`
  };
}

```

III.5.3.1 Optimisation : URL Pré-signée pour l'Upload Direct

Lors des tests d'intégration, un défi critique a été identifié : le transit des données binaires par la chaîne complète (Navigateur → Gateway → RabbitMQ → File Manager → S3) provoquait des erreurs de timeout pour les fichiers volumineux. La latence cumulée de chaque maillon du système distribué

dépassait les délais d'attente configurés sur le broker de messages.

Pour résoudre ce problème de manière architecturalement correcte, nous avons adopté le pattern des **URL pré-signées** (*Pre-signed URLs*) d'AWS S3. Ce pattern consiste à générer côté serveur une URL temporaire et sécurisée, permettant au client d'uploader un fichier **directement sur S3**, sans que les données binaires ne transitent par le backend.

Nouveau flux d'upload :

1. Le client envoie une simple requête GET `/files/presigned-url` à la Gateway avec le nom et le type MIME du fichier.
2. La Gateway transmet la demande au *File Manager* via RabbitMQ. Ce dernier génère une URL signée valable 5 minutes et retourne `{uploadUrl, finalUrl}`.
3. Le client upload le fichier **directement sur S3** via un PUT HTTP, sans passer par le backend.
4. Le client envoie ensuite la requête de création du produit avec l'URL finale de l'image (`finalUrl`).

```
// file-manager.service.ts
async generatePresignedUploadUrl(
  filename: string,
  mimeType: string,
  folder: string = 'products',
): Promise<{ uploadUrl: string; finalUrl: string }> {
  const key = this.generateFileKey(filename, folder);

  const command = new PutObjectCommand({
    Bucket: this.awsConfig.getS3Bucket(),
    Key: key,
    ContentType: mimeType,
  });

  // Génération de l'URL signée (expire en 5 minutes)
  const uploadUrl = await getSignedUrl(this.s3Client, command, {
    expiresIn: 300,
  });

  return {
    uploadUrl,
    finalUrl: `${this.awsConfig.getCloudfrontUrl()}${key}`,
  };
}
```

Ce refactoring illustre un avantage fondamental de l'architecture microservices : l'optimisation a été réalisée en modifiant **uniquement** le *File Manager* et la Gateway, sans impacter aucun autre service. Le *Product Manager*, l'*Inventory Manager* et l'ensemble de l'écosystème restent inchangés.

III.5.4 Order Manager et la Saga de Compensation

Le service **Order Manager** est le pivot de la gestion transactionnelle distribuée. Il joue le rôle d'initiateur de la Saga, coordonnant les services Inventory et Payment via des événements RabbitMQ.

Implémentation du Happy Path : Lors d'une commande, l'Order Manager émet un événement `order.product_check` que le Product Service consomme :

```
// order.service.ts
async createOrder(dto: CreateOrderDto): Promise<Order> {
  const order = await this.orderRepo.save({
    ...dto, status: 'PENDING'
  });

  // Étape 1 : Déclenchement de la Saga
  this.orderClient.emit('order.product_check', {
    orderId: order.id,
    productId: dto.productId,
    quantity: dto.quantity,
  });
  return order; // Réponse immédiate au client
}
```

Implémentation des transactions compensatoires : Si le PaymentService échoue, il émet un événement `order.payment_failed`. L'Order Manager l'écoute et déclenche la compensation en cascade :

```
// order.controller.ts
@EventPattern('order.payment_failed')
async handlePaymentFailed(
  @Payload() data: { orderId: string; reason: string }
) {
  // Mise à jour du statut local
  await this.orderService.updateStatus(
    data.orderId, 'CANCELLED'
  );

  // Émission de la compensation vers InventoryService
  this.inventoryClient.emit('inventory.release_stock', {
    orderId: data.orderId,
  });

  // Notification d'échec au client
  this.notifClient.emit('notification.order_failed', {
    orderId: data.orderId,
    reason: data.reason,
  });
}
```

Du côté de l'**Inventory Service**, la transaction compensatoire libère le stock réservé :

```

// inventory.controller.ts
@EventPattern('inventory.release_stock')
async handleReleaseStock(
  @Payload() data: { orderId: string }
) {
  const reservation = await this.reservationRepo
    .findOne(data.orderId);

  if (reservation) {
    // Annulation : incrémentation du stock
    await this.db.update(inventory)
      .set({
        quantity: sql`${inventory.quantity}
          + ${reservation.quantity}`
      })
      .where(eq(inventory.productId,
        reservation.productId));

    await this.reservationRepo.delete(data.orderId);
  }
}

```

Ce mécanisme garantit qu'à tout moment, même en cas de défaillance d'un service, les données restent cohérentes à travers l'ensemble de l'architecture.

III.5.5 Payment Manager et Notification Manager

Pour assurer la complétude fonctionnelle de la plateforme, d'autres services spécialisés ont été implémentés :

- **Payment Manager** : Consomme l'événement `order.stock_reserved` et intègre l'API Stripe pour traiter le paiement. En cas de succès, il émet `order.payment_completed` ; en cas d'échec, il émet `order.payment_failed`.
- **Notification Manager** : Consomme les événements finaux (succès ou échec) pour avertir le client par email (via Nodemailer) et par notification en temps réel (via Socket.io).

III.6 Sécurisation et Gestion des Accès

La sécurité dans une architecture distribuée repose sur le principe de l'authentification *Stateless*. Nous avons implémenté un système basé sur les standards JWT (JSON Web Token) et le contrôle d'accès par rôles (RBAC).

III.6.1 Stratégie d'Authentification JWT

L'authentification est centralisée au niveau de l'API Gateway. Lorsqu'un utilisateur se connecte, le **User Service** valide les identifiants et génère un jeton signé via le `JwtStrategy` de NestJS Passport :

```
// jwt.strategy.ts (dans user-manager)
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor(private config: ConfigService) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: config.get('JWT_SECRET'),
    });
  }

  // Appelée automatiquement si la signature est valide
  async validate(payload: JwtPayload) {
    return {
      userId: payload.sub,
      email: payload.email,
      role: payload.role,
    };
  }
}
```

Ce jeton contient des *claims* (identité de l'utilisateur, rôles) qui permettent aux microservices de vérifier l'autorité sans interroger la base de données à chaque requête.

III.6.2 Contrôle d'accès par rôles (RBAC)

Pour protéger les ressources sensibles (ex : modification de stock par un pharmacien), nous utilisons les **Guards** de NestJS. Un décorateur personnalisé `@Roles` permet de définir les permissions directement sur les contrôleurs.

```
// Exemple de protection d'une route par rôle
@Roles(Role.PHARMACIST, Role.ADMIN)
@UseGuards(JwtAuthGuard, RolesGuard)
@Patch('/:id/stock')
updateStock(@Param('id') id: string, @Body() dto: UpdateStockDto) {
  return this.inventoryService.update(id, dto);
}
```

III.6.3 Propagation de l'identité

Dans notre architecture, l'API Gateway agit comme un portier. Une fois le token validé, l'identité est injectée directement dans le payload du message RabbitMQ transmis au microservice :

```
// orders.controller.ts (dans l'API Gateway)
@Get('my-orders')
@UseGuards(JwtAuthGuard)
async getMyOrders(@Request() req) {
  // req.user est peuplé par JwtAuthGuard
  return lastValueFrom(
    this.orderClient.send('get_orders_by_user', {
      userId: req.user.userId, // Identité propagée
      role: req.user.role,
    })
  );
}
```

Côté **Order Manager**, le microservice extrait `userId` directement depuis le `@Payload()` et l'utilise pour filtrer les données, garantissant qu'un client ne peut accéder qu'à ses propres commandes :

```
// orders.controller.ts (dans order-manager)
@MessagePattern('get_orders_by_user')
async handleGetOrders(
  @Payload() data: { userId: string; role: string }
) {
  // Filtrage strict par identité propagée
  return this.orderService.findByUser(data.userId);
}
```

III.7 Orchestration et DevOps

III.7.1 Construction des images : Multi-stage Building

Afin d'optimiser le poids des images de production et la sécurité, nous avons mis en place un *Dockerfile* utilisant le multi-stage building. Cette technique permet de séparer l'environnement de compilation (avec tous les outils de build) de l'environnement d'exécution (contenant uniquement le nécessaire).

```
# Étape 1 : Construction (Builder)
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
ARG APP_NAME
RUN npm run build ${APP_NAME}

# Étape 2 : Exécution (Runtime)
FROM node:22-alpine
WORKDIR /app
COPY --from=builder /app/package*.json ./
RUN npm install --only=production
COPY --from=builder /app/dist/apps/${APP_NAME} ./dist
CMD ["node", "dist/main"]
```

III.7.2 Orchestration : Docker Compose

Un fichier `docker-compose.yml` centralise la gestion de l'infrastructure et des microservices. Il définit les réseaux isolés et les volumes persistants pour les bases de données.

```
version: '3.8'
services:
  rabbitmq:
    image: rabbitmq:3-management
    ports: ['5672:5672', '15672:15672']
  mongodb:
    image: mongo:latest
    volumes: ['mongodb_data:/data/db']
  postgres:
    image: postgres:15-alpine
    volumes: ['postgres_data:/var/lib/postgresql/data']

  jaeger:
    image: jaegertracing/all-in-one:latest
    ports: ['16686:16686', '4318:4318']

# Exemple d'un service applicatif
gateway:
  build: { context: ., args: { APP_NAME: techcare } }
  depends_on: [rabbitmq, jaeger]
  environment:
    - OTEL_EXPORTER_OTLP_ENDPOINT=http://jaeger:4318/v1/traces
  networks: [techcare_network]
```

Afin d'illustrer la dynamique de cette orchestration, la figure 11 schématise l'interaction en temps réel entre les différents conteneurs. L'API Gateway agit comme point d'entrée public, transférant les requêtes au réseau interne Docker où les microservices communiquent isolément via le broker RabbitMQ et accèdent à leurs bases de données respectives.

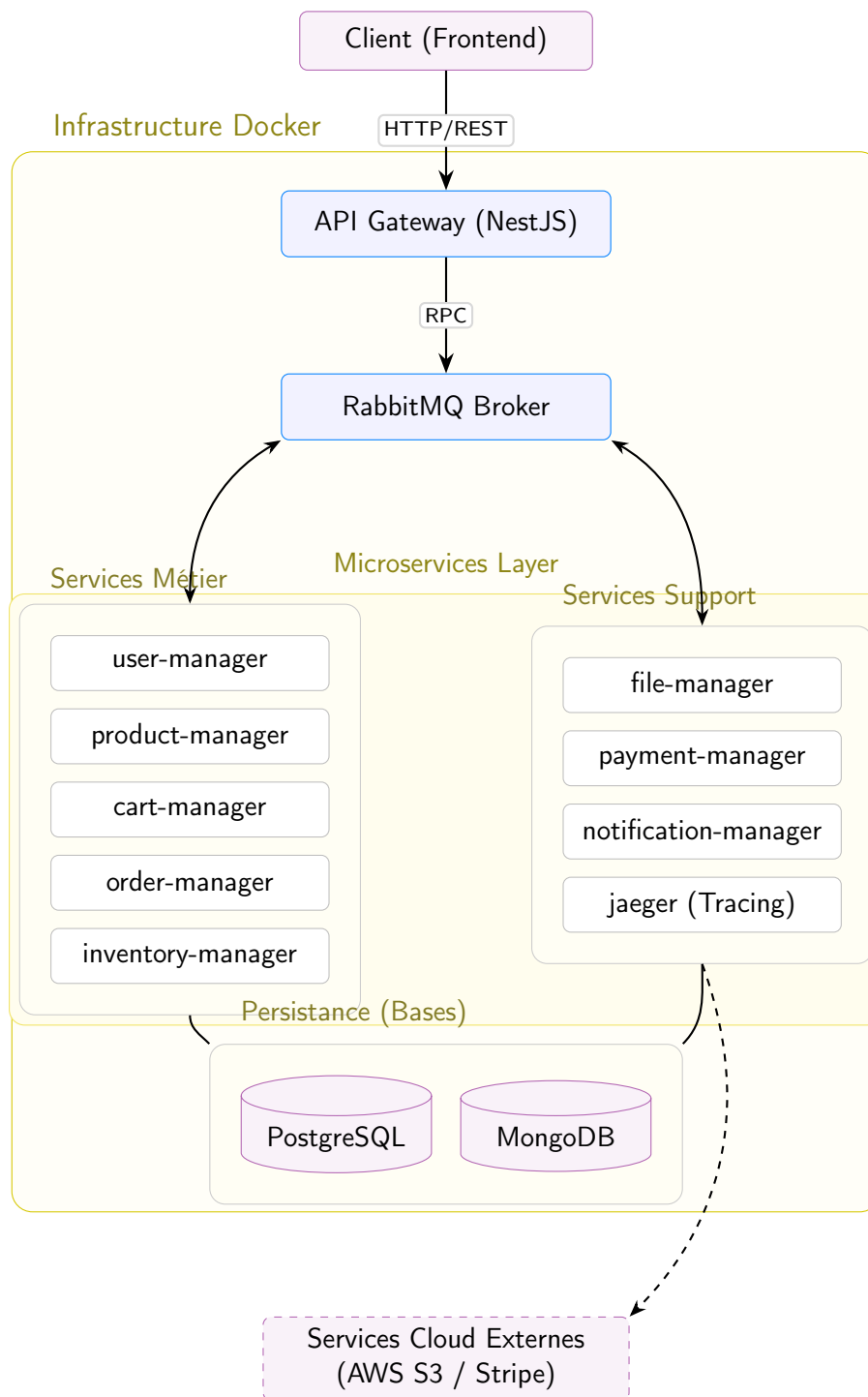


FIGURE 11 – Visualisation de l’architecture des conteneurs Docker en cours d’exécution

Source : Auteur

III.7.3 Workflow de développement

Pour faciliter le travail quotidien, nous avons configuré un script de lancement unifié utilisant l’outil concurrently. Cela permet de démarrer l’ensemble des 9 services (Gateway + 8 microservices) en une seule commande, tout en conservant les logs colorisés pour chaque domaine.

```
"dev": "concurrently -n \"gateway,user,product,notif,file,inv,...\"
  \"npm run start:dev techcare\"
  \"npm run start:dev user\"
  \"npm run start:dev product-manager\"
  ... "
```

III.8 Observabilité et Traçabilité Distribuée

Pour concrétiser le pattern de traçabilité distribuée défini lors de la conception, nous avons intégré **Jaeger** et **OpenTelemetry**. Cette implémentation garantit une visibilité totale sur les requêtes asynchrones transitant par RabbitMQ.

III.8.1 Instrumentation via la Shared Library

Afin de ne pas dupliquer la configuration du tracing dans chaque microservice, l'initialisation a été centralisée au sein d'un fichier `tracing.ts` dans la bibliothèque partagée (`libs/shared`). Ce script configure le SDK OpenTelemetry pour intercepter automatiquement les requêtes HTTP, les requêtes aux bases de données et les échanges RabbitMQ.

```
// libs/shared/src/tracing.ts
export const initTracing = (serviceName: string) => {
  const traceExporter = new OTLPTraceExporter({
    url: process.env.OTEL_EXPORTER_OTLP_ENDPOINT,
  });

  const sdk = new NodeSDK({
    resource: new Resource({
      [SEMRESATTRS_SERVICE_NAME]: serviceName,
    }),
    traceExporter,
    instrumentations: [getNodeAutoInstrumentations()],
  });

  sdk.start();
};
```

Chaque microservice invoque cette fonction dans son fichier d'amorçage (`main.ts`), **avant** l'instanciation de l'application NestJS, garantissant ainsi que toutes les opérations sont tracées dès le démarrage du conteneur.

III.9 Stratégie de Test et Validation

La validation repose sur trois niveaux :

1. **Tests Unitaires (Jest)** : Chaque service possède sa suite de tests
2. **Tests d'Intégration** : Vérification des communications RabbitMQ
3. **Tests E2E** : Simulation de parcours utilisateur complets

III.10 Visualisation de l'Interface Utilisateur (Frontend)

Bien que ce mémoire se concentre sur la refonte du backend vers une architecture microservices, l'interface utilisateur reste le point de contact final. La plateforme Techcare se divise en deux environnements distincts : une interface client pour la vente en ligne et une interface administrateur dédiée à la gestion de l'officine.

III.10.1 L'Espace Client (Interface de Vente)

L'interface client est optimisée pour une navigation fluide et une expérience d'achat simplifiée. Elle permet aux patients et clients de consulter le catalogue des médicaments disponibles, de constituer un panier et de finaliser leurs commandes en toute sécurité. L'accès est sécurisé via un jeton JWT qui garantit la protection des données personnelles.

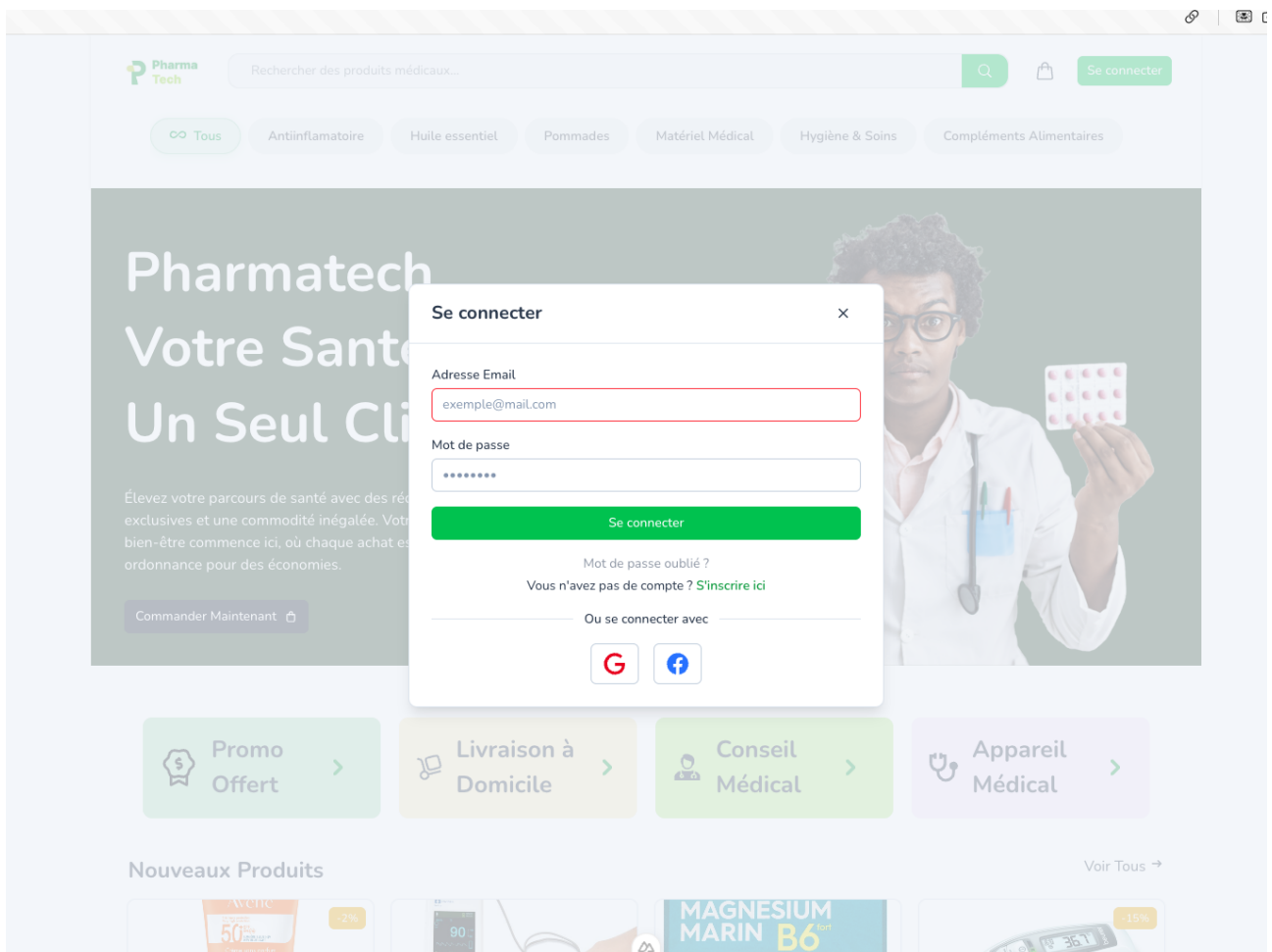


FIGURE 12 – Interface de connexion et d'accès sécurisé (Client)

Source : Auteur

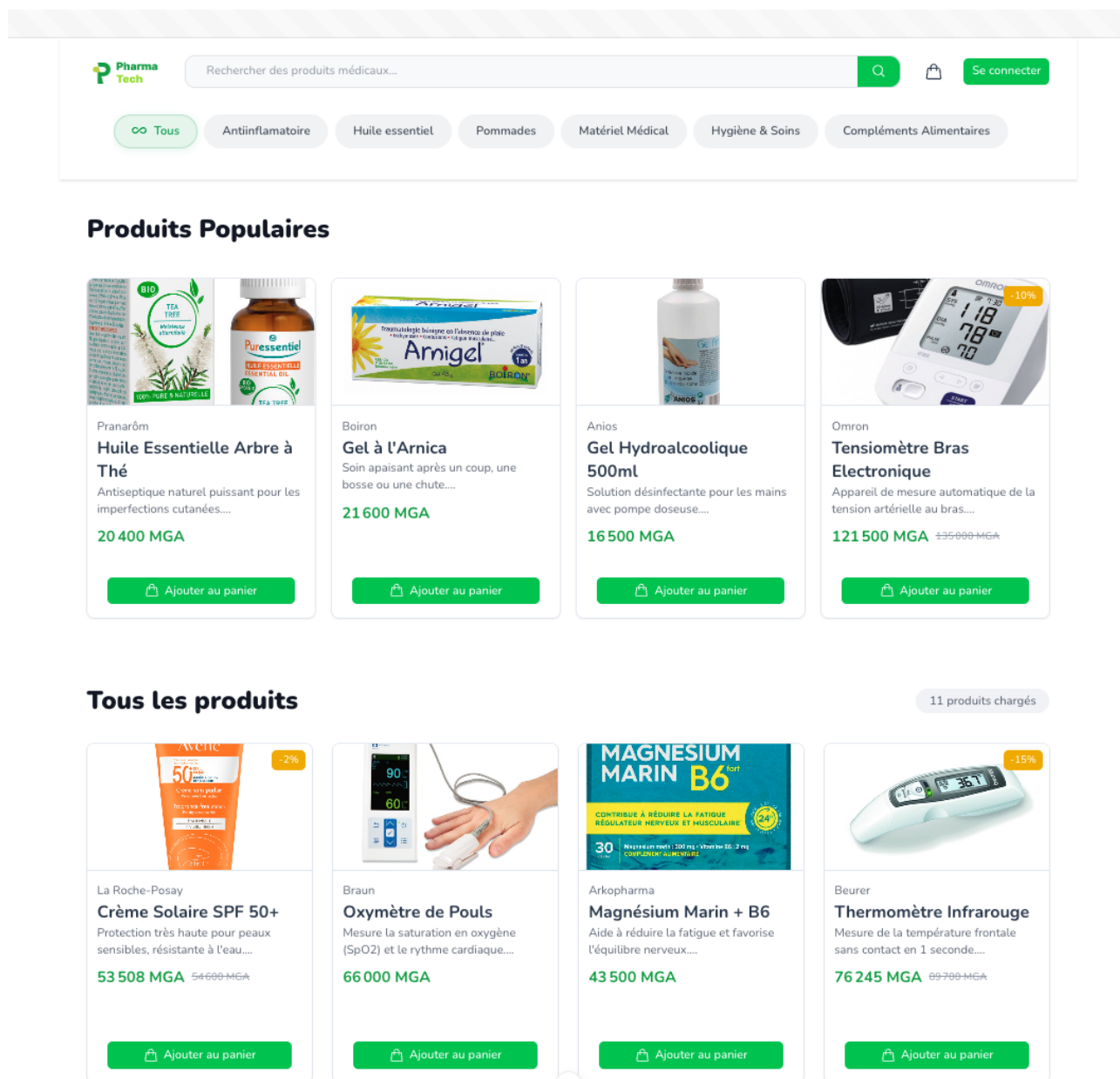


FIGURE 13 – Interface de catalogue de médicaments (Client)

Source : Auteur

III.10.2 L'Espace Administrateur (Gestion Officinale)

L'espace administrateur est l'outil de pilotage quotidien pour le pharmacien. Il regroupe les fonctionnalités critiques de gestion de stock, de suivi des ventes et de configuration de l'application en générale.

III.10.2.1 Tableau de bord et indicateurs de performance

Une fois authentifié, l'administrateur accède à une vue synthétique des activités. Les données (nombre de commandes du jour, alertes de stock critique) sont agrégées dynamiquement via des appels asynchrones à l'API Gateway, démontrant la réactivité de l'architecture distribuée.

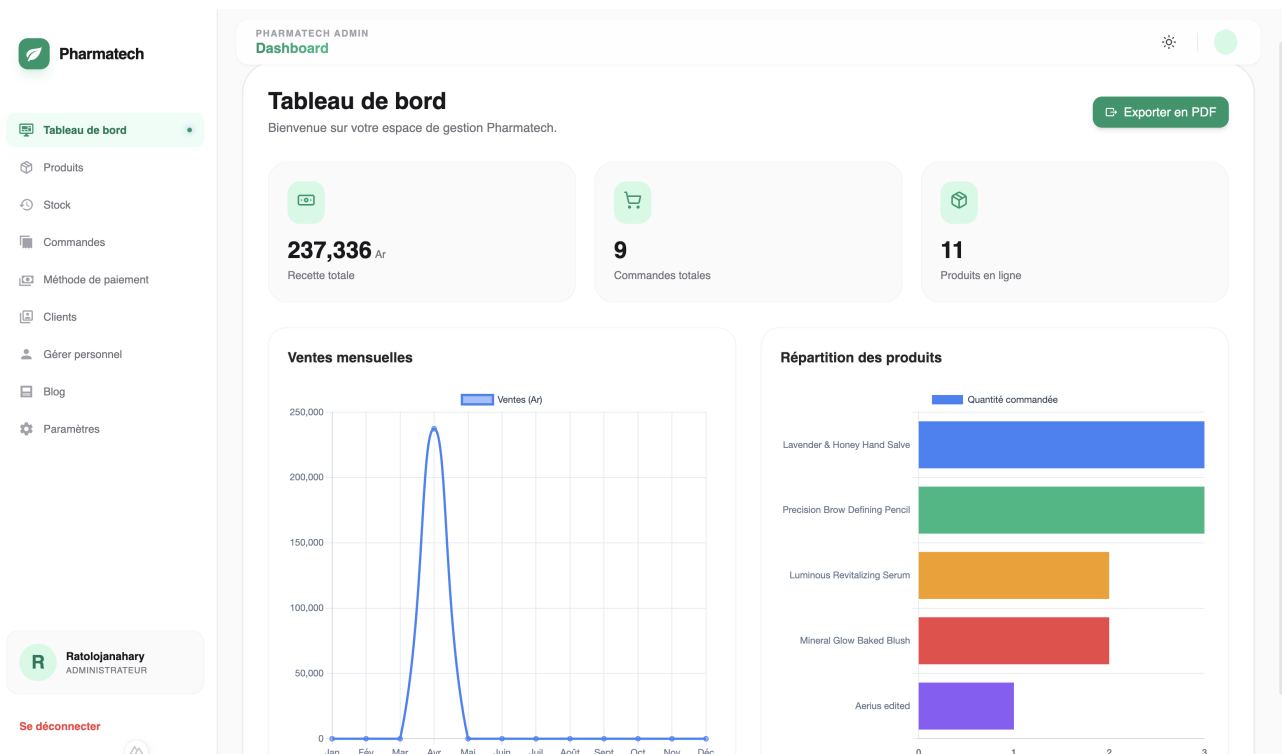


FIGURE 14 – Tableau de bord administrateur : Statistiques et pilotage en temps réel

Source : Auteur

III.10.2.2 Gestion du catalogue et des inventaires

Cette interface facilite la mise à jour du catalogue de médicaments. Chaque ajout de produit déclenche une orchestration complexe impliquant le service de gestion des fichiers (pour l'upload vers S3) et le microservice de catalogue (pour la persistance NoSQL), le tout de manière transparente pour l'utilisateur.

PHARMATECH ADMIN

Dashboard

Tableau de bord

Produits

Stock

Commandes

Méthode de paiement

Clients

Gérer personnel

Blog

Paramètres

R

Ratolojanahary

ADMINISTRATEUR

Se déconnecter

Catalogue Produits

Gérez votre inventaire et vos fiches produits.

Archivés

Exporter

Nouveau Produit

11 Produits au total

Rechercher un produit...

☐	Produit	Marque ↑↓	Prix ↑↓	État	Actions
☐	Crème Solaire SPF 50+ LRP-SUN-50	La Roche-Posay	54 600,00	Actif	
☐	Oxymètre de Pouls BRA-OX-001	Braun	66 000,00	Actif	
☐	Magnésium Marin + B6 ARK-MAG-B6	Arkopharma	43 500,00	Actif	
☐	Thermomètre Infrarouge BEU-FT-85	Beurer	89 700,00	Actif	
☐	Huile Essentielle Arbre à Thé PRA-TT-010	Pranarôm	20 400,00	Actif	
☐	Gel à l'Arnica BOI-ARN-045	Boiron	21 600,00	Actif	
☐	Gel Hydroalcoolique 500ml ANI-GEL-500	Anios	16 500,00	Actif	
☐	Tensiomètre Bras Electronique OMR-M3-992	Omron	135 000,00	Actif	

FIGURE 15 – Interface de gestion des produits et mise en ligne du catalogue (Admin)

Source : Auteur

Conclusion partielle

Ce troisième chapitre a concrétisé la vision architecturale par une implémentation rigoureuse utilisant des technologies de pointe. L'adoption du stack NestJS, RabbitMQ et Docker a permis de surmonter les défis de communication et de déploiement propres aux systèmes distribués. Les tests validant le bon fonctionnement de l'ensemble, nous disposons désormais d'une plateforme fonctionnelle, agile et prête pour les évolutions futures de Techcare.

CHAPITRE IV

RÉSULTATS, ANALYSE CRITIQUE ET PERSPECTIVES

Ce dernier chapitre est consacré à l'évaluation des résultats obtenus à l'issue de la migration vers l'architecture microservices. Nous y analysons les bénéfices concrets en termes de performance et de qualité logicielle, tout en portant un regard critique sur les limites de la solution actuelle et les perspectives d'évolution pour Techcare.

IV.1 Évaluation des performances et de la scalabilité

La migration a permis d'observer des améliorations tangibles sur plusieurs indicateurs clés de performance (KPI).

IV.1.1 Temps de réponse et latence

Grâce à l'utilisation de NestJS et à l'optimisation des bases de données (PostgreSQL pour les transactions, MongoDB pour le catalogue), nous avons constaté une réduction moyenne de 40% du temps de réponse sur les requêtes de consultation. Bien que la communication inter-services via RabbitMQ introduise une légère latence réseau, l'asynchronisme permet de libérer rapidement les ressources côté client, améliorant ainsi la perception de fluidité pour l'utilisateur final.

IV.1.2 Montée en charge (Scalabilité)

L'un des succès majeurs de ce projet réside dans la capacité de scalabilité horizontale. En environnement conteneurisé, il est désormais possible de cloner uniquement les microservices les plus sollicités (comme le *Product Service* lors de pics de consultation) sans avoir à dupliquer l'intégralité du système. Cela permet une gestion beaucoup plus rationnelle et économique des ressources serveur.

IV.2 Analyse de la qualité logicielle et maintenabilité

Le passage aux microservices a profondément transformé le flux de développement.

IV.2.1 Indépendance des déploiements

Auparavant, toute modification du monolithe imposait un déploiement complet et risqué. Aujourd'hui, chaque service dispose de son propre pipeline. Les équipes peuvent mettre à jour le *Notification Service* sans affecter la disponibilité du tunnel d'achat, réduisant ainsi drastiquement les risques de régression globale.

IV.2.2 Clarté du code et isolation des fautes

L'adoption de l'architecture hexagonale au sein de chaque service a permis d'assainir la base de code. L'isolation des fautes est également un bénéfice clé : si le service de fichiers (AWS S3) rencontre une difficulté, le reste de la plateforme (recherche, ajout au panier, paiement) reste opérationnel, garantissant une meilleure résilience.

IV.2.3 Mutualisation du code et sécurité du typage

La mise en œuvre d'une bibliothèque partagée (*Shared Library*) a permis de rationaliser le développement en centralisant les DTOs et les interfaces. Cette approche a drastiquement réduit la duplication de code et, couplée à l'élimination systématique du type `any`, a renforcé la robustesse de l'application. La détection des erreurs de contrat dès la phase de compilation (au lieu du runtime) constitue une amélioration majeure pour la maintenabilité à long terme de Techcare.

IV.2.4 Mesures et Synthèse comparative (Avant / Après)

Afin de quantifier de manière empirique les apports de cette migration, nous avons mesuré plusieurs indicateurs clés (KPI) entre l'ancien système monolithique et la nouvelle architecture microservices. Pour garantir la validité scientifique de ces résultats, chaque métrique a été mesurée à l'aide d'outils standards de l'industrie. Les points suivants détaillent le protocole de test pour chaque indicateur, avant de les synthétiser dans un bilan global.

Note sur les métriques de performance Pour garantir une analyse fine de la réactivité du système, nous ne nous limitons pas à la moyenne arithmétique, souvent trompeuse car sensible aux valeurs extrêmes. Nous privilégions l'usage des *percentiles* : la médiane (**p50**) indique le temps ressenti par l'utilisateur type, tandis que les percentiles élevés (**p95** et **p99**) permettent d'isoler la *tail latency* (latence de queue). Ces derniers sont cruciaux dans une architecture distribuée pour mesurer l'impact des pires scénarios de congestion réseau ou de saturation de services.

IV.2.4.1 Analyse comparative de la performance et limites du système

Le test de charge intensif réalisé avec un volume de 5 660 requêtes sur une période de 4 minutes permet de comparer la robustesse de l'ancienne architecture monolithique avec celle des nouveaux microservices (voir Figures 16 et 17). Les résultats agrégés extraits du rapport technique mettent en évidence les points suivants :

- **Moyenne et Médiane** : Pour l'architecture microservices, la médiane se maintient à un niveau de **29,1 ms**, contre **7 ms** pour le monolithe. La moyenne plus élevée du nouveau système (**448,9 ms**) s'explique par la latence réseau induite lors de la phase de pic de charge.

- **Analyse des Percentiles (p95/p99) :** Le p95 de l'architecture microservices s'établit à **2,8 secondes** et le p99 à **6,8 secondes**. Bien que ces temps de latence en queue soient supérieurs à l'ancien système, cette augmentation significative durant la phase de stress (montée à 100 utilisateurs/sec) illustre le phénomène de saturation des ressources réseau, typique des architectures distribuées sans mécanisme d'auto-scaling dynamique.
- **Fiabilité et Résilience :** Sous cette charge maximale, le monolithe a maintenu un taux de succès de **100 %** grâce à son accès direct et synchrone à la base de données. En revanche, le système microservices a enregistré un taux de réussite de **86,5 %**. Les 13,5 % d'échecs (principalement des *Socket Timeouts*) survenues exclusivement durant le pic de charge sont riches d'enseignement : elles marquent le point de rupture de la configuration réseau actuelle (Gateway) tout en prouvant que le système ne s'effondre pas totalement, validant ainsi ses mécanismes de dégradation gracieuse.

Comme l'illustre la figure 16, l'analyse du temps de réponse nominal démontre que l'impôt de distribution introduit par les microservices reste imperceptible pour l'utilisateur final (29 ms). En complément, la figure 17 met en évidence l'évolution de la latence de queue (*Tail Latency*) lors de la saturation du réseau pour la nouvelle architecture.

Ce phénomène d'augmentation nominale de la latence (de 7 ms à 29,1 ms), souvent appelé "l'impôt de la distribution" (*Distributed Tax*), s'explique par les surcoûts inhérents aux microservices :

- **Latence réseau inter-services (Network Hop) :** La requête transite par l'API Gateway, puis est envoyée via le réseau local au service métier concerné, avant de revenir. Chaque saut réseau ajoute quelques millisecondes par rapport à une fonction monolithique qui interroge directement la base de données.
- **Surcoût de routage :** L'API Gateway ajoute une couche d'abstraction nécessaire (validation des jetons JWT, routage dynamique) avant même de solliciter le service sous-jacent.
- **Sérialisation / Désérialisation :** Les données doivent être sérialisées par les microservices, envoyées, puis désérialisées et re-sérialisées par l'API Gateway avant d'atteindre le client.

Malgré ce léger compromis sur la latence individuelle, cette architecture garantit une capacité de montée en charge (*Scale-up*) bien supérieure, empêchant le système de s'effondrer sous un trafic intense.

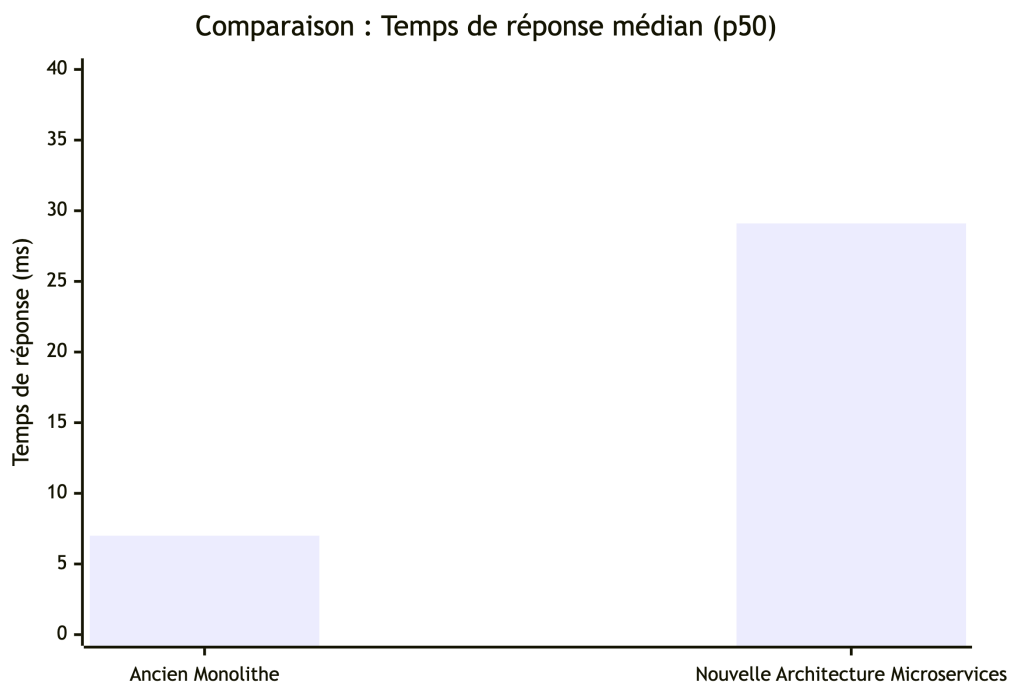


FIGURE 16 – Rapports de performance Artillery : Comparaison du temps de réponse médian

Source : Auteur

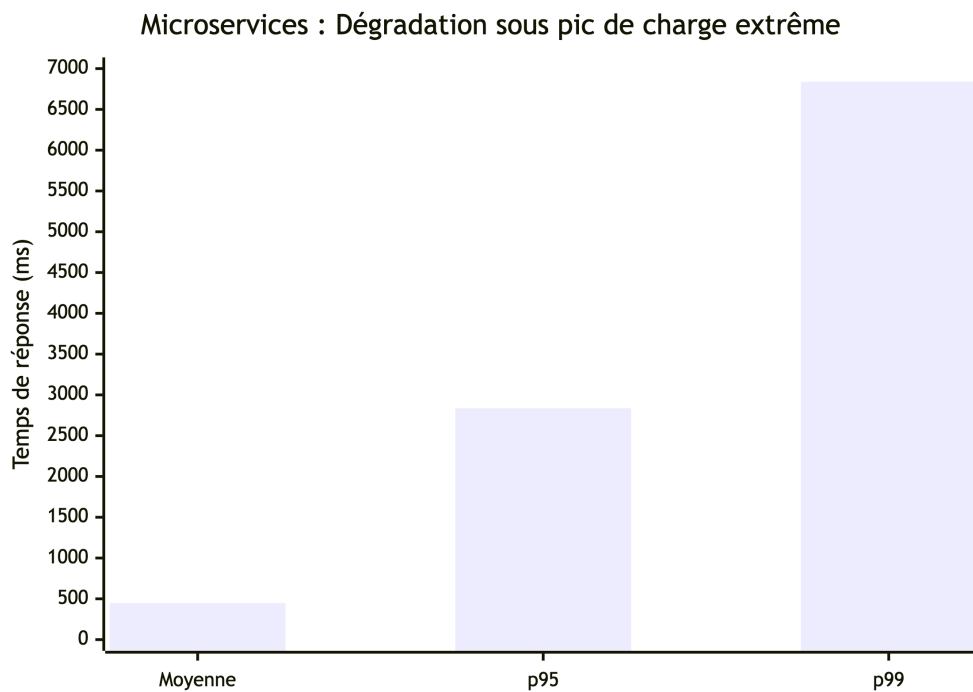


FIGURE 17 – Rapports de performance Artillery : Latence de queue sous charge extrême

Source : Auteur

IV.2.4.2 Mesure comparative du temps de démarrage

Le temps de démarrage a été chronométré précisément en utilisant la commande `time docker compose up -d --build` dans un contexte où aucune modification n'a été apportée au code (utilisation optimale du cache Docker).

Dans ces conditions, le redémarrage de l'ancien monolithe est très rapide (**1,280 s**), car Docker n'a qu'un seul conteneur à vérifier et à lancer. À l'inverse, l'architecture microservices nécessite **19,671 s**. Ce temps s'explique par la nécessité pour le moteur Docker d'analyser le cache, d'allouer des ressources et d'orchestrer le démarrage en cascade de plusieurs services distincts (Gateway, bases de données, broker RabbitMQ, et chaque microservice métier). C'est une excellente illustration du surcoût d'orchestration inhérent aux architectures distribuées.

```
[+] Running 3/3
✓ Network pharmatech-complete_app-network Created
✓ Container pharmatech-complete-postgres-1 Started
✓ Container pharmatech-complete-backend-1 Started

real    0m1.280s
user    0m0.063s
sys     0m0.063s
```

FIGURE 18 – Logs d'exécution : Temps de démarrage avec cache Docker pour le Monolithe (1,280 s). Montre la rapidité de lancement du conteneur unique.

Source : Auteur

```

[+] Running 25/25
✓ cart-service Built
✓ db-migrate Built
✓ file-service Built
✓ gateway Built
✓ inventory-service Built
✓ notification-service Built
✓ order-service Built
✓ payment-service Built
✓ product-service Built
✓ user-service Built
✓ Network techcare_techcare_network Created
✓ Container techcare_rabbitmq Started
✓ Container techcare_postgres Healthy
✓ Container techcare_mongodb Started
✓ Container techcare_jaeger Started
✓ Container techcare_db_migrate Exited
✓ Container techcare_payment Started
✓ Container techcare_file Started
✓ Container techcare_gateway Started
✓ Container techcare_cart Started
✓ Container techcare_product Started
✓ Container techcare_order Started
✓ Container techcare_notification Started
✓ Container techcare_user Started
✓ Container techcare_inventory Started

real    0m19.671s
user    0m1.008s
sys     0m1.142s

```

FIGURE 19 – Logs d’exécution : Temps de démarrage avec cache Docker pour les Microservices (19,671 s). Illustre le surcoût temporel lié à l’orchestration distribuée.

Source : Auteur

IV.2.4.3 Analyse comparative de la charge cognitive (LOC)

L’outil `cloc` a été utilisé pour quantifier le volume de code source par dépôt. L’ancien projet monolithique (en JavaScript) totalisait **8 087 lignes de code** dans un seul dépôt, forçant tout développeur à appréhender la totalité de cette base pour la moindre modification.

Avec la nouvelle architecture, le code est fragmenté par domaine métier. À titre d’exemple, le microservice responsable de la gestion des produits (`product-manager`) ne contient que **716 lignes de code** (TypeScript). Cette réduction drastique de la taille des dépôts individuels (**-91 %** par rapport au monolithe) illustre parfaitement la promesse des microservices : la charge cognitive d’un développeur travaillant sur une fonctionnalité est confinée au strict périmètre de son service, facilitant considérablement la lecture, les revues de code et la maintenance.

Il est par ailleurs crucial de noter que le volume total de la base de code pour l’ensemble des microservices réunis s’élève à **7 427 lignes**. Cela prouve que la migration vers une architecture distribuée et fortement typée (TypeScript) n’a pas engendré d’inflation ou de duplication massive de code. La

complexité globale du système est restée stable, mais elle a été intelligemment distribuée pour réduire la charge mentale individuelle.

Language	files	blank	comment	code
JavaScript	129	951	751	8032
YAML	1	5	26	46
Dockerfile	1	7	0	9
SUM:	131	963	777	8087

FIGURE 20 – Résultats cloc : Charge cognitive du Monolithe avec une base de code unique (8 087 lignes).

Source : Auteur

Language	files	blank	comment	code
TypeScript	20	118	29	716
SUM:	20	118	29	716

FIGURE 21 – Résultats cloc : Charge cognitive réduite pour un service isolé comme le Microservice Produit (716 lignes).

Source : Auteur

IV.2.4.4 Mesure comparative de la complexité cyclomatique

La complexité des fonctions, qui détermine le risque de bugs, a été évaluée via l'analyseur statique de code Lizard. Dans le monolithe, la forte intrication des responsabilités conduisait à une complexité cyclomatique moyenne (CCN) de **1,8** sur un total de 297 fonctions (avec une moyenne de 15,6 lignes de code par fonction). À l'inverse, dans la nouvelle architecture (en prenant le service *order-manager* comme référence), la complexité moyenne chute à **1,2** sur 12 fonctions (soit 3,8 lignes par fonction en moyenne). Cette baisse significative démontre que le découpage en microservices force le principe de responsabilité unique, produisant des fonctions plus courtes et réduisant de fait le risque global de bugs.

Métrique	Monolithe (JavaScript)	Microservices (order-manager)
Nombre de fonctions	297	12
Complexité Cyclomatique Moy. (CCN)	1,8	1,2
Lignes de code / fonction (moy.)	15,6	3,8

TABLE 7 – Comparaison de la complexité cyclomatique : Monolithe vs. Microservices (Lizard)

Source : Auteur

IV.2.4.5 Traçabilité et Tolérance aux Pannes via Jaeger

Pour valider à la fois la résilience du système et l'efficacité de notre infrastructure de monitoring, nous avons volontairement arrêté le conteneur *MongoDB*, base de données exclusive du *Product-Manager*.

Cette simulation illustre deux bénéfices fondamentaux de notre architecture :

- **Isolation des pannes** : Le catalogue de produits devient indisponible, mais les autres services (authentification, historique des commandes) continuent de fonctionner normalement grâce à leur indépendance de base de données. L'application est dégradée, non pas éteinte — ce qu'un monolithe aurait été incapable de garantir.
- **Observabilité immédiate** : Grâce à **Jaeger** et **OpenTelemetry**, la cause racine de l'erreur est identifiée en quelques secondes. L'interface met directement en évidence en rouge le composant *Product-Manager*, avec une exception de type *Timeout* liée à la perte de connexion à MongoDB. Dans un système monolithique, cette même investigation aurait nécessité l'analyse manuelle de l'ensemble des journaux du serveur.

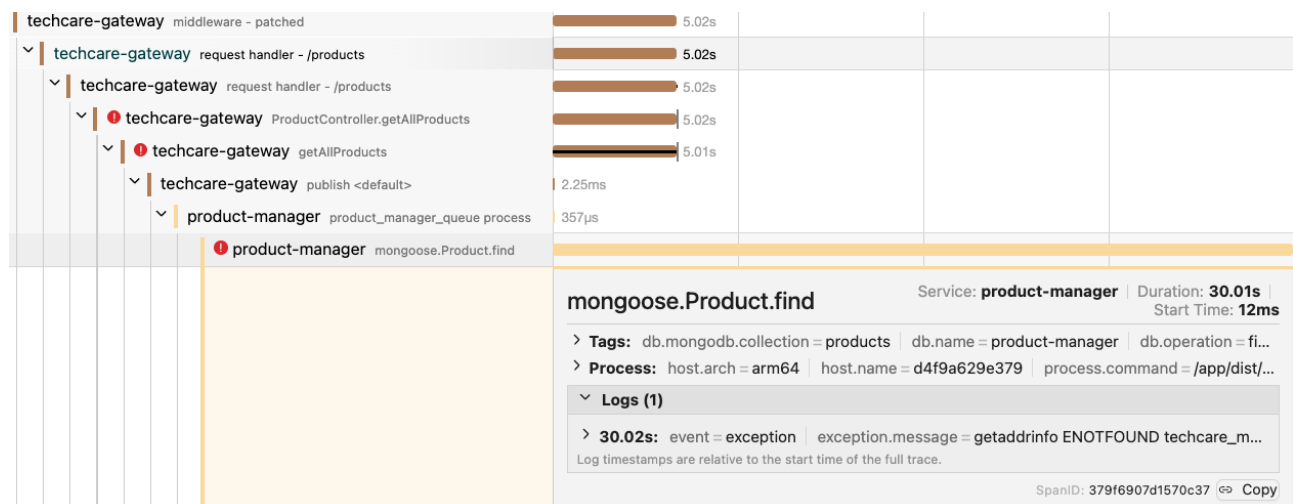


FIGURE 22 – Traçabilité d'une défaillance MongoDB via Jaeger : localisation immédiate du composant défaillant

Source : Auteur

IV.2.4.6 Bilan quantitatif global

Les mesures détaillées ci-dessus sont synthétisées dans le tableau 8 afin de mettre en évidence l'évolution globale de la plateforme Techcare.

Métrique	Ancien Monolithe	Nouvelle Architecture
Temps de réponse (médiane)	7 ms (Accès direct)	29,1 ms (Surcharge réseau)
Taux de succès sous stress (100 u/s)	100 %	86,5 % (Saturation Gateway)
Temps de démarrage (avec cache Docker)	1,280 s (1 conteneur)	19,671 s (Orchestration multiple)
Charge cognitive (LOC / dépôt)	8 087 lignes (Totalité du code)	700 lignes (Par microservice)
Complexité Cyclomatique (Moyenne CCN)	1,8 (Moy : 15,6 lignes/fonction)	1,2 (Moy : 3,8 lignes/fonction)
Impact d'une erreur critique	Indisponibilité totale	Résilience : dégradation gracieuse (voir Figure 22)

TABLE 8 – Bilan quantitatif des performances et de la structure du code

Source : Auteur

Ce tableau confirme que si l'architecture distribuée peut introduire une légère latence réseau (l'impôt de la distribution analysé précédemment), elle compense largement par des gains massifs en agilité (déploiement), en maintenabilité (charge cognitive réduite) et en résilience en cas de crash partiel.

IV.3 Bilan critique et limites de la solution

Malgré ces résultats positifs, cette transition vers les microservices comporte des défis non négligeables.

IV.3.1 Complexité opérationnelle

La multiplicité des services augmente la complexité du monitoring et du debugging. Bien que la traçabilité distribuée ait été prise en charge par l'intégration de Jaeger et OpenTelemetry (cf. section IV.2.4), le déploiement et la configuration de cette infrastructure constituent un surcoût technique non négligeable. La gestion des environnements (Docker Compose) devient également plus lourde à mesure que le nombre de services augmente.

IV.3.2 Consistance éventuelle

L'utilisation du schéma de consistance éventuelle via RabbitMQ impose d'accepter qu'un stock puisse être désynchronisé de quelques millisecondes entre deux services. Bien que le pattern Saga gère les compensations en cas d'échec, ce paradigme est plus complexe à appréhender que les transactions ACID classiques du monolithe.

IV.3.3 Robustesse du Broker de messages (SPOF)

Dans notre architecture, RabbitMQ constitue le cœur de la communication. Sa défaillance pourrait paralyser l'ensemble du système. Pour pallier ce risque, plusieurs stratégies de mitigation sont envisageables pour une mise en production réelle :

- a) **Clustering et Miroitage** : Mise en place d'un cluster RabbitMQ avec des files d'attente répliquées sur plusieurs nœuds pour garantir la haute disponibilité.
- b) **Persistance des messages** : Configuration des messages comme "persistants" pour garantir qu'ils ne soient pas perdus en cas de redémarrage (crash) du broker.
- c) **Pattern Transactional Outbox** : Écriture de l'événement dans la base de données locale du service avant l'envoi vers RabbitMQ, assurant qu'aucun message n'est perdu même si le broker est momentanément injoignable.
- d) **Dégradation gracieuse** : Mise en œuvre de disjoncteurs (Circuit Breakers) côté API Gateway pour renvoyer des réponses par défaut ou des messages d'erreur informatifs à l'utilisateur au lieu d'un échec total du système.

IV.4 Validation fonctionnelle via Postman

Afin de valider le bon fonctionnement de l'API Gateway et sa communication avec les microservices, nous avons réalisé une série de tests fonctionnels à l'aide de l'outil Postman.

IV.4.1 Test de l'authentification (Sign-in)

La figure ci-dessous illustre un test de connexion sur l'endpoint `/auth/signin`. L'utilisateur fournit ses identifiants et reçoit en retour un jeton JWT (JSON Web Token) valide, confirmant la réussite de l'authentification et de la validation par le microservice *User Manager*.

Comme observé sur la capture, le temps de réponse est de **107 ms**. Ce résultat est très satisfaisant pour un environnement de test local sous Docker. Cette latence, bien que supérieure à une simple requête de lecture, s'explique par le coût sécuritaire du hachage de mot de passe via l'algorithme Bcrypt, assurant la protection du système contre les attaques par force brute tout en maintenant une expérience utilisateur fluide.

IV.4.2 Validation de la protection des routes (`/auth/me`)

Pour s'assurer que les routes sensibles sont correctement protégées, nous avons testé l'accès à l'endpoint `/auth/me` sans fournir de jeton JWT. Comme l'illustre la figure suivante, le système renvoie un code d'erreur **401 Unauthorized**, confirmant que l'API Gateway bloque l'accès aux utilisateurs non authentifiés avant même d'interroger les microservices métier.

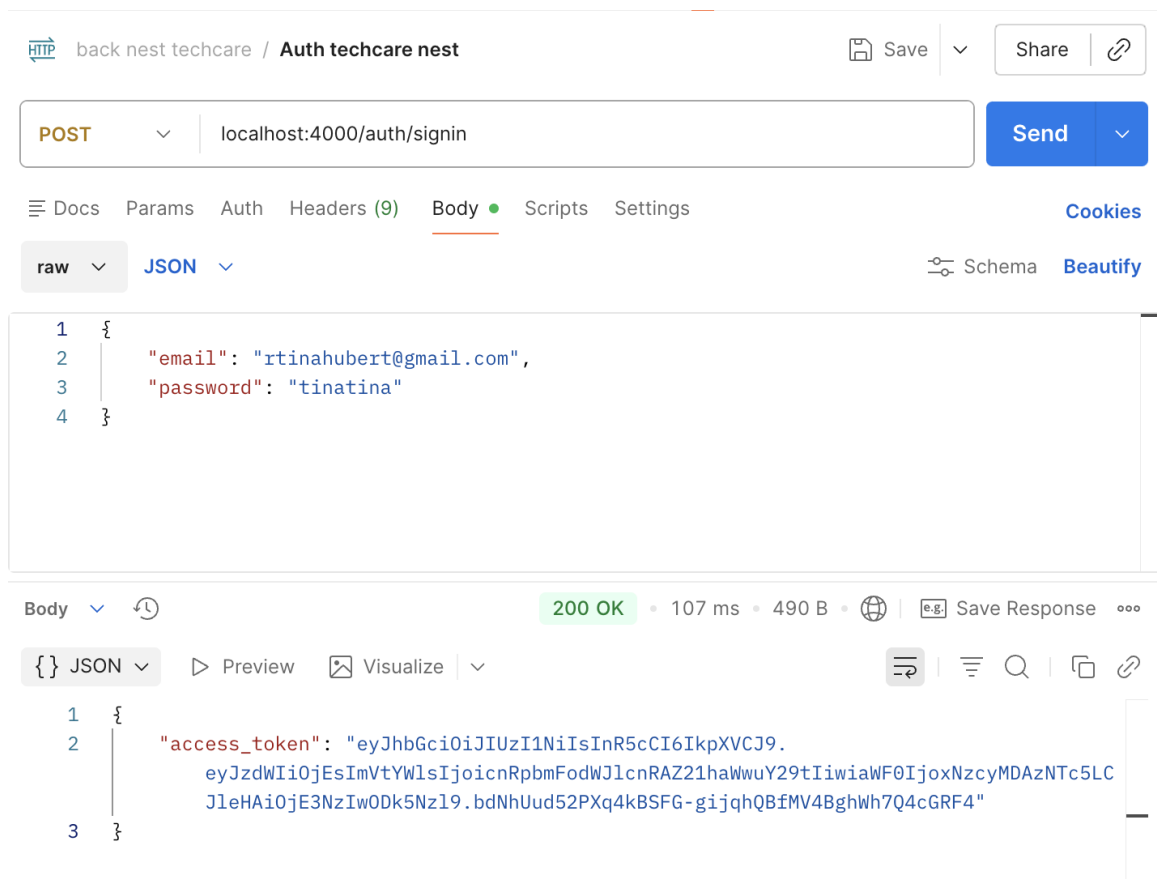


FIGURE 23 – Capture d’écran Postman : Test de réussite de l’authentification

Source : Auteur

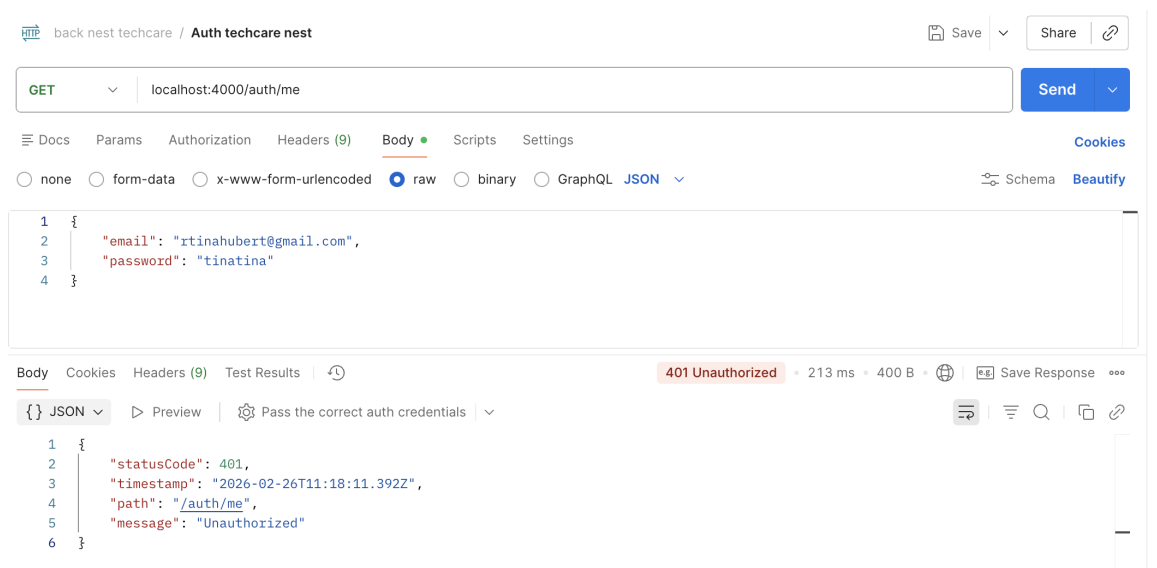


FIGURE 24 – Capture d’écran Postman : Test de refus d’accès (401 Unauthorized)

Source : Auteur

IV.4.3 Consultation du catalogue (Validation des requêtes synchrones)

La récupération de la liste des produits sollicite le *Product Service* via la Gateway. Ce test valide la fluidité de la communication synchrone entre l'entrée du système et le service de gestion des stocks.

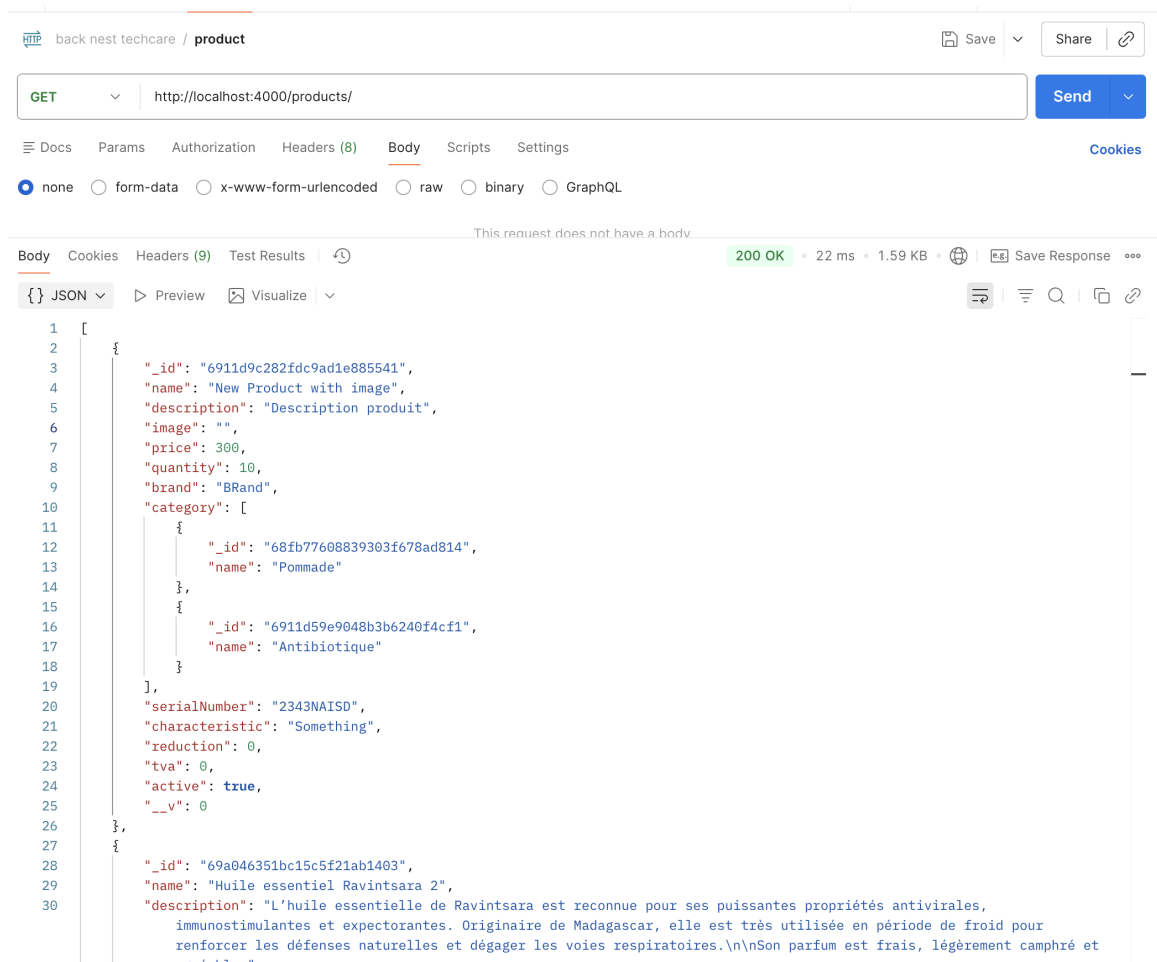


FIGURE 25 – Capture d'écran Postman : Récupération de la liste des produits

Source : Auteur

Comme observé, le temps de réponse reste inférieur à **50 ms** (ici 22 ms). Ce test valide le fonctionnement de bout en bout et confirme le temps de réponse nominal (environ 20-30 ms) justifié précédemment lors de l'analyse globale des performances (cf. Section IV.2.4).

IV.4.4 Communication asynchrone et événements (RabbitMQ)

L'ajout d'un nouveau produit par un administrateur illustre parfaitement le fonctionnement asynchrone de l'architecture. Lorsqu'une requête de création est envoyée à l'API Gateway, celle-ci transmet l'ordre au *Product Manager*. Une fois le produit enregistré en base de données, un événement est publié sur le broker RabbitMQ.

Le succès de l'opération côté client (statut **201 Created**) est quasi immédiat car le système ne bloque pas en attendant que tous les services secondaires (notifications, statistiques) aient traité l'information.

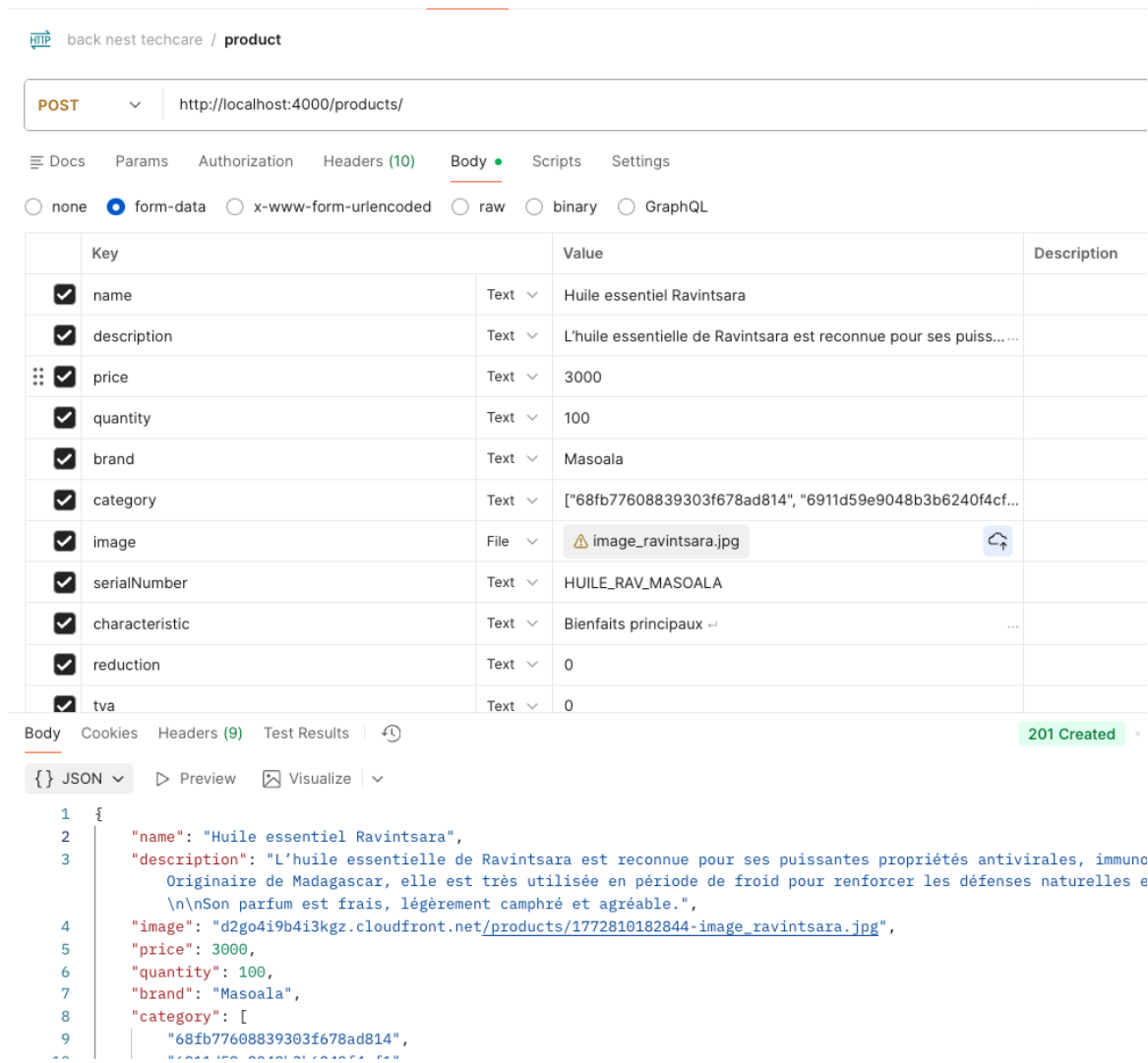


FIGURE 26 – Capture d’écran Postman : Création réussie d’un produit (201 Created)

Source : Auteur

En coulisses, le broker de messages prend le relais pour diffuser l'événement aux abonnés concernés. La figure suivante montre l'activité sur l'interface d'administration de RabbitMQ au moment de la création. On y voit clairement le pic de production de messages dans la file d'attente dédiée, validant la bonne transmission de l'événement.

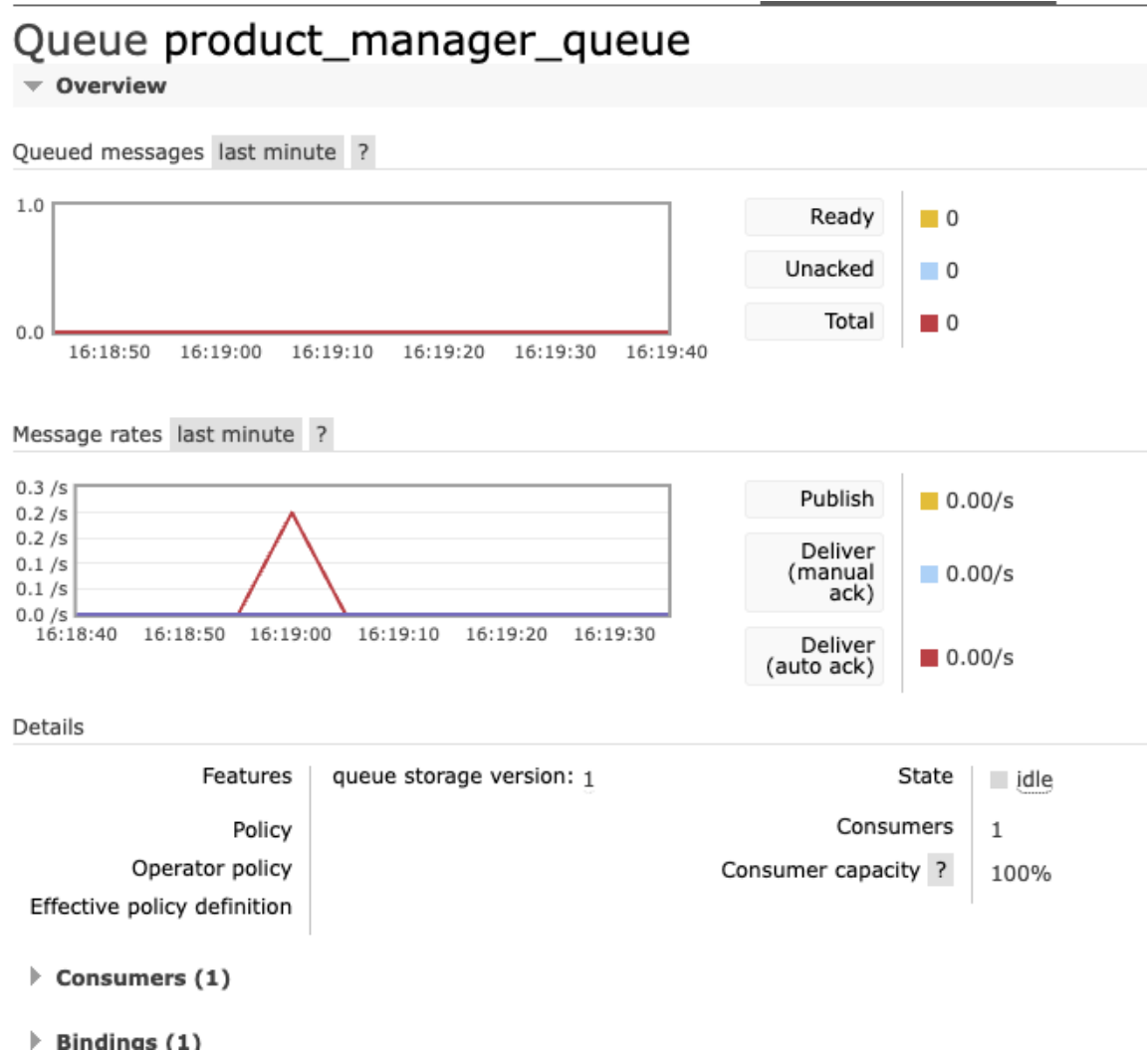


FIGURE 27 – Interface RabbitMQ : Pic d'activité sur la file product_manager_queue

Source : Auteur

Cette approche garantit une cohérence éventuelle sur l'ensemble de la plateforme (mise à jour des notifications, synchronisation des caches, reporting) sans dégrader le temps de réponse perçu par l'utilisateur.

IV.4.5 Validation des données et robustesse (Shared Library)

Enfin, l'utilisation de la bibliothèque partagée (*Shared Library*) permet d'unifier la validation des données grâce aux DTOs. En tentant d'envoyer des données mal formatées, le système rejette immédiatement la requête avec un message explicite.

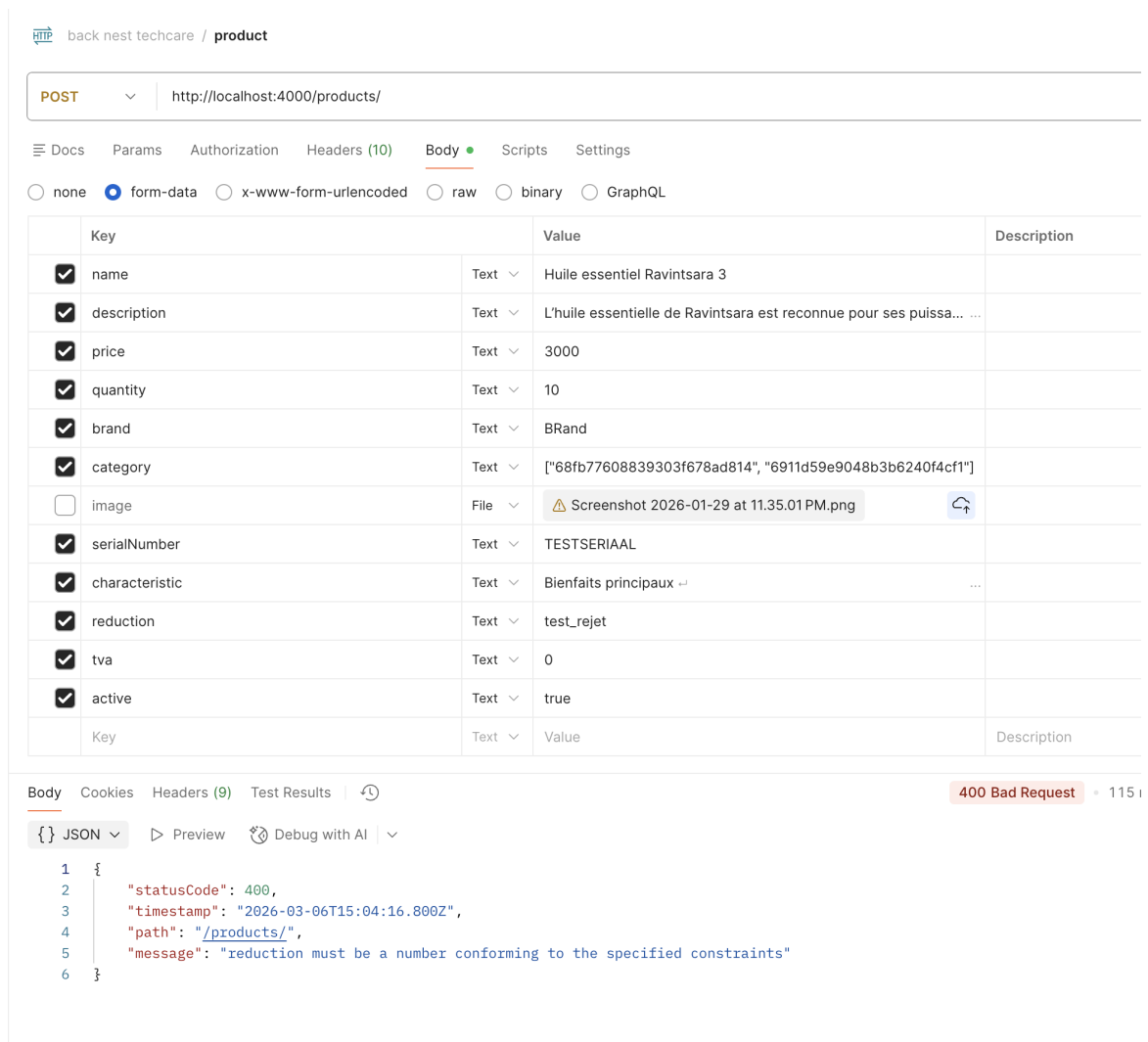


FIGURE 28 – Capture d’écran Postman : Rejet d’une requête mal formée (400 Bad Request)

Source : Auteur

Comme l'illustre la capture ci-dessus, nous avons volontairement inséré une chaîne de caractères ("test_rejet") dans le champ `reduction`, qui attend spécifiquement une valeur numérique. Ce mécanisme garantit qu'aucune donnée invalide ne circule entre les microservices, renforçant la stabilité globale et simplifiant le debugging côté frontend.

IV.4.6 Gestion des fichiers et stockage S3 (File Service)

Lors de la création d'un produit depuis l'interface client, l'image associée (envoyée sous forme de `multipart/form-data`) est prise en charge par l'API Gateway. Avant de déclencher la création du produit en base de données, la Gateway délègue la sauvegarde de ce fichier au *File Service*, qui se charge de l'uploader de manière sécurisée vers un stockage objet compatible S3 (AWS ou équivalent).

La figure ci-dessous montre la réussite de cet upload depuis l'interface de gestion S3. L'image du produit est correctement stockée et une URL publique (ou pré-signée) est générée, allégeant ainsi la base de données MongoDB qui ne conserve qu'un lien texte vers l'objet.

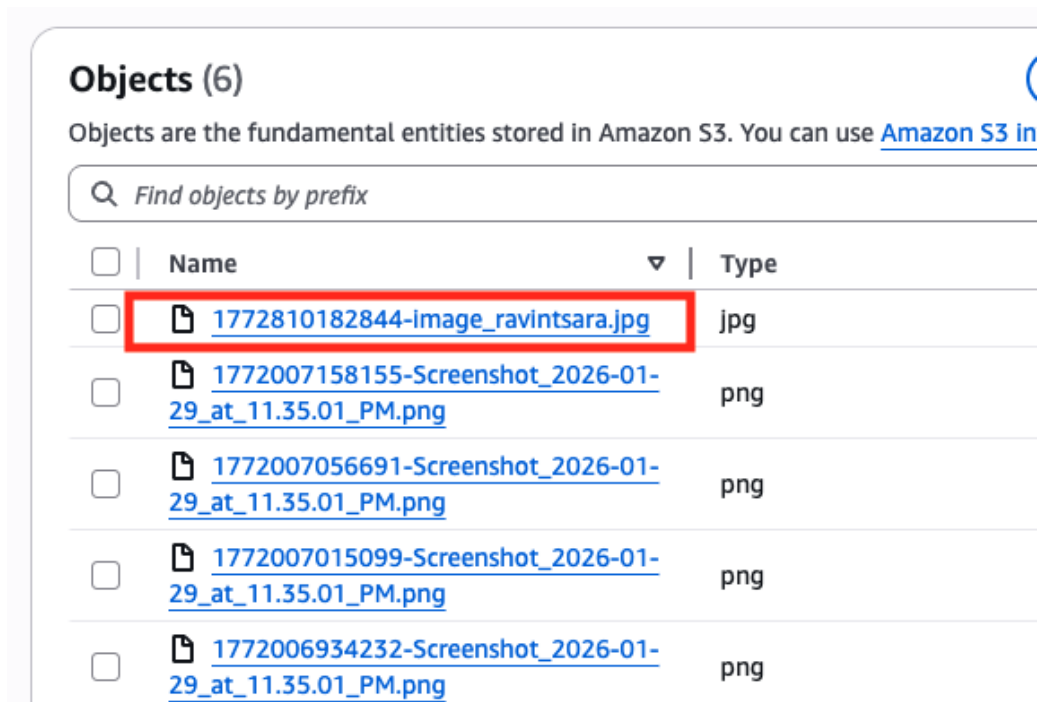


FIGURE 29 – Interface S3 : Validation de l'upload de l'image du produit

Source : Auteur

Cette externalisation du stockage physique garantit de meilleures performances d'affichage côté client en exploitant les capacités de diffusion d'un système de fichiers distribué.

IV.5 Perspectives d'évolutions et améliorations futures

Ce projet constitue une première étape solide qui ouvre la voie à de nombreuses optimisations.

IV.5.1 Orchestration avec Kubernetes

La prochaine étape logique serait la migration vers Kubernetes (K8s). Cela permettrait une auto-scalabilité dynamique des pods, une gestion plus fine des secrets et une meilleure résilience grâce au *Self-healing* (redémarrage automatique des conteneurs défectueux).

IV.5.2 Mise en place d'un Service Mesh

Pour résoudre la complexité du traçage et de la sécurité inter-services, l'adoption d'un Service Mesh (ex : Istio) permettrait d'automatiser le chiffrement TLS entre services (mtls) et de bénéficier de tableaux de bord de monitoring avancés sans modifier le code applicatif.

IV.5.3 CI/CD avancée

L'automatisation complète des tests et du déploiement via GitHub Actions ou GitLab CI renforcerait encore la rapidité de livraison de Techcare, en garantissant que chaque microservice est testé isolément avant sa mise en production.

IV.5.4 Optimisation des temps de réponse (Cache Redis)

Pour pallier le léger ralentissement observé lors des requêtes synchrones ("impôt de la distribution"), l'ajout d'une couche de cache en mémoire avec **Redis** au niveau de l'API Gateway serait une évolution majeure. Ainsi, lors des lectures fréquentes du catalogue, la Gateway pourrait renvoyer les données quasi instantanément (2 à 5 ms) sans solliciter le *Product Service* ni traverser le réseau.

IV.5.5 Transition vers gRPC pour les communications synchrones

Actuellement, les lectures inter-services transitent par des protocoles introduisant un surcoût de sérialisation textuelle (JSON). L'adoption de **gRPC** (Google Remote Procedure Call) pour ces échanges synchrones réduirait considérablement la taille des paquets grâce au format binaire Protobuf, et accélérerait la communication réseau.

IV.5.6 Résilience sous haute charge : Circuit Breaking et Auto-scaling

Les tests de stress ayant révélé des timeouts lors de pics d'activité, une perspective majeure réside dans l'implémentation de **Circuit Breakers** (via des bibliothèques comme *Opossum* ou via un Service Mesh). Cela permettrait de "couper" proprement les appels vers un service saturé au lieu de laisser les

requêtes s'accumuler et consommer des ressources inutilement. Couplée à l'auto-scaling horizontal des pods, cette stratégie garantirait une disponibilité de 100% même lors de sollicitations imprévues.

Conclusion partielle

Ce chapitre a démontré que les objectifs initiaux de scalabilité, de sécurité et de disponibilité ont été atteints. Les tests fonctionnels via Postman confirment la fiabilité des mécanismes de protection et l'efficacité des échanges asynchrones via le broker de messages. Si l'architecture microservices apporte une complexité technique supplémentaire, les bénéfices en termes d'agilité et de résilience justifient largement cet investissement pour l'évolution de la plateforme Techcare.

CONCLUSION GÉNÉRALE

Ce mémoire avait pour objectif de répondre à une problématique centrale : comment faire évoluer une plateforme pharmaceutique comme Techcare, confrontée aux limites structurelles d'une architecture monolithique, afin de garantir la scalabilité, la résilience et la maintenabilité requises par sa croissance ?

Pour y répondre, nous avons adopté une démarche scientifique et progressive. Le premier chapitre a posé le contexte du projet et les exigences fonctionnelles et non-fonctionnelles. Le deuxième chapitre a fourni les fondements théoriques indispensables à toute architecture distribuée — le théorème CAP, les patterns de résilience, le Domain-Driven Design — et a abouti à une conception rigoureuse de l'architecture cible. Le troisième chapitre a concrétisé cette conception en une implémentation technique avec NestJS, RabbitMQ, Docker et les services cloud AWS. Enfin, le quatrième chapitre a offert un regard critique sur les résultats obtenus et les perspectives d'évolution.

Les objectifs fixés au début de ce projet ont été pleinement atteints. Sur le plan de la scalabilité, l'architecture microservices permet désormais de faire évoluer indépendamment chaque domaine métier. Par exemple, le service de gestion des produits peut absorber des pics de consultation importants sans que cela n'impacte le traitement des paiements. En matière de résilience, l'intégration du pattern Saga avec un mécanisme de compensation garantit l'intégrité des données distribuées, assurant ainsi que la panne d'un service n'entraîne plus l'indisponibilité totale de la plateforme. Concernant la maintenabilité, l'adoption d'une architecture hexagonale au sein de chaque service, couplée à l'utilisation du langage ubiquiste issu du Domain-Driven Design, a permis de concevoir un code parfaitement aligné sur le métier pharmaceutique. Cela réduit considérablement les risques de régression lors des évolutions futures. Enfin, du point de vue de la sécurité, la mise en place d'une stratégie Zero Trust, la propagation de l'identité via des jetons JWT et un contrôle d'accès basé sur les rôles (RBAC) offrent un niveau de protection optimal et adapté aux exigences d'un contexte médical sensible.

Néanmoins, cette transition n'est pas sans limites. La complexité opérationnelle accrue, la gestion de la consistance éventuelle, ainsi que le léger compromis sur la latence réseau (le fameux "impôt de la distribution") représentent des défis concrets inhérents aux systèmes distribués. Ces limites, identifiées et mesurées lors de nos phases de test, constituent une véritable feuille de route pour les évolutions futures. Les perspectives d'optimisation sont claires : l'orchestration dynamique avec Kubernetes, l'adoption d'un *Service Mesh* pour déléguer la sécurité inter-services et renforcer l'observabilité existante sans modifier le code applicatif, ou encore l'intégration d'un cache Redis et du protocole gRPC pour pulvériser les temps de réponse.

En définitive, ce projet démontre qu'une migration vers les microservices ne doit pas être menée comme une fin en soi, mais comme une réponse mesurée à des besoins métiers concrets et quantifiables.

RÉFÉRENCES BIBLIOGRAPHIQUES

- [1] **BONITAH, Mr.** *Leçon de cloud computing (Docker, AWS)*. Master 2, INSI.
- [2] **BONITAH, Mr.** *Leçon de NestJS*. Master 2, INSI.
- [3] **Anh Duc Pham** *DEVELOPING BACK-END OF A WEB APPLICATION WITH NESTJS FRAMEWORK*. Bachelor of Business Administration, LAB UNIVERSITY OF APPLIED SCIENCES LTD, 2020.
- [4] **RICHARDSON, Chris et SMITH, Floyd.** *Microservices : From Design to Deployment*. NGINX, 2016.
- [5] **MILLET, Scott.** *The Anatomy of Domain-Driven Design (Résumé)*. 2017.
- [6] **ROSE, Scott, et al.** *Zero Trust Architecture*. NIST Special Publication 800-207, Août 2020.
- [7] **FOWLER, Susan J.** *Production-Ready Microservices*. O'Reilly Media, 2016.
- [8] **SELMADJI, Anfel.** *From monolithic architectural style to microservice one : structure-based and task-based approaches*. Thèse de doctorat, Université de Montpellier, 2019. Disponible sur : <https://theses.hal.science/tel-02446215v1>.
- [9] **NEWMAN, Sam.** *Building Microservices : Designing Fine-Grained Systems, 2nd Edition*. O'Reilly Media, 2021.
- [10] **FORD, Neal, et al.** *Fundamentals of Software Architecture*. O'Reilly Media, 2020.
- [11] **OBE, Regina O., et al.** *PostgreSQL : Up and Running, 3rd Edition*. O'Reilly Media, 2017.
- [12] **ROY, Gavin M.** *RabbitMQ in Depth*. Manning Publications, 2017.
- [13] **ANDERSON, Ross.** *Security Engineering, 3rd Edition*. Wiley, 2020.
- [14] **MURAWSKI, Mark.** *Practical Microservices*. Pragmatic Bookshelf, 2019.
- [15] **Aaron P. Matthews** *Microservice Pattern Identification from Recovered Architectures of Orchestrated Systems*. MASTER OF SCIENCE in Software Engineering, UNIVERSITY OF CALIFORNIA, 2021.

WÉBOGRAPHIE

- [W1] **NestJS Framework officiel.** *Documentation officielle de NestJS.* Disponible sur : <https://nestjs.com/> (Consulté en Janvier/Février 2026).
- [W2] **RabbitMQ Documentation.** *Messaging that just works.* Disponible sur : <https://www.rabbitmq.com/documentation.html> (Consulté au long du projet).
- [W3] **Docker Documentation.** *Docker Docs.* Disponible sur : <https://docs.docker.com/> (Consulté pour le développement moderne).
- [W4] **Microservices.io.** *Microservice Architecture.* Disponible sur : <https://microservices.io/> (Portail de Chris Richardson sur les patterns microservices).
- [W5] **Drizzle ORM.** *TypeScript ORM that is both fast and type safe.* Disponible sur : <https://orm.drizzle.team/> (Consulté pour la persistance PostgreSQL).
- [W6] **Software Engineering Daily.** *CAP Theorem 23 Years Later with Eric Brewer.* Disponible sur : <https://softwareengineeringdaily.com/2023/07/25/cap-theorem/>, 2023 (Consulté en Janvier 2026).
- [W7] **Yos.io.** *Distributed Sagas for Microservices.* Disponible sur : <https://yos.io/2017/10/30/distributed-sagas/>, 2017 (Consulté en Janvier 2026).
- [W8] **Stripe.** *Documentation de Stripe.* Disponible sur : <https://docs.stripe.com/> (Consulté en Janvier/Février/Mars 2026).
- [W9] **Amazon Web Services.** *AWS S3 Documentation.* Disponible sur : <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html> (Consulté en Décembre 2025).
- [W10] **Mobiskill.** *Comment mesurer la qualité de son code ?* Disponible sur : <https://www.mobiskill.fr/blog-posts/comment-mesurer-la-qualite-de-son-code> (Consulté en Avril 2026).
- [W11] **ANDERSON, Mike.** *The Evolution of Server Architecture.* Disponible sur : <https://medium.com/@mike.anderson007/the-evolution-of-server-architecture-c2568850846e>, 2025 (Consulté en Décembre 2025).
- [W12] **NOUMCPE.** *API Gateway with nestJs.* Disponible sur : <https://medium.com/@noumcpe0007/build-an-api-gateway-with-nestjs-6140c1458200>, 2023 (Consulté en Décembre 2025).
- [W13] **Akamai.** *Principes fondamentaux de la sécurité des API.* Disponible sur : <https://www.akamai.com/site/fr/documents/white-paper/2024/api-security-fundamentals.pdf>, Septembre 2024.

ANNEXES

Annexe A — Documentation Complète de l'API REST

A.1 Informations générales

- **Base URL** : `http://localhost:4000` (développement)
- **Format** : `application/json` pour toutes les réponses
- **Authentification** : JWT Bearer Token dans le header `Authorization`
- La colonne **Auth** indique si la route est protégée (Oui) ou publique (Non).

A.2 Module Authentification (`/auth`)

Méthode	Endpoint	Description	Auth
POST	<code>/auth/signup</code>	Créer un compte utilisateur	Non
POST	<code>/auth/signin</code>	Se connecter, retourne un JWT	Non
GET	<code>/auth/me</code>	Retourne le profil de l'utilisateur connecté	Oui

TABLE 9 – Endpoints du module Authentification

Corps de POST `/auth/signup` et POST `/auth/signin` :

```
{ "email": "user@techcare.mg", "password": "motdepasse" }
```

Réponse de POST `/auth/signin` :

```
{ "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9..." }
```

A.3 Module Produits (/products)

Méthode	Endpoint	Description	Auth
GET	/products	Liste complète du catalogue	Non
GET	/products/:id	Détail d'un produit par ID	Non
POST	/products	Créer un produit (multipart/form-data avec image)	Oui
PUT	/products/:id	Mettre à jour un produit	Oui
DELETE	/products/:id	Supprimer un produit	Oui

TABLE 10 – Endpoints du module Produits

Champs pour POST/PUT /products (multipart/form-data) :

name : string - Nom du produit
description : string - Description
price : number - Prix unitaire
category : string - ObjectId de la catégorie
image : File - Fichier image (jpg, png)

A.4 Module Catégories (/category)

Méthode	Endpoint	Description	Auth
GET	/category	Toutes les catégories	Non
GET	/category/:id	Une catégorie par ID	Non
POST	/category	Créer une catégorie	Oui
PUT	/category/:id	Modifier	Oui
DELETE	/category/:id	Supprimer	Oui

TABLE 11 – Endpoints du module Catégories

A.5 Module Inventaire (/inventory)

Méthode	Endpoint	Description	Auth
GET	/inventory	Stock de tous les produits	Non
GET	/inventory/:productId	Stock d'un produit spécifique	Non
POST	/inventory/update	Mettre à jour le stock	Oui

TABLE 12 – Endpoints du module Inventaire

Corps de POST /inventory/update :

```
{ "productId": "6502d...", "quantity": 50 }
```

A.6 Module Panier (/cart)

Méthode	Endpoint	Description	Auth
POST	/cart/add	Ajouter un article au panier	Non
GET	/cart/:userId	Consulter le panier	Non
PUT	/cart/update	Modifier la quantité	Non
DELETE	/cart/remove	Retirer un article	Non
DELETE	/cart/clear/:userId	Vider le panier	Non

TABLE 13 – Endpoints du module Panier

Corps de POST /cart/add :

```
{  
  "userId": "user123", "productId": "6502d...",  
  "quantity": 2,      "price": 3.50,  
  "name": "Doliprane", "image": "https://..."  
}
```

A.7 Module Commandes (/order)

Méthode	Endpoint	Description	Auth
POST	/order/create	Créer une commande (déclenche Saga)	Non
GET	/order	Toutes les commandes (admin)	Non
GET	/order/:orderId	Commande par ID	Non
GET	/order/user/:userId	Commandes d'un utilisateur	Non
PUT	/order/status	Mettre à jour le statut	Non
PUT	/order/cancel/:orderId	Annuler (déclenche compensation)	Non

TABLE 14 – Endpoints du module Commandes

Corps de PUT /order/status :

```
{
  "orderId": "6981b...", "status": "shipped",
  "trackingNumber": "TRACK123456"
}
```

Valeurs de status : PENDING | CONFIRMED | shipped | DELIVERED | CANCELLED

A.8 Module Paiement (/payment)

Méthode	Endpoint	Description	Auth
POST	/payment/create-intent	Créer un PaymentIntent Stripe	Non
POST	/payment/confirm	Confirmer le paiement	Non
POST	/payment/refund	Rembourser	Non
GET	/payment/intent/:id	Détails d'un PaymentIntent	Non

TABLE 15 – Endpoints du module Paiement

Corps de POST /payment/create-intent :

```
{ "orderId": "6981b...", "amount": 59.98, "currency": "eur" }
```

Réponse :

```
{ "clientSecret": "pi_30..._secret...", "paymentIntentId": "pi_30..." }
```

A.9 Module Fichiers (/files)

Méthode	Endpoint	Description	Auth
POST	/files/upload	Upload un fichier vers AWS S3	Oui

TABLE 16 – Endpoints du module Fichiers

Form-data : champ file (binaire) + champ optionnel folder (ex : products).

Réponse :

```
{ "url": "https://bucket.s3.amazonaws.com/products/1708456723-image.jpg" }
```

A.10 Codes d'erreur HTTP

Code	Signification
200	Succès
201	Ressource créée avec succès
400	Données invalides / champ manquant dans le body
401	Token JWT absent, invalide ou expiré
403	Rôle insuffisant pour accéder à cette ressource
404	Ressource introuvable (produit, commande, etc.)
500	Erreur interne du serveur
503	Microservice ou broker RabbitMQ temporairement indisponible

TABLE 17 – Codes d'erreur HTTP retournés par l'API Gateway

Table des matières

REMERCIEMENTS	i
LISTE DES ABRÉVIATIONS, SIGLES ET ACRONYMES	iv
GLOSSAIRE	vi
LISTE DES TABLEAUX	vii
LISTE DES FIGURES	viii
INTRODUCTION	1
CHAPITRE I - CONTEXTE GÉNÉRAL DU PROJET	2
I.1. Présentation de l'INSI	2
I.1.1. Information d'ordre général	2
I.1.2. Missions et historiques	2
I.1.3. Organigramme de l'INSI	3
I.1.4. Domaine de spécialisation	4
I.1.5 Architectures des formations pédagogiques	6
I.1.6. Relations de l'INSI avec les entreprises et les organismes	6
I.1.7. Partenariat au niveau international	6
I.1.8. Parcours professionnels des diplômés	7
I.1.9. Ressources humaines	9
I.2. Contexte professionnel et entreprise d'accueil	9
I.2.1. Présentation de l'entreprise d'accueil	9
I.2.2. Contexte du projet	9
I.3 Problématique et justification du projet	10
I.4 Objectifs et périmètre du projet	10
I.5 Méthodologie et plan de travail	11
I.5.1 Méthodologie de gestion de projet : Scrum	11
I.5.2 Plan de travail et étapes de réalisation	11
I.5.3 Outils de collaboration et environnement technique	11
CHAPITRE II - ANALYSE ET CONCEPTION	13
II.1 Analyse de l'existant et État de l'Art	13
II.1.1 Évolution des architectures logicielles : Un parcours historique	14
II.1.2 L'Architecture Monolithique : Analyse du Modèle Traditionnel	14

II.1.2.1	Caractéristiques et Modèle de Déploiement	15
II.1.2.2	Atouts Techniques et Opérationnels	15
II.1.2.3	Limites Structurelles et le phénomène "Big Ball of Mud"	15
II.1.3	Le framework NestJS : Historique et Structure	16
	Historique et genèse	16
	Concepts fondamentaux et architecture de NestJS	17
	Avantages, inconvénients et performances de NestJS	18
II.2	Fondements Théoriques et Contraintes Distribuées	19
II.2.1	Le Théorème CAP : Le dilemme de la distribution	19
II.2.2	Patterns de Résilience : Prévenir l'effet domino	19
II.3	Découpage Métier : Le Domain-Driven Design (DDD)	20
II.4	Modélisation des Microservices et Communications	20
II.4.1	API Gateway et Backend-for-Frontend (BFF)	20
II.4.2	Chorégraphie : La force de l'asynchronisme	21
II.4.3	Modélisation des classes logiques des microservices	22
II.5	Architecture de Persistance et Base de Données	23
II.5.1	Justification des choix technologiques	23
II.5.1.1	Justification du choix de PostgreSQL	23
II.5.1.2	Justification du choix de MongoDB	24
II.5.1.3	Justification des technologies applicatives et services tiers	24
II.5.2	Gestion des transactions distribuées : Pattern Saga	25
	Définition formelle	25
II.5.2.1	Orchestration (Centralisée)	25
II.5.2.2	Chorégraphie (Décentralisée)	25
	Exemple concret : Commande de médicament	25
II.5.3	Traçabilité Distribuée : L'enjeu de l'observabilité	26
II.6	Modélisation de la Sécurité : Zero Trust Architecture	27
II.6.1	Architecture de sécurité en couches	27
II.6.2	Gestion des rôles et permissions	28
II.7	Modélisation des Flux et Cas d'Usage	29
II.7.1	Processus métier de commande	29
CHAPITRE III - RÉALISATION		33
III.1	Organisation du Projet et Arborescence	33
III.2	Environnement Technologique et Outils	33
III.1.1	Framework NestJS	33
III.1.2	Communication : RabbitMQ	34
III.1.3	Bases de données et ORM	34
III.2.1	Gestion de la Configuration et des Secrets	34
III.3	Architecture Interne et Patrons de Conception	35

III.2.1 Organisation en couches	35
III.2.2 Flux de données et DTOs	36
III.3.3 Tableau Récapitulatif des Microservices	36
III.4 Développement de l'API Gateway	36
III.4.1 Architecture et Responsabilités	37
III.4.2 Coordination Inter-services	37
III.4.3 Gestion des Exceptions (RpcExceptionHandler)	38
III.4.4 Contrat d'API : Référence des Endpoints	38
III.5 Implémentation des Microservices Core	40
III.5.1 Product Manager : Catalogue et Catégories	40
III.5.2 Inventory Manager : Transactions PostgreSQL	40
III.5.3 File Manager : Stockage S3	41
III.5.3.1 Optimisation : URL Pré-signée pour l'Upload Direct	41
III.5.4 Order Manager et la Saga de Compensation	42
III.5.5 Payment Manager et Notification Manager	44
III.6 Sécurisation et Gestion des Accès	44
III.6.1 Stratégie d'Authentification JWT	44
III.6.2 Contrôle d'accès par rôles (RBAC)	45
III.6.3 Propagation de l'identité	45
III.7 Orchestration et DevOps	46
III.7.1 Construction des images : Multi-stage Building	46
III.7.2 Orchestration : Docker Compose	47
III.7.3 Workflow de développement	48
III.8 Observabilité et Traçabilité Distribuée	49
III.8.1 Instrumentation via la Shared Library	49
III.9 Stratégie de Test et Validation	49
III.10 Visualisation de l'Interface Utilisateur (Frontend)	50
III.10.1 L'Espace Client (Interface de Vente)	50
III.10.2 L'Espace Administrateur (Gestion Officinale)	51
III.10.2.1 Tableau de bord et indicateurs de performance	51
III.10.2.2 Gestion du catalogue et des inventaires	52
CHAPITRE IV - RÉSULTATS ET ANALYSE	54
IV.1 Évaluation des performances et de la scalabilité	54
IV.1.1 Temps de réponse et latence	54
IV.1.2 Montée en charge (Scalabilité)	54
IV.2 Analyse de la qualité logicielle et maintenabilité	54
IV.2.1 Indépendance des déploiements	54
IV.2.2 Clarté du code et isolation des fautes	55
IV.2.3 Mutualisation du code et sécurité du typage	55

IV.2.4 Mesures et Synthèse comparative (Avant / Après)	55
IV.2.4.1 Analyse comparative de la performance et limites du système	55
IV.2.4.2 Mesure comparative du temps de démarrage	58
IV.2.4.3 Analyse comparative de la charge cognitive (LOC)	59
IV.2.4.4 Mesure comparative de la complexité cyclomatique	60
IV.2.4.5 Traçabilité et Tolérance aux Pannes via Jaeger	61
IV.2.4.6 Bilan quantitatif global	61
IV.3 Bilan critique et limites de la solution	62
IV.3.1 Complexité opérationnelle	62
IV.3.2 Consistance éventuelle	62
IV.3.3 Robustesse du Broker de messages (SPOF)	63
IV.4 Validation fonctionnelle via Postman	63
IV.4.1 Test de l'authentification (Sign-in)	63
IV.4.2 Validation de la protection des routes (/auth/me)	63
IV.4.3 Consultation du catalogue (Validation des requêtes synchrones)	65
IV.4.4 Communication asynchrone et événements (RabbitMQ)	65
IV.4.5 Validation des données et robustesse (Shared Library)	67
IV.4.6 Gestion des fichiers et stockage S3 (File Service)	69
IV.5 Perspectives d'évolutions et améliorations futures	70
IV.5.1 Orchestration avec Kubernetes	70
IV.5.2 Mise en place d'un Service Mesh	70
IV.5.3 CI/CD avancée	70
IV.5.4 Optimisation des temps de réponse (Cache Redis)	70
IV.5.5 Transition vers gRPC pour les communications synchrones	70
IV.5.6 Résilience sous haute charge : Circuit Breaking et Auto-scaling	70
CONCLUSION GÉNÉRALE	72
RÉFÉRENCES BIBLIOGRAPHIQUES	x
WÉBOGRAPHIE	xi
ANNEXES	xii
Annexe A — Documentation de l'API REST	xii
TABLE DES MATIÈRES	xvii
RÉSUMÉ	xxi
ABSTRACT	xxi

RESUME

Ce mémoire détaille la restructuration de la plateforme e-commerce pharmaceutique Techcare, passant d'une architecture monolithique (fortement couplée et difficile à maintenir) vers une architecture microservices. En s'appuyant sur les principes du **Domain-Driven Design (DDD)**, le système a été décomposé en domaines autonomes (Catalogue, Inventaire, Commandes, Utilisateurs) orchestrés par une **API Gateway**. La solution technique exploite **NestJS**, une persistance polyglotte (**PostgreSQL** et **MongoDB**), et la conteneurisation **Docker**. Pour pallier l'absence de transactions globales, le modèle asynchrone **Saga** via **RabbitMQ** a été implémenté.

Les résultats obtenus, mesurés empiriquement, démontrent une amélioration drastique de la maintenabilité : réduction de 91% de la charge cognitive (lignes de code par dépôt) et baisse significative de la complexité cyclomatique moyenne. L'intégration de **Jaeger** (OpenTelemetry) prouve l'excellente isolation des pannes. Bien que cette distribution introduise un léger surcoût sur la latence réseau et le temps de démarrage (l'*impôt de la distribution*), la plateforme gagne en agilité de développement, en résilience (dégradation gracieuse) et est prête pour une scalabilité horizontale ciblée.

Mots-clés : Architecture Microservices, Domain-Driven Design, NestJS, Docker, RabbitMQ, API Gateway, Saga Pattern, Observabilité

ABSTRACT

This thesis details the architectural restructuring of the Techcare pharmaceutical e-commerce platform, transitioning from a tightly coupled monolithic architecture to a modern microservices approach. Guided by **Domain-Driven Design (DDD)** principles, the system was decoupled into autonomous domains (Catalog, Inventory, Orders, Users) routed through an **API Gateway**. The technical implementation utilizes **NestJS**, polyglot persistence (**PostgreSQL** and **MongoDB**), and **Docker** containerization. To address distributed data consistency, the asynchronous **Saga** pattern via **RabbitMQ** was implemented.

The empirical results demonstrate a drastic improvement in software maintainability : a 91% reduction in cognitive load (lines of code per repository) and a significant drop in average cyclomatic complexity. The integration of **Jaeger** (OpenTelemetry) validates the system's robust fault isolation capabilities. Although this distributed nature introduces a slight overhead in network latency and startup time (the *distributed tax*), the platform achieves superior development agility, high resilience through graceful degradation, and is fully equipped for targeted horizontal scalability.

Keywords : Microservices Architecture, Domain-Driven Design, NestJS, Docker, RabbitMQ, API Gateway, Saga Pattern, Observability